

Lecture notes of 6.484 Computational Sensorimotor Learning

MIT

Zhang-Wei Hong, Pulkit Agrawal

Thanks to (especially) Abhishek Gupta, Daniel Yang and other members of the Improbable AI Group for discussions that have substantially influenced and shaped these lecture notes.

Contents

1	Introduction	3
2	Simple Decision Making	7
2.1	Multi Arm Bandit (MAB)	7
2.1.1	Explore-then-Commit (ETC)	9
2.1.2	Upper Confidence Bound (UCB)	10
2.2	Contextual Bandit (CB)	12
2.2.1	LinUCB	13
3	Sequential Decision Making	15
3.1	Issues of CB	15
3.2	Sequential Decision Process	16
3.3	Finite and Infinite Horizon	17
3.4	Trajectory and Episode	18
3.5	Summary	19
4	Policy Gradient	19
4.1	Derivation and Properties of Policy Gradient	20
4.2	REINFORCE	21
4.3	Credit Assignment	22
4.4	Actor-critic	23
4.4.1	Advantage Actor-Critic (A2C)	24
4.4.2	Generalized Advantage Estimation (GAE)	25
4.5	Exploration and Data Diversity	26
4.6	Conservative Policy Optimization	26
4.6.1	Parameter space constraint	27
4.6.2	Parameterization-independent constraints on output space	29
4.6.3	Monotonic Policy Improvement	30
4.7	Practical Considerations	33
4.7.1	Code level optimization	33

4.7.2	Neural network architecture, Reward scale, and Random seed	34
4.8	Rules of thumb in practice	34
4.9	Debugging	34
5	Reward Function Design	34
5.1	Challenges of Reward Function Design	35
5.2	Reward Engineering for Multiple Objectives	37
5.3	Principles of Reward Shaping	37
6	Learning from Demonstrations	38
6.1	Behavior cloning	39
6.2	Improving over suboptimal demonstration	43
6.2.1	Pre-training the policy with BC	44
6.2.2	Combining BC and policy gradient in policy optimization	44
6.2.3	Summary	45
6.3	Copycat problem	45
6.4	Causal confusion and spurious features	46
6.5	Multimodality problem	47
6.5.1	Mixture of Gaussian (MoG)	47
6.5.2	Discretized Autoregressive models	48
6.5.3	Latent variable models	48
6.5.4	Implicit models	49
6.6	Beyond task specific demonstration	49
6.7	Inverse reinforcement learning (draft)	50
6.7.1	Max margin inverse reinforcement learning	51
6.7.2	Maximum entropy inverse reinforcement learning	53
6.7.3	Generative adversarial imitation learning	57
7	Learning from simulator	57
7.1	Domain randomization	58
7.1.1	Performance-robustness Trade-off	58

7.1.2	Advanced Domain Randomization	59
7.2	System identification and domain randomization	60
7.2.1	Training system parameter conditioned policy	60
7.2.2	System identification	60
8	Model-based control (in progress)	62
8.1	Model Predictive Control	62
8.1.1	Cross entropy method (CEM)	62
8.2	Inverse model approach	64
9	Dynamic Programming Method (draft)	64
9.1	Policy iteration & Value Iteration	65
9.1.1	Policy iteration	65
9.1.2	Value iteration	67
9.1.3	Comparison between policy iteration and value iteration	67
9.2	Q-Learning	68
9.2.1	Tabular Q-Learning	68
9.2.2	Fitted Q iteration	69
9.2.3	Deep Q Network (DQN)	70
9.2.4	Deep deterministic policy gradient	73
9.2.5	Soft actor critic	75
10	Offline reinforcement learning	76
10.1	Overcoming distributional shift in offline RL	76
10.2	Conservative Q-Learning	76
11	Multi-goal Reinforcement Learning	76
11.1	Goal-conditioned value function	76
12	Hierarchical Reinforcement Learning	76
13	Curriculum learning	77

Lecture notes of Machine Learning for Sequential Decision-making	2
A Supervised Learning v.s. Reinforcement Learning	82
B Second-order KL-divergence Approximation	82
C Behavior cloning derivation	83
D Policy Parameterization	84
E Consideration of Markov assumption	85
F Consideration of action space design	86

1 Introduction

Consider the problem of designing a recommendation system for a competitor to the popular music service *Spotify* called *MusicApp*. To maximize earnings, the app’s goal is not just to have a user enjoy a song once but to return to *MusicApp* repeatedly (i.e., long-term engagement). Whenever the user returns to the app, *MusicApp* must decide what song to recommend. These recommendation decisions are taken to minimize “regret”, which in this example refers to the loss in earnings (i.e., performance) resulting from sub-optimal decisions accumulated over the entire duration a user avails the music service. Such scenarios necessitating a sequence of decisions to minimize regret are referred to as *sequential decision making* (SDM) problems.

The entity making decisions (or “actions”) is referred to as the “agent”, which is *MusicApp* in our example. The entity effected by actions is called the “environment”, a dynamical system described by “state” variables. In our recommendation system example, “environment” refers to the user since the recommendation made by *MusicApp* could promote users to buy more songs. State corresponds to the mental state of the user, which is not directly accessible. The environment’s state may only be partially observable to the agent and is changed by the action. However, the state can be guessed based on observables (also called “features” in machine learning) such as the user demographics, history of liked songs, time of the day, and other information that *MusicApp* might have access to. Based on the “imperfect” and possibly noisy guess of the state inferred from observables, *MusicApp* must make decisions. The decisions, in turn effect the environment. For instance, listening to the music recommendation will likely change the user’s mental state. The amount of time user spends on the song, the frequency of returning of app, whether he/she purchases a subscription or, in other cases, pays for the song are indicators of user’s interest in the recommendation and correlate either directly or indirectly with earnings of *MusicApp*. Such feedback (also called *reward* or *cost*) is to be utilized to judge the effectiveness of the decision and update the strategy for making future decisions.

The function that determines the actions based on past observations is referred to as the policy: $a_t = \pi(o_{1:t}, a_{1:t-1})$. The central aim of sequential decision-making (SDM) is to determine the policy π . The problem of determining π has been investigated in many fields with many different names, like optimal control, feedback control, non-linear control, trajectory optimization, model predictive control, reinforcement learning, imitation learning, planning languages, task-and-motion planning, etc. These different lines of study primarily differ in the assumptions under which the policy is determined. Many approaches assume knowledge of the dynamics model and reward function, in which case a good policy can be determined by leveraging optimization methods. An example is the well-studied *Linear Quadratic Regulator* which corresponds to a convex optimization problem and can, therefore, be efficiently solved to obtain the *optimal* policy. Methods that rely on the knowledge of the dynamics model to obtain a policy (also called controller) are referred to as *model-based*. Here, the agent can obtain a policy without any “real” interactions with the environment as the effect of actions and subsequent rewards can be simulated. While various optimality criteria are of interest, from a practical perspective what matters is whether a method is able to obtain the optimal policy and the run time (or the wall-clock time) of the optimization procedure.

In a more general scenario, the dynamics and the reward models are both unknown. For

instance, in the *MusicApp* example, how the mental state of the user changes a function of music recommendations and the interest of a user in a particular song are unknown. Similarly, if one is interested in devising a policy for investing in the stock market, the dynamics of stock prices in the future are unknown. In such scenarios, the agent must estimate the utility of different action sequences by executing actions in reality which can quickly become prohibitive. The criterion to judge optimality is the overall regret which subsumes two factors: whether an optimal policy is found and how many actions were required to determine this policy. Minimizing the number of interactions is of paramount interest: for instance, if millions of interactions are required to determine a good policy, a robotic system might have to run for months to collect data. In other scenarios such as healthcare, collecting large data where a substantial fraction of decisions are sub-optimal and were taken in the process of figuring out better decisions might be unacceptable.

Which formulation to use is highly scenario dependent. While our primary focus is on learning methods for policy determination without knowledge of environment dynamics and access to the functional form of the reward function, we hope to provide readers with the understanding of trade-offs between different approaches. We will describe other approaches to the extent necessary to understand the trade-offs.

There are many considerations and aspects of the problem that are worth considering:

- Bandits and Contextual Bandits** Bandits present the simplest instance of decision making where the environment is *stateless*. The bandit problem is akin to playing with a slot machine to identify the best slot (or the arm). The environment returns a reward after an action is taken. The rewards are assumed to be drawn from a random distribution unknown to the agent. A simplified version of *MusicApp* problem fits the Bandits setup: *MusicApp* has to recommend the best song but cannot make use of any information about the user (i.e., the exact prediction for all the users) and the user interest is assumed to be the same at all times. Consider another example: a pharmaceutical company has multiple drug candidates and is interested in finding the best drug while testing on minimum number of people. Over here, each drug constitutes an arm and its efficacy on patients would be the reward function. An optimal algorithm in such a scenario is the one that can identify the best action in as few interactions as possible. Luckily, approximately optimal algorithms are known in such a setup (e.g., Upper Confidence Bound or the UCB algorithm) that we will discuss. A more general setup is where for instance, in the *MusicApp* example, user features (or context) can be used for decision making, but the user's mental state does not change with time (i.e., no dynamics). Such a setup is referred to as *Contextual Bandits* for which approximately optimal algorithms are also known. We will discuss the definition of optimality (*worst-case regret criterion*), a proof-sketch for deriving lower-bound on regret and practical algorithms for Bandits and Contextual Bandits that approximately achieve the lower-bound. The exposition to Bandits largely elucidates the exploration-exploitation dilemma and when it can be optimally solved. A detailed discussion of Bandit algorithms is beyond the scope of this work, and other excellent sources provide a thorough coverage [Lattimore and Szepesvári, 2020].
- Policy Learning:** We will consider the scenario wherein the policy is a learnable function, π_θ , with parameters θ . The most general problem setup is where the agent is required to learn the policy from scratch (i.e, random initialization of parameters

θ) without any knowledge of the environment dynamics and without access to the functional form of the reward function. Furthermore, the environment dynamics, the reward function and the policy itself might be stochastic for various reasons that we will discuss. In this most general scenario, optimal decision-making algorithms are unknown, yet impressive empirical success has been achieved [Abbeel and Ng, 2004, Mnih et al., 2015, Vinyals et al., 2019][Cite LinUCB, ContextualBandits, AlphaGo, Nuclear Fusion, Tao’s Paper, Gabe’s Paper, Marco Hutter’s Paper]. We will describe such algorithms and cover special cases where optimal algorithms are known (e.g., Bandits, Contextual Bandits, Linear Quadratic Regulator, etc.).

- **Task Specification:** We discussed that the goal of policy learning to minimize regret. However, designing and measuring a reward function in many scenarios is challenging. Consider building self-driving cars: designing a reward function that captures safe driving can be more demanding than simply providing demonstrations of safely driving the car. Depending on the scenario, different ways of specifying the task: via reward function, demonstration, configuration of the desired state, or language instructions, etc. might be more appropriate [Agrawal, 2018]. While each task-specification can be reduced to a reward function, different task-specifications afford different algorithms that might perform better in some scenarios. We will discuss the pros/cons of various task-specification and algorithms for decision making suited to different task specifications.
- **Exploration-Exploitation Dilemma:** If *MusicApp* always suggest the user’s current favorite song (i.e., “exploitation”), the user might get bored after some time and not return to the app. To maximize user retention, it might be worthwhile to make “exploratory” recommendations that diversify the music portfolio. One reason for diversification is that it might increase user engagement and thereby reduce regret. The other reason is that it enables *MusicApp* to obtain more information about the user (i.e., a better idea of the user’s mental state) that aids future decision making. However, such *exploration* comes at a cost: one might make song recommendations that the user may not like at all, resulting in *regret*. The agent is therefore faced with a dilemma, should it *exploit* the current best decision or *explore* more decisions in the hope of finding better decisions? This *exploration-exploitation dilemma* is at the core of sequential decision making: both too little and too much exploration can increase regret. In certain scenarios (e.g., Bandits / Contextual Bandits), it is possible to (almost) optimally resolve the dilemma; in many other scenarios, optimal algorithms are unknown. We will look at this topic in great detail.

- **Model-based Planning**

Furthermore, Without additional assumptions, a naive solution is to execute the policy for T time steps to obtain the reward,

How to communicate the task. On-Policy, Off-Policy Learning Offline Learning Learn from demos / Imitation Learning Model based learning Reward. design – Setting up the problem Self-Supervised Decision Making: Inverse Models, Forward Models, Curiosity.

are multiple scenarios the agent might be faced with: learning the policy from scratch, or it might have access to a dataset of previous decisions, or it might have access to a sub-optimal policy, or knowledge of the environment dynamics or the functional form of reward function.

-
- **Online and Offline Learning:** Learning policy requires users' response data but deploying a system with an untrained policy is risky. One solution is to learn from historical data generated by a heuristic recommendation policy and improves over it.
- How to deal with new users and songs?
- Distinction between Generalization and solving one instance of a problem.
-
- Sequential Decision Making Problem – need to make a sequence of decisions, e.g., what website layout to show. For a doctor, a sequence of diagnoses / tests / medications. What prior knowledge /data might the agent have (learning from scratch / existing dataset that we can improve over / experience solving similar problems) gather information while attempting to make good predictions for every state. Actions reveal information about the environment and can also change it.
- RL is not the problem, but RL is solution to a problem. One can make progress in SDM without invoking RL – e.g., decision transformers, decision diffusers, etc.
- When one is searching for a solution, there are many options: perform monte-carlo rollouts and hope to find a solution. Hopeless in high-dimensional spaces, yet still seems to work. Central to the assumptions are questions about variance reduction, avoiding a local minima.

It is possible that the app company is just starting and has no data about user preferences. In such a case, to at

Sequential decision making refers to a class problems where an actor (also referred to as the agent) needs to make a sequence of choices to minimize regret. In loose terms, regret measures the quality of choices compared to an oracle that makes the optimal choice

2 Simple Decision Making

A central challenge in reward-based learning is in deciding between actions that exploit the presently known source of reward or those that explore. The questions of interest are:

- Does there exist an optimal algorithm that can balance exploration and exploitation?
- If **Yes**, what is this algorithm?

In this chapter, we answer these questions. First we introduce the simple setup of multi-arm bandits (MAB) 2.1, followed by decision making algorithms for MAB (Sections 2.1.1, 2.1.2). Next, we will focus on contextual bandits (CB) in Section 2.2.

2.1 Multi Arm Bandit (MAB)

MAB is the simplest reward-based learning problem where the agent is required to choose one out of K possible actions. The MAB setup assumes that rewards are independent of the *state* and the history of actions. The reward only depends on the current action, $r(a_t) = r(a_{1:t}, s_{1:t})$. The goal is construct a policy that maximizes the total payoff over a horizon of T time steps, $\sum_{t=1}^T r_t(a_t)$. Because we are maximizing the sum of rewards, not only is it important that the agent eventually discovers the most rewarding action (i.e., asymptotic optimality), but also how quickly it discovers the optimal action. As Figure 1b shows, the convergence speed of two asymptotically optimal algorithms can be very different.

The Landing Page Example: To be more concrete, let's consider a MAB example (illustrated in Figure 1a) of recommending landing pages with the goal of maximizing the total profit. Whenever a user visits the website, our algorithm selects out of K possible choices of the landing page showing different ordering of products to be sold. Given a displayed page, the user makes purchases which makes profit for the website. Our goal is to build a recommendation systems that displays the landing page maximizing the profit. Since we have no apriori knowledge of which landing page maximizes sales, each page needs to be displayed multiple times to calculate the average profit earned per page. This knowledge can utilized to choose pages that maximize profit in the future. We can formalize this problem as following:

- **Horizon** T : Total number of users visiting the website.
- **Round** t : $t = i$ denotes the i^{th} user visiting the website.
- **Environment**: The website.
- **Agent**: The recommendation system.
- **Action space** $\mathcal{A}$: The available landing pages on the website. $\mathcal{A} := \{1, \dots, K\}$, where K denotes the number of pages and $k \in [1, K]$ corresponds to the action of recommending i -th landing page.
- **Action** a_t : The choice of machine at round t , where $a_t \in \mathcal{A}$, $\forall t \in [1, T]$.
- **Reward** $r(a_t) \in \mathbb{R}$: Profit earned by displaying the page a_t . We assume that $r(a_t)$ is a random variable from an unknown distribution.

- **Action counts** $T_k(t)$: The number of time that action k has been taken within t rounds.
- **Mean reward** μ_k : True mean reward of taking action k . The optimal mean reward is defined as $\mu^* = \max_k \mu_k$.
- **Empirical mean reward** $\hat{\mu}_k(t)$: Empirical mean reward of taking action k , estimated by our algorithm over t rounds. It is defined as

$$\hat{\mu}_k(t) := \frac{1}{T_k(t)} \sum_{t=1}^T \mathbb{I}\{a_t = k\} r(a_t),$$

where $\mathbb{I}\{a_t = k\}$ is a binary indicator that equals to 1 if $a_t = k$ and 0 otherwise.

In this formalism, the problem of maximizing the total profit can be expressed as:

$$\max_{a_1 \dots a_T} \mathbb{E} \left[\sum_{t=1}^T r(a_t) \right], \quad (1)$$

where the objective is the expected total rewards over days.

Before we start introducing MAB algorithms that solve the above reward maximization problem, let's think of two problems: (i) how to design such an algorithm and (ii) how to evaluate the performance of an algorithm. The short answer is: *balance exploration and exploitation such that the regret of taking suboptimal actions is minimized.*

Recall that in the landing page example, our system experiments with landing page recommendations to maximize profits. However, in this experimentation phase not all actions will result in high rewards – i.e., some choices of the landing page can yield a low profit. However, without displaying all pages the agent cannot know which pages are good or bad. As such, we must balance exploration and exploitation to smartly allocate time on experimenting with the user responses on a web page (i.e., “*exploration*”) and time on recommending the users' favorite page (i.e., “*exploitation*”). To disambiguate exploration and exploitation, we make the concept of exploration and exploitation slightly more concrete below.

Definition 2.1 (*Exploration*). We define any action selection a such that

$$\hat{\mu}_a(t) \neq \max_k \hat{\mu}_k(t)$$

as exploration. Note that this definition says that the empirical mean reward of the selected action a can not be higher than the highest empirical mean reward, instead of not being higher than the highest true mean of reward $\max_k \mu_k$.

Definition 2.2 (*Exploitation*). Conversely, any action selection a such that

$$\hat{\mu}_a(t) = \max_k \hat{\mu}_k(t)$$

is defined as exploitation.

We can formalize these definitions mathematically. Initially, empirical mean rewards are inaccurate $\hat{\mu}_a \neq \mu_a \forall a \in \mathcal{A}$. To make $\hat{\mu}_a \approx \mu_a$, we need samples of $r(a)$ to improve the accuracy of empirical mean reward $\hat{\mu}_a$ (exploration). However, exploration may incur a cost when the true mean reward of the selected action is smaller than that of other actions, $\mu_a < \mu_k, k \in \mathcal{A} \setminus \{a\}$. To maximize the total rewards, we sometimes need to select the

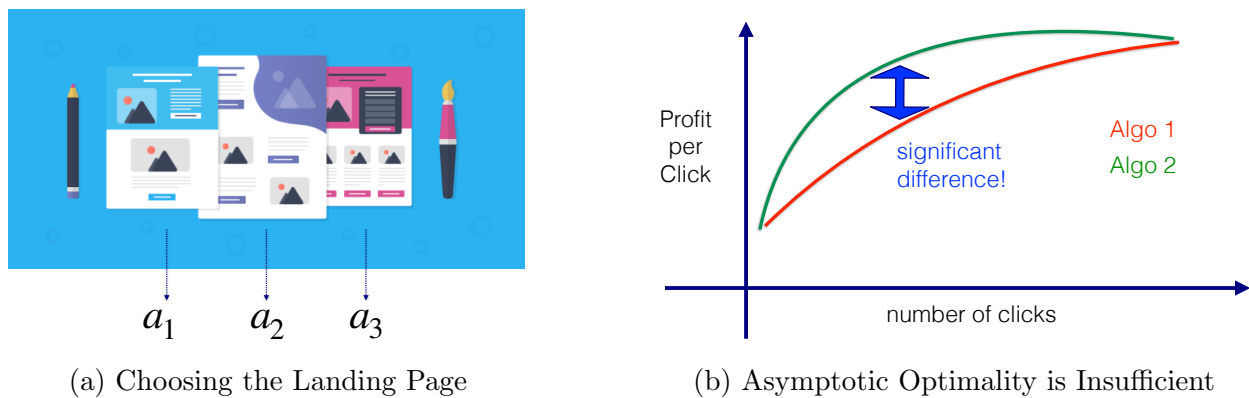


Figure 1: (a) An example of the Multi-Arm Bandits (MAB) problems is the task of selecting website landing pages. (b) Different algorithms that achieve the same asymptotic performance can have different rates of convergence which directly correlated with the profit made by the website owner. Our goal is find an algorithm that converges to the optimal decision making strategy as fast as possible.

best action based on the estimated empirical mean reward (exploitation). We, nevertheless, cannot always exploit since we do not know if the empirical mean reward is close to the true mean. That is why we must balance the actions allocated for exploration and exploitation.

The performance of an algorithm on balancing exploration and exploitation is characterized by *regret*. In plain language, regret means “the loss incurred by past actions.” Quantitatively speaking, regret characterizes the gap between the optimal expected reward and the expected reward obtained by our algorithm. Regret over T time steps is formally denoted as:

$$\mathbf{Regret}(T) = T\mu^* - \mathbb{E}\left[\sum_{t=1}^T r(a_t)\right], \quad (2)$$

where the expectation is taken over multiple episodes of T rounds (one episode corresponds to one day).

Back to the two questions above, the performance of a MAB algorithm is evaluated by $\mathbf{Regret}(T)$ and the principle to design a MAB algorithm is to minimize the regret.

$$\min_{a_1 \dots a_T} T\mu^* - \mathbb{E}\left[\sum_{t=1}^T r(a_t)\right].$$

Note that minimizing regret is equivalent to maximizing rewards. The reason of using regret instead of reward as the objective is that regret offers better theoretical properties to analyze the trade-off of exploration and exploitation (see Chapter 4.4 in [Lattimore and Szepesvári \[2020\]](#) for details).

2.1.1 Explore-then-Commit (ETC)

Let’s start by the simplest strategy to balance exploration-exploitation, **explore-then-commit (ETC)** [[Lattimore and Szepesvári, 2020](#)] algorithm. The design of ETC can be motivated by considering a MAB problem with deterministic rewards. In such a case, an optimal strategy is to try every action once and then choose the action with highest reward for all time steps in the future. Now in stochastic settings, we need to try each action multiple times to get a good estimate of the reward. Let us assume that each action is attempted m times to estimate the mean reward of each action. Afterwards, the agent chooses to execute the optimal action with the highest empirical mean reward. This strategy is ETC. It

spends the first mK rounds on exploration and exploits at the rest of $T - mK$ rounds. ETC algorithm can be simply expressed as below:

Algorithm 1 Explore-then-commit (ETC)

```

1: procedure ETC( $t, m$ )
2:   if  $t \leq mK$  then                                      $\triangleright K$  is the size of action space  $\mathcal{A}$ 
3:      $a_t := (t \bmod K) + 1$ 
4:   else
5:      $a_t := \arg \max_{a_k} \hat{\mu}_k(mK)$ 
6:   end if
7: end procedure

```

The next question is: *how good is this algorithm?* As [Lattimore and Szepesvári \[2020\]](#) states, the regret of ETC is bounded as below

$$\mathbf{Regret}(T) \leq \underbrace{m \sum_{k=1}^K \Delta_k}_{(i)} + \underbrace{(T - mK) \sum_{k=1}^K \Delta_k \exp(-\frac{m\Delta_k^2}{4})}_{(ii)}, \quad (3)$$

where $\Delta_k := \mu^* - \mu_k$ denotes the suboptimality gap of action k . The formal derivation can be found in Chapter 6.1 of [Lattimore and Szepesvári \[2020\]](#). Intuitively, this regret can be treated as two parts: (i) regret incurred in exploration and (ii) the regret of exploiting the found optimal action.

Here we notice that the regret of ETC is determined by choice of m . As such, the natural question is how to choose m to meet the desired regret bound. If the agent turns into the exploitation stage (i.e., m is small) early, the estimated mean rewards of each action can be erroneous. Selecting actions based on erroneous empirical mean rewards will lead to suboptimal rewards. On the other hand, choosing an overly large m can waste too much time on exploration, and thus the total rewards can be low. How is m typically set? In practice, $m := T^{2/3}$ and the corresponding regret is bounded as

$$\mathbf{Regret}(T) \leq O(KT^{2/3}).$$

This is better than the worst regret $O(T)$. The worst regret can be trivially obtained if the agent constantly chooses a suboptimal action or randomly selects actions through T rounds.

2.1.2 Upper Confidence Bound (UCB)

Though ETC (Section 2.1.1) improves the regret bound over the worst case, the caveat is that the exploration is “unguided”. Recall that in the exploration phase (i.e., $t \leq mK$ in Algorithm 1), we select actions a uniformly over the action space $\mathcal{A}$. Uniformly trying out all actions can be wasteful if we are already certain about the expected reward of an action. As our certainty about each action changes over time, designing an adaptive exploration strategy based on uncertainty would be preferable. To adaptively explore in such a way, we need a mechanism to model uncertainty for each action and an action selection rule to take the most promising action for exploration.

Upper confidence bound (UCB) [[Lattimore and Szepesvári, 2020](#)] algorithm constructs uncertainty of actions using confidence bounds of mean reward. It employs the principle of

optimism in the face of uncertainty to select actions. This principle essentially says that we should overestimate the expected reward of each action when we are uncertain about it. In the landing page recommendation example (see Section 2.1), consider we have never tried recommending a page to users. We should overestimate its value; otherwise, we might never try this option and possibly miss an option better than the currently known optimal one. On the other hand, after recommending this page many times and realizing that the expected reward is low, we should stop trying this page and shift to other options.

As we see, the UCB algorithm is an adaptive strategy that changes the tendency of exploration adaptively over time based on the principle of optimism in the face of uncertainty. Acting optimistically means that we aim for constructing $\hat{\mu}'_k$ such that $\hat{\mu}'_k \geq \mu_k$. Finding such $\hat{\mu}'_k$ can be intractable since we do not know μ_k . Luckily, assuming $r(a_t)$ is drawn from a 1-subgaussian distribution, we can construct $\hat{\mu}_k$ in a probabilistic manner. Remember that μ_k is the mean reward. Thus the following inequality holds (see Chapter 5 and Chapter 7 in Lattimore and Szepesvári [2020] for more details)

$$P(\mu_k \geq \hat{\mu}_k + \epsilon) \leq \exp(-\frac{T_k(t)\epsilon^2}{2}), \quad \epsilon \geq 0. \quad (4)$$

Equivalently, solving for ϵ when equality holds, we transform the inequality 4 to

$$P(\mu_k \geq \hat{\mu}_k + \sqrt{\frac{2 \log 1/\delta}{T_k(t)}}) \leq \delta, \quad \delta \in (0, 1)$$

As a result, $\hat{\mu}'_k$ can be constructed by

$$\hat{\mu}'_k := \hat{\mu}_k + \sqrt{\frac{2 \log 1/\delta}{T_k(t)}}.$$

$\hat{\mu}'_k$ is termed as "*upper confidence bound*." of rewards because $\hat{\mu}_k + \sqrt{\frac{2 \log 1/\delta}{T_k(t)}} \geq \mu_k$ with at least probability $1 - \delta$.

Since $\hat{\mu}'_k$ is an overestimate, it enables the agent to choose actions optimistically. The action selection rule is outlined in Algorithm 2.

Algorithm 2 Upper Confidence Bound (UCB)

1: **procedure** UCB(t)

2: Select action $a_t = \arg \max_k \hat{\mu}_k + \sqrt{\frac{2 \log 1/\delta}{T_k(t)}}$

3: **end procedure**

The term $\sqrt{\frac{2 \log 1/\delta}{T_k(t)}}$ would encourages the agent to select suboptimal actions when the agent has not yet selected this action sufficiently many times. Suppose there are only 2 actions and the optimal action is 1 and $\mu_1 \geq \mu_2$. When $T_2(t)$ is small enough, the upper confidence bound $\hat{\mu}'_2$ can be larger than $\hat{\mu}'_1$ and the agent thus will prefer taking suboptimal actions 2 over the optimal action 1. Also, it can be seen that tendency of trying suboptimal actions $2 \neq 1$ decays along $T_2(t)$. After the agent has selected action 2 a number of times and $\hat{\mu}_2 \approx \mu_2$, the upper confidence bounds of suboptimal action 2 will be smaller than that of the optimal action 1 (i.e., $\hat{\mu}'_2 < \hat{\mu}'_1$). As a result, the agent will commit to the optimal action 1 eventually after sufficient samples are collected.

In addition, the **confidence level** δ also controls the how quickly we become certain about the expected reward of an action k . After we tried an action k many times, we are certain

about its expected reward and hope $\hat{\mu}_i = \mu_i$. How do we set δ to make sure the term $\sqrt{\frac{2\log 1/\delta}{T_k(t)}}$ disappears when t is large? An excessively large δ can waste much time on exploration, while an overly small δ will cause the agent to prematurely stick to suboptimal actions. Following the setting suggested in [Auer et al., 2002] we set $\delta = 1/t^2$ for the rest of analysis in this section. Different choices of δ lead different variants of UCB algorithms with different regret. For more detailed discussion of δ , please refer to Chapter 7.1 in Lattimore and Szepesvári [2020] and Auer et al. [2002]. Let's look at if UCB achieves lower regret than ETC. As stated in [Auer et al., 2002], with $\delta = 1/t^2$, the regret of UCB is expressed as

$$\mathbf{Regret}(T) \leq 8\left(\sum_{k=2}^K \frac{\log T}{\Delta_k}\right) + \left(1 + \frac{\pi^2}{3}\right)\left(\sum_{k=1}^K \Delta_k\right)$$

where without loss generality we let $\mu_1 = \mu^*$ for simplicity. The regret of UCB is bounded as

$$\mathbf{Regret}(T) \leq O(\sqrt{KT \log T}),$$

which is tighter than that of ETC. The detailed derivation can be found in Auer et al. [2002].

2.2 Contextual Bandit (CB)

In the previous sections, we considered the special case where the reward was independent of the state. However, this is rarely the case in the real world. For instance, in the landing page example, when a user visits a website, many traits about the user such as the location, gender, age, time of visit, etc might be known. Ideally, we would utilize this information to improve recommendations. We can think user traits (or features) as the *state* (x) of the system. Unlike, MAB where the reward is assumed to independent of the state, in CB we assume that reward is state (or *context*) dependent, $r(s_t, a_t) = r(s_{1:t}, a_{1:t})$. Thus, algorithms making use of context information are termed as **contextual bandit (CB)** algorithms. Alternatively, if were to ignore the context, we would return back to the MAB formalism. However this would be sub-optimal because MAB algorithms don't adapt according to the user.

A naive way to account for user information is to solve a separate MAB problem per user. In this way, over time we can specialize to each user. However, this approach might be suboptimal since we would need to solve the exploration-exploitation problem separately for every user. If two users are similar, it should be possible to transfer knowledge between them. Second, if the website encounters a new user, instead of re-starting the exploration-exploitation process it will be more profitable to re-use knowledge of other similar users. This would result in the user encountering meaningful recommendation in his/her first visit.

To introduce the concepts of CB, we augment MAB as follows:

1. **Context** x_t : The context used to decide on the action a_t at round t , where $x_t \in \mathcal{X}$. $\mathcal{X}$ is the context space that contains all possible contexts. E.g., in the landing page recommendation system, x_t can be clients' age, gender, location, nationality, etc.
2. **Contextual reward** $r(x_t, a_t)$: The reward at round t , conditioned on context x_t and action a_t , where $r(x_t, a_t) \in \mathbb{R}$. $r(x_t, a_t)$ is the profit earned by the website. a_t corresponds to the page to recommend. Same as MAB, the true reward is inaccessible to the agent.

3. **Contextual expected regret $\text{Regret}(T)$** : The expected regret of an CB algorithm, that can be expressed as

$$\text{Regret}(T) := \mathbb{E} \left[\sum_{t=1}^T \max_{a \in \mathcal{A}} r(x_t, a) - r(x_t, a_t) \right],$$

where the expectation is taken over past interactions between the agent and the environment.

The key difference to MAB is that the reward is conditioned on the contexts. What we haven't discussed yet is how to select actions in CB. We will soon show that UCB can be easily extended to CB problems Section 2.2.1.

2.2.1 LinUCB

In this section, we introduce a simple method that extends UCB to CB problems – LinUCB [Li et al., 2010]. Before we dive into the detail, let's think of what are the problems of applying the UCB algorithm in the CB framework. Obviously, the first problem is that the expected reward $\hat{\mu}_k$ in UCB is context-independent. Even though we make $\hat{\mu}_k$ context-dependent as the follows:

$$\hat{\mu}_{x,k}(t) := \frac{1}{T_{x,k}(t)} \sum_{t=1}^T \mathbb{I}\{a_t = k, x_t = x\} r(x_t, a_t),$$

The problem is how to obtain $T_{x,k}(t)$ without knowing the context space $\mathcal{X}$. $T_{x,k}(t)$, the counts that action k has been taken at context x after round t , are required in computing the empirical expected reward $\hat{\mu}$ and upper confidence bound $\hat{\mu}'$. If $\mathcal{X}$ cannot be enumerated, there will be an infinite number of $T_{x,k}$. Therefore, we need an approach that does not require enumerating all possible contexts $x \in \mathcal{X}$ for estimating (i) the empirical mean of rewards and (ii) the confidence bound of rewards. Below, we start illustrating how LinUCB estimates mean and confidence intervals without enumerating contexts.

The key idea of LinUCB is to estimate the context-dependent empirical mean and confidence interval by linear regression. In LinUCB, contexts $x \in \mathbb{R}^d$ are all assumed to be d -dimensional vectors. There are many different implementations of LinUCB in Li et al. [2010]. Here we focus on LinUCB with disjoint linear models proposed in Li et al. [2010]. Hereinafter, we will refer LinUCB to this disjoint version of LinUCB. LinUCB constructs a linear regression model for each action k with parameter vector $\hat{\theta}_k \in \mathbb{R}^d$. The context-dependent empirical mean is constructed as

$$\hat{\mu}_{\hat{\theta}_k}(x) := \hat{\theta}_k^T x.$$

Empirical mean Since rewards of taking action k at context x are assumed to be drawn from a 1-subgaussian distribution, the true mean $\mu_{x,k}$ of the reward distribution can be approximated by solving for $\hat{\theta}_k^*$ of the least-square regression problem

$$\hat{\theta}_k^* = \arg \min_{\hat{\theta}_k} \mathbb{E} [(\hat{\mu}_{\hat{\theta}_k}(x) - r(x, k))^2],$$

where the expectation is taken over the distribution of contexts and rewards. Regarding why least-square regression approximates the mean of a gaussian distribution, interested

readers can refer to Chapter 19 in [Lattimore and Szepesvári, 2020]. LinUCB employs ridge regression [Marquardt and Snee, 1975] to solve for $\hat{\theta}_k^*$. As ridge regression is a general method for solving regression problems, we will not go into the details of the derivation of ridge regression but show how to fit mean reward estimation into the closed-form solution of ridge regression. The closed-form solution of the optimal parameters $\hat{\theta}_k^*$ can be expressed as follows

$$\hat{\theta}_k^* = (\mathbf{D}_k^T \mathbf{D}_k + \lambda \mathbf{I})^{-1} \mathbf{D}_k^T \mathbf{c}_k, \quad (5)$$

where $\mathbf{D}_k$ is a $n \times d$ matrix, $\mathbf{I}$ is a $d \times d$ identity matrix, $\mathbf{c}_k$ is a d -dimensional vector, and λ is an arbitrary scalar for controlling the regularization in ridge regression. Each row of $\mathbf{D}_k$ contains a context vector x received when the agent took action k . Each column i of $\mathbf{c}_k$ indicates a reward $r(x_i, k)$, where x_i is i -th row of $\mathbf{D}_k$.

Confidence bound Recall that in Section 2.1.2 we constructed the upper confidence bound by the following property of 1-subgaussian distribution

$$P(\mu_k \geq \hat{\mu}_k + \epsilon) \leq \exp(-\frac{T_k(t)\epsilon^2}{2}), \quad \epsilon \geq 0. \quad (6)$$

However, since $T_k(t)$ is unavailable in CB, we need a workaround to construct the confidence bound. Li et al. [2010] suggests constructing the bound in the following manner

$$P(\mu_{x,k} \geq \hat{\mu}_{\hat{\theta}_k}(\mathbf{x}_{t,k}) + \alpha \sqrt{\mathbf{x}_{t,k}^T (\mathbf{D}_k^T \mathbf{D}_k + \lambda \mathbf{I})^{-1} \mathbf{x}_{t,k}}) \leq \delta, \quad \delta \in (0, 1),$$

where $\alpha := 1 + \sqrt{\log(2/\delta)/2}$. Note that in the original LinUCB paper Li et al. [2010], $\mathbf{x}_{t,k}$ denotes the context of action k at round t since their $\mathbf{x}_{t,k}$ contain features of actions. The full derivation of this inequality can be referred to Chapter 20 of Lattimore and Szepesvári [2020] and Walsh et al. [2012].

Algorithm Putting the empirical mean and confidence bound together, we outline the algorithmic procedure of LinUCB in Algorithm 3 adapted from the original algorithm presented in Li et al. [2010].

Algorithm 3 LinUCB

```

1: procedure LINUCB( $t, \lambda$ )
2:   Receive contexts for all  $k \in \mathcal{A}$ :  $x_{t,k} \in \mathbb{R}^d$ 
3:   for  $k \in \mathcal{A}$  do
4:     if  $k$  has never been taken then
5:        $\mathbf{A}_k \leftarrow \lambda \mathbf{I}_{d \times d}$ 
6:        $\mathbf{b}_k \leftarrow \mathbf{0}_{d \times 1}$ 
7:     end if
8:      $\hat{\theta}_k \leftarrow \frac{\mathbf{A}_k^{-1} \mathbf{b}_k}{(\mathbf{D}_k^T \mathbf{D}_k + \lambda \mathbf{I})^{-1} \mathbf{D}_k^T \mathbf{c}_k}$ 
9:
10:  end for
11:  Select action  $a_t \leftarrow \arg \max_k \hat{\theta}_k^T \mathbf{x}_{t,k} + \alpha \sqrt{\mathbf{x}_{t,k}^T \mathbf{A}_k^{-1} \mathbf{x}_{t,k}}$ 
12:   $\mathbf{A}_k \leftarrow \mathbf{A}_k + \mathbf{x}_{t,k} \mathbf{x}_{t,k}^T$ 
13:   $\mathbf{b}_k \leftarrow \mathbf{b}_k + r_t \mathbf{x}_{t,k}$  ( $r_t \in \mathbb{R}$  is the received reward  $r(x_{t,k}, k)$ )
14: end procedure

```

Readers might notice that the connection of Algorithm 3 to the Equation 5 and the inequality 6 is not obvious or at least not straightforward. Algorithm 3 states a procedure to construct $\mathbf{D}_k^T \mathbf{D}_k + \lambda \mathbf{I}$ and $\mathbf{D}_k^T \mathbf{c}_k$ in an online fashion. $\mathbf{D}_k^T \mathbf{D}_k$ can be written as $\sum_{t=1}^T \mathbf{x}_{t,k} \mathbf{x}_{t,k}^T$. $\mathbf{D}_k^T \mathbf{c}_k$ is equivalent to $\sum_{t=1}^T r_t \mathbf{x}_{t,k}$. Factorizing $\mathbf{D}_k^T \mathbf{D}_k + \lambda \mathbf{I}$ and $\mathbf{D}_k^T \mathbf{c}_k$ as the summation over time allows for the agent to update them online during interaction with the environment.

3 Sequential Decision Making

In the last chapter, we covered decision-making algorithms that utilize contextual information to maximize reward. However, there are many real-world problems where the *context (or state) can be modified by actions* and therefore don't fit the contextual bandit setup. We will delve into such problems. To develop an intuitive understanding of problems that are not well modeled by CB, let's revisit the landing page recommendation example from Section 2.

Modified landing page recommendation example: In the running example of recommending landing pages, our goal was to maximize the website's profit. Until now, we considered limited interaction where the user's interaction with the website does not change the page viewed by the user. The algorithms we studied maximize profit by choosing one out of many available pages *exactly once*. However, a user clicking on a link on the webpage expresses a user's interest in a specific topic, and it makes sense to display new content in response to the user's interest. To the website designer, each click provides additional information about the user and can be incorporated in decision-making by modifying the user features (or context). In this scenario, the website host is required to make decisions on what content to show in response to every click of the user. The problem is, therefore, to make a *sequence of decisions* that maximize the website's profit.

3.1 Issues of CB

The problematic assumption in CB formulation is that the context at each round t is independent of past actions and will not change according to users' activities on the web page. Let's consider a concrete case when we apply CB in this modified example. Initially, our system presented a variety of items that could interest women (e.g., cars, watches, jackets, etc) to female users based on the gender attributes in the context x_1 . If a user is particularly interested in jackets and clicks on links to jackets, this indicates that this user might be interested in items similar to jackets and more likely to spend money. Ideally, incorporating what users have seen in the website into the context at next round x_2 will enable our system to personalize the recommended contents to users and increase the chance that users spend money. However, because CB does not account for changes in context based on actions, CB will be suboptimal in its inability to exploit the user's interests reflected by the clicks.

The key issue of CB is overlooking context transitions affected by actions. We refer to the decision-making problem with context transition as *sequential decision making* (SDM) in the following sections. Before we dive into the introduction to SDM algorithms, let's recall what we have learned previously and what we will learn in this section. We list the comparison of MAB, CB, and SDM problems in Table 1. It can be seen that SDM differs from MAB and

CB in that actions affect context transitions, which can account for more realistic problems in the real world.

Table 1: Comparison of MAB, CB, and SDM.

	Objective	Stochastic rewards	Context-dependent	Actions affect context transition
MAB	$\max \mathbb{E} \left[\sum r(a_t) \right]$	Yes	No	No
CB	$\max \mathbb{E} \left[\sum r(x_t, a_t) \right]$	Yes	Yes	No
SDM	$\max \mathbb{E} \left[\sum r(x_t, a_t) \right]$	Yes	Yes	Yes

3.2 Sequential Decision Process

Since the CB framework overlooks context transitions, we need to modify our framework to account for that. The critical difference to the CB framework is that we introduce the notion of a *transition function* to characterize the context transitions. We introduce this new framework by defining notation necessary to understand the SDM setup. Note that some of the notation is the same as CB.

- **Timestep t :** Timestep is the same as round in CB. At each timestep t , the agent observes a state and takes an action.
- **State $s \in \mathcal{S}$, Action $a \in \mathcal{A}$, and Reward $r(s_{1:t}, a_{1:t}) \in \mathbb{R}$:** A state s_t contains all variables or observations related to the decision making problem (e.g., user activities, gender, etc) at time step t , where $\mathcal{S}$ denotes the state space. This is identical to contexts (x) used in CB, but typically they are called states in the SDM literature. Action a is identical to CB. We rewrite reward $r(s_{1:t}, a_{1:t})$ as a state-action dependent function and abbreviate it as r_t for brevity.
- **Transition function $\mathcal{T}(s_{t+1}|s_{1:t}, a_{1:t})$:** The probability of transitioning to s_{t+1} conditioned on history of states and actions $s_{1:t}$ and $a_{1:t}$ from timestep 1 to t . This is unique to the SDM formulation. The state transition is usually modeled as a stochastic function because of unobserved factors or random phenomena (e.g., Brownian motion) in state transitions. Consider users' activities as parts of a state in the modified page recommendation example. When we recommend a page (take action a_t) for a user (state s_t), we are unlikely to know what items the user will buy. As such, the next state s_{t+1} (the user's activities at the next step $t + 1$) of taking action a_t at state s_t cannot be predicted certainly. Another example from the lecture is the rock-paper-scissors game. The opponent's decision is unpredictable, and thus the state transition has to be stochastic.
- **Policy π :** At each time step t , the agent takes action $a_t = \pi(s_{1:t}, a_{1:t-1})$. In SDM, a policy can be either deterministic or stochastic. As such, we sometimes treat π as a conditional probability distribution over actions $\pi : (\mathcal{S} \times \mathcal{A})^t \mapsto [0, 1]^{|\mathcal{A}|}$ and sample from it.

Similar to CB, the objective in SDM is to find a policy π (i.e., decision making rule) that maximizes the expected total reward over time under a distribution of states. This objective

can be expressed as

$$\max_{\pi} \mathbb{E}_{s_1 \sim P(\mathcal{S}), a_t \sim \pi(s_{1:t}), s_{t+1} \sim \mathcal{T}(s'_{1:t}, a_{1:t})} \left[\sum_{t=1}^T r(s_{1:t}, a_{1:t}) \right], \quad (7)$$

where $P(\mathcal{S})$ denotes an arbitrary distribution over the state space. Note that for brevity, we will refer to the sum of rewards as “**return**”. In essence, we want to maximize the *return* when the agent is initialized at state sampled from $P(\mathcal{S})$ and takes action using the policy π . This objective only differs from the CB objective in that the next state s_{t+1} at each time step depends on previous state/actions via the transition function $\mathcal{T}$. We only assume that the initial state, s_1 is independent of $\mathcal{T}$. In consequent references to this objective function, for notational convenience, we will not explicitly write down what the expectation is with respect to.

Some notes,

1. Readers who are familiar with Markov Decision Processes (MDP), might be confused why we do not use MDPs to model the sequential decision-making process. It is because we consider more general SDM problems. In other words, solutions to SDM problems in this chapter do not require the Markov assumption.
2. It is a modelling choice to maximize *expected* rewards. One could be interested in optimizing other quantities – e.g., worst case performance or best case performance. Many real-world problems are well modelled by maximizing expected return and that’s why we use this formulation. Later in the course we will discuss other formulations which might become necessary for safety, being risk-averse etc.

3.3 Finite and Infinite Horizon

There are two classes of problems that arise depending on the time-horizon T in Equation 7. In some scenarios, one intends to maximize rewards forever (e.g., happiness) while in other scenarios rewards must be optimized in a fixed time interval (e.g., minimizing the cost to reach the airport within a pre-specified time). These two problem setups are referred to as *infinite* and *finite* time horizon problems. In the running example of online shopping, if users can only spend a certain maximum time on the website then the problem is finite-horizon. On the other hand, if a user can stay at the website forever, the problem will be infinite-horizon. We introduce the concepts of finite- and infinite- horizons in this section because, in the later sections, we will visit SDM problems with infinite horizons.

Notice that the framework of SDM developed so far cannot handle both finite and infinite SDM problems. This is because, first, if $T \approx \infty$, the objective in 7 will be unbounded since the sum of rewards over an infinite horizon results in infinity. Second, it is tricky to apply the finite horizon setting to SDM. Let’s consider the modified page recommendation example. The remaining time $T - t$ could affect how users make decisions. For example, if only one step remains in the user’s stay at our website, our system should direct the user to the checkout page instead of presenting more items to browse. Although it makes more sense to incorporate the remaining time into the state and write the policy as $\pi(s_t, T - t)$, it requires knowledge of the horizon T in advance Harada [1997], which is difficult in practice (i.e., how do you know how much time a user will spend in your website?).

Typically instead of using two different frameworks for finite and infinite time-horizon problems, it is common problem to use a single unified view of infinite horizon to solve SDM problems. To develop such a view, a *heuristic* is introduced in the infinite horizon formulation which intuitively translates into rewards in the distant future being less important than immediate rewards. The hypothesis behind this heuristic is that attenuating the value of future rewards is often a valid guideline for decision-making (i.e., our system should not expect the rewards that could happen 1,000 years later). With this heuristic, we restate the objective 7 to the following

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) \right], \quad (8)$$

where γ denotes the discount factor for decaying the importance of future rewards, and we term $\mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) \right]$ as **discounted objective** and $\mathbb{E} \left[\sum_{t=1}^{\infty} r(s_t, a_t) \right]$ as **undiscounted objective**.

Though the discounted objective is a biased objective (more discussion later) of the undiscounted objective, we use discounted objective for both finite- and infinite- horizon problems in practice due to modeling convenience. Specifically, the use of discounting unifies the two settings as:

1. For the infinite horizon case, for reasonably general choice of r_t , the the sum of rewards is bounded.
2. One can model a finite time horizon problem by choosing an appropriate γ .

3.4 Trajectory and Episode

Now that we have formalized the problem setup, we will consider how to maximize the expected return. To maximize the expectation, it is necessary to find a good estimate of the expected return. The quality of this estimate can be increased by collecting more data. Specifically, we can initialize the agent at a state $s_1 \sim P(\mathcal{S})$ and then execute the policy π . Because of stochasticity in the policy or the environment, such execution will result in a different sequence of state, actions and rewards. This process is known as “*rolling out*” the policy, and the collected data, $(s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T)$, is referred to as “*rollout*”. In practice we cannot rollout for infinite time step, only roll out for T time steps. One such rollout is referred to as a *trajectory*. Intuitively, performing multiple rollouts, colloquially also known as collecting multiple trajectories, provides a better estimate of the expected reward.

Suppose we have a budget of N steps to collect data. One possibility is to collect a single trajectory of N steps. Let the set of states visited by this policy be $\mathcal{S}^N$ (*case 1*). The other possibility is to collect K trajectories initialized at s_1 , but each with $\frac{N}{K}$ steps (*case 2*). Let the states visited by such a policy be $\mathcal{S}^K$. Because we are terminating the policy earlier, intuitively we would visit a smaller set of states (i.e., $|\mathcal{S}^K| < |\mathcal{S}^N|$), but we will visit the some of these states more often. Consequently, if we are estimating the expectation with samples,

$$\frac{1}{K} \sum_{k=1}^K \left[\sum_{t=1}^{\frac{N}{K}} \gamma^{t-1} r(s_t^k, a_t^k) \right] \quad s.t. \ s_t^k == s_1 \forall k$$

then in case 1, we will visit more states the estimate might be high-variance in these states. In case 2, we will visit a smaller states (so potentially biased), but will be able to make estimates with lower variance. The other consideration is that if we have only a single rollout, then if the agent enters an irrecoverable state (e.g., a robotic walker falling down) then all the data will be collected in potentially bad states. While we have not formalized these arguments yet, intuitively it makes sense to sample multiple trajectories and we will see more formal reasons later in our discussion. The process of sampling a single trajectory is also known as an *episode*.

Some tasks are naturally *episodic*. For instance, in the shopping example, an episode corresponds to the sequence of links the user makes to complete the purchase. When the user re-visits the website again, it would be a different episode. Other tasks like getting a robot to walk are not innately episodic. However, for the (*hand-wavy*) reasons discussed above it is common practice to break data collection into episodes.

As such, we typically “*artificially*” reset the agent to other states every $T = \frac{N}{K}$ (where T is a finite number) steps in an infinite-horizon problem. There are some questions to ponder about: (i) what states should the environment be reset to?; (ii) How practical is the reset assumption; (iii) Is there a reason to develop reset-free algorithms? While we will not answer these questions now, its good to keep them at back of your mind. In summary, the sequence of states, actions, and rewards over T timesteps is then termed as **Trajectory (or Episode)**. We denote a trajectory as τ

$$\tau_{1:T}^i := (s_1^i, a_1^i, r_1^i, s_2^i, a_2^i, r_2^i, s_3^i, \dots, s_{T-1}^i, a_{T-1}^i, r_{T-1}^i, s_T^i),$$

where $1 : T$ denotes the starting and ending step of τ and i denotes the trajectory index. With notions of trajectories, we can treat the infinite horizon problem as many chunks of fixed-horizon trajectories instead of a single large trajectory with an infinite horizon. As such, we rewrite the problem of maximizing discounted objectives as the follows

$$\begin{aligned} & \max_{\pi} \mathbb{E}_{s_1 \sim P(\mathcal{S}), a_t \sim \pi(s_{1:t}), s_{t+1} \sim \mathcal{T}(s' | s_{1:t}, a_{1:t})} \left[\sum_{t=1}^T \gamma^{t-1} r(s_{1:t}, a_{1:t}) \right] \\ &= \max_{\pi} \mathbb{E}_{\tau_{1:T}^i} \left[\sum_{t=1}^T \gamma^{t-1} r(s_{1:t}, a_{1:t}) \right] \end{aligned}$$

3.5 Summary

To wrap up SDM, we summarize interaction between the agent and the environment as the follows. For an episode i , starting from an initial state $s_1^i \sim P(\mathcal{S})$, the agent takes action $a_{1:t}^i = \pi(s_{1:t}^i)$, observes reward $r(s_{1:t}^i, a_{1:t}^i)$, and transitions to the next state s_{t+1}^i for each step $t > 1$. After $t = T$, we start a new episode $i + 1$ and reset the agent to a state $s_1^{i+1} \sim P(\mathcal{S})$. The overall goal of the agent is to find a policy π such that

$$\max_{\pi} \mathbb{E}_{\tau_{1:T}^i} \left[\sum_{t=1}^T \gamma^{t-1} r(s_{1:t}, a_{1:t}) \right]. \quad (9)$$

4 Policy Gradient

In this section, we introduce a family of algorithms for solving SDM problems – *policy gradient*. The idea of policy gradient is to take the gradient of the sum of rewards with

respect to the policy π and maximize the sum of rewards by gradient ascent. We will start by introducing the first version of policy gradient in Section 4.2 and introduce more advanced versions in Section 4.3.

4.1 Derivation and Properties of Policy Gradient

The objective of policy gradients is to find a policy that maximizes the total sum of rewards defined in Equation 7:

$$\max_{\theta} J(\theta), \quad J(\theta) = \mathbb{E}_{\pi_{\theta}}[R^{\gamma}(\tau)], \quad (10)$$

where $R^{\gamma}(\tau)$ denotes the discounted return of a trajectory τ and π_{θ} denotes a policy parameterized by θ are parameters of a function approximator.¹ Rewriting the objective in this form will make derivation of policy gradients easier and clearer. Let $p_{\theta}(\tau)$ be the probability of a trajectory τ being sampled. We can express the gradient of the expected return $\nabla_{\theta}J(\theta)$ as

$$\begin{aligned} \nabla_{\theta}J(\theta) &= \nabla_{\theta}\mathbb{E}_{\pi_{\theta}}[R^{\gamma}(\tau)] \\ &= \nabla_{\theta} \int_{\tau} p_{\theta}(\tau) R^{\gamma}(\tau) d\tau \\ &= \int_{\tau} \nabla_{\theta} p_{\theta}(\tau) R^{\gamma}(\tau) d\tau \\ &= \int_{\tau} p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} R^{\gamma}(\tau) d\tau && \text{(by } \frac{d \log x}{dx} = \frac{1}{x} \text{)} \\ &= \int_{\tau} p_{\theta}(\tau) \nabla_{\theta} \log(p_{\theta}(\tau)) R^{\gamma}(\tau) d\tau \\ &= \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log(p_{\theta}(\tau)) R^{\gamma}(\tau)]. \end{aligned}$$

This expression $\nabla_{\theta}J(\theta)$ can then be intuitively connected to the three procedures shown in the beginning of this section. The term $\log(p_{\theta}(\tau))$ corresponds to log-likelihood of a trajectory and $R^{\gamma}(\tau)$ is the weight of a trajectory τ . Updating θ by gradient ascent $\theta \leftarrow \theta + \alpha \nabla_{\theta}J(\theta)$ directs the policy to increase expected returns $J(\theta)$, where $\alpha \in \mathbb{R}$ is learning rate.

Remarks Policy gradient has some noteworthy properties.

Remark 4.1 (Continuous action spaces). Recall that in CB, we select action a_t from the best one over K actions

$$a_t = \arg \max_{k \in \mathcal{A}} \hat{\mu}_k(x_t).$$

It is impossible to enumerate all actions in a setup with continuous action space (e.g., motor commands in a robot). Policy gradient makes no assumption about discrete or continuous action spaces. We can use any choice of a probability distribution for π and sample actions as:

$$a_t \sim \pi(a|s_t).$$

¹Note that we usually parameterize the policy as an arbitrary function class; otherwise, without proper parameterization, we need to solve optimization problems using variational calculus, which is difficult.

Remark 4.2 (Model-free). The expression of policy gradient $\nabla_{\theta} J(\theta)$ can be rewritten as

$$\mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log(p_{\theta}(\tau)) R^{\gamma}(\tau)] = \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \sum_{t=1}^T \log(\pi_{\theta}(a_t | s_{1:t}, a_{1:t})) R(\tau_{t:T}) \right].$$

It can be seen that the expression of policy gradient is independent of state transition function P , which means that policy gradient is “*model-free*”². This indicates that estimating policy gradients simply requires sampling trajectories from the environment. The model-free property is helpful in environments where the state transition function is unknown or hard to approximate (e.g., humanoid robots, cheetah running in complex terrain, etc.).

Remark 4.3 (Markov Assumption is not Required). In context of a policy, the markov assumption holds if:

$$\pi(a_t | s_t, s_{t-1}, \dots, s_1) = \pi(a_t | s_t),$$

It essentially states that the choice of the current action only depends on the current state (s_t). As we will see, many sequential decision-making algorithms rely on the Markov assumption. However, policy gradient does not:

$$\mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \sum_{t=1}^T \underbrace{\log(\pi_{\theta}(a_t | s_{1:t}, a_{1:t}))}_{\text{Depend on past states and actions}} R(\tau_{t:T}) \right]$$

does not rely on the Markov property. However, it is possible to make the Markov assumption to simplify the learning. We will discuss more about the Markov property later in Section E.

4.2 REINFORCE

The policy gradient in its vanilla form described above is also referred to as the REINFORCE [Sutton et al., 1999] algorithm. Intuitively, at every update, REINFORCE increases the likelihood of action sequences that yield higher returns. The key steps in the REINFORCE algorithm are:

1. Roll out a trajectory (τ^i) using the current policy π_{θ} .
2. Compute the gradient of the log-likelihood of all actions, $g^i = \nabla_{\theta} \log \pi_{\theta}(\cdot | \cdot)$.
3. Weigh the gradient by the corresponding returns, $g_i R(\tau^i)$.
4. Update the policy parameters by weighted gradients, $\Delta \theta = \alpha g_i R(\tau^i)$, where α is the learning rate.

Pseudocode We outline the implementation of REINFORCE in Algorithm 4 using slight modifications in notation to make the pseudo-code closer to actual code one might write to implement the algorithm.

- $\tau_{t:T}^i$: The segment of from timestep t to timestep T of i -th trajectory in the collected trajectories.
- s_t^i : The state at timestep t in i -th trajectory.

Algorithm 4 REINFORCE

- 1: **procedure** REINFORCE
 - 2: Collect N trajectories $\{\tau_{1:T}^i\}_{i=1}^N$ by policy π_θ , where each τ^i has length T
 - 3: Compute gradient $\nabla_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi(a|s_t^i) R^\gamma(\tau_{t:T}^i)$
 - 4: Update the policy by gradient ascent $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
 - 5: **end procedure**
-

Note that in this implementation, we assume Markov property and thus use a history-independent policy $\pi(a|s_t)$ for simplicity. Modeling a history dependent policy will make the policy more complex, which is beyond the scope of this lecture.

4.3 Credit Assignment

Assigning credit to each decision $\pi(a|s_t^i)$ properly is at the core of the success of policy gradient. The idea of policy gradient is to increase the probability of good decisions and decrease the probability of poor decisions. Policy gradient methods realize this idea by weighting the gradients corresponding to each decision $\pi(a|s_t^i)$ proportional to their return $R(\tau_{t:T}^i)$, as shown below

$$\sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi(a|s_t^i) \underbrace{R(\tau_{t:T}^i)}_{\text{Credit of decision } \pi(a|s_t^i)}.$$

Unfortunately, credit assignment is far from easy due to high variance in sampled trajectory returns $R(\tau_{t:T}^i)$. Consider the modified page recommendation example. When our system recommends a page to a user, the return corresponding to this action in many steps later is of high variance since the trajectory return is summed from many steps. The fundamental reason is that it is challenging to distinguish the contribution of each action to the observed trajectory returns. Moreover, the variance of future rewards increases with the length of the trajectory.

Discount One straightforward thinking is to down-weight the importance of future rewards. Obviously, in $R^\gamma(\tau_{t:T})$ the reward r_i with larger i receives larger discount γ^{i-t} . Variance resulted in future will be annealed since future reward magnitude is largely reduced. However, that leads to a bias-variance trade-off. An excessively small γ enables the agent to converge fast due to low variance of returns, but causes the agent to bias toward intermediate rewards (i.e., myopic).

Baselines The idea of introducing a baseline is to increase the probability of trajectories with above-average returns and decrease that with below-average, where the baseline is the average return. Intuitively, we increase the probabilities of trajectories with “advantage”. For the trajectories starting from state s_t , advantage quantifies the gap between the return of a trajectory $R^\gamma(\tau_{t:T})$ to the average trajectory return $b(s_t) = \mathbb{E}[R^\gamma(\tau_{t:T})]$. With the notion of advantage, we rewrite the expression of policy gradient as

$$\sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi(a|s_t^i) \underbrace{(R^\gamma(\tau_{t:T}^i) - b(s_t))}_{\text{Advantage}},$$

²We usually term a method without the needs of state transition function as a model-free method.

In addition to intuitive reasons, the more fundamental reason that baselines reduce variance is because of the following inequality

$$\text{Var}[\nabla_{\theta} \log(p_{\theta}(\tau))(R^{\gamma}(\tau) - b(\tau))] \leq \text{Var}[\nabla_{\theta} \log(p_{\theta}(\tau))R^{\gamma}(\tau)].$$

This inequality holds when $R^{\gamma}(\tau)$ and $b(\tau)$ are correlated since for any random variable X and Y , it is known that

$$\text{Var}[X - Y] = \text{Var}[X] - (2\text{Cov}[X, Y] - \text{Var}[Y]).$$

It can be seen that if $2\text{Cov}[X, Y] > \text{Var}[Y]$ (i.e., X and Y correlate), then $\text{Var}[X - Y] \leq \text{Var}[X]$. We start concretizing $b(\tau)$ here. In practice, $b(\tau) := (b(s_1), \dots, b(s_T))$ is implemented as

- **Weighted average:** $b(s_t) = \frac{\mathbb{E}[\|\nabla_{\theta} \log \pi_{\theta}(a|s_t)\|_2^2 R(\tau_{t:T})]}{\mathbb{E}[\|\nabla_{\theta} \log \pi_{\theta}(a|s_t)\|_2^2]}$
- **Value function:** $V^{\pi}(s_t) = \mathbb{E}_{\pi_{\theta}}[R^{\gamma}(\tau_{t:T})]$

The weighted average can be easily calculated from the samples of trajectories. However, approximating the value function V^{π} is a bit tricky. A naive approach to obtain the value function V^{π} is parameterizing it as V_{ψ}^{π} , where ψ denotes the parameters of a model, and optimize ψ with respect to

$$\min_{\psi} \mathbb{E}[\|R^{\gamma}(\tau_{t:T}) - V_{\psi}^{\pi}(s_t)\|_2^2], \quad (11)$$

where $R^{\gamma}(\tau_{t:T})$ is termed as Monte Carlo (MC) estimation of expected return of a trajectory starting from s_t . We will cover another more commonly used approach, temporal difference (TD) [Tesauro \[1995\]](#) learning, for approximating the value function in the later sections.

Increase sample size The number of trajectories, N , is the sample size of the empirical mean of policy gradients. Trivially increasing N can make the estimated policy gradients closer to the expected policy gradients.

$$\sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi(a|s_t^i) R^{\gamma}(\tau_{t:T}^i) \approx \mathbb{E} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi(a|s_t^i) R^{\gamma}(\tau_{t:T}^i) \right]$$

Nevertheless, the challenge is that increasing N entails sampling a large number of trajectories with the current policy π , which is often impractical in the real world, especially in robotics applications.

4.4 Actor-critic

Actor-critic methods [[Konda and Tsitsiklis, 1999](#)] are introduced to reduce variance of policy gradient. The idea of actor critic is to use the value function (see [Section 4.3](#)) to replace the MC estimates of trajectory returns $R^{\gamma}(\tau)$. In this section, we introduce a commonly used actor-critic method, **Advantage Actor Critic (A2C)** [[Mnih et al., 2016](#)]³, and an advantage estimation method, **Generalized Advantage Estimation (GAE)** [[Schulman et al., 2015a](#)], that balances the bias-variance trade-off between MC estimation and value predictions.

³The initial version of advantage actor critic proposed in [[Mnih et al., 2016](#)] is abbreviated as A3C because of the use of asynchronous sampling, but later on, it is shown that the synchronous version, A2C, attained similar results

4.4.1 Advantage Actor-Critic (A2C)

To reduce the variance of policy gradients, A2C smooths the estimates of trajectory returns $R^\gamma(\tau)$ by averaging. The reason to smooth trajectory returns is that it is unlikely to obtain samples of trajectory returns from exactly the same state. Consider the modified page recommendation example. It is unlikely to serve the same user (i.e., state s) many times. As a result, our system will not be able to observe sufficient samples of trajectory returns $R^\gamma(\tau)$ starting from state s . Taking the average trajectory returns starting from similar states (i.e., those with similar features) will reduce the variance of estimating the trajectory return starting from the state s . This approach is likely to be an effective proxy estimate to $R^\gamma(\tau)$, since similar states will likely lead to a similar return at the end of trajectories.

To smooth the estimates of $R^\gamma(\tau_{t:T})$, A2C replaces the MC estimates of trajectory returns $R^\gamma(\tau_{t:T})$ starting from a state s_t with

$$R^\gamma(\tau_{t:T}) = Q^\pi(s_t, a_t) = \mathbb{E}\left[\sum_{i=t}^T \gamma^{i-t} r(s_i, a_i) | s_t, a_t\right],$$

where $Q^\pi(s_t, a_t)$ denotes the expected return of taking action a_t at state s_t , and $\mathbb{E}[\cdot | s_t, a_t]$ means that s_t and a_t are fixed and not random variables in the expectation. $Q^\pi(s_t, a_t)$ can be obtained from the value function baseline V^π (see Section 4.3) by constructing Q^π in a recursive way

$$\begin{aligned} Q^\pi(s_t, a_t) &= \mathbb{E}\left[\sum_{i=t}^T \gamma^{i-t} r(s_i, a_i) | s_t, a_t\right] = \mathbb{E}[r(s_t, a_t) + \gamma V^\pi(s_{t+1})] \\ &\approx \mathbb{E}[r(s_t, a_t) + \gamma V_\psi^\pi(s_{t+1})] \quad (V^\pi \approx V_\psi^\pi). \end{aligned}$$

The objective of policy gradient with baselines is then rewritten as the following

$$\begin{aligned} &\sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi(a | s_t^i) (R^\gamma(\tau_{t:T}) - V_\psi^\pi(s_t)) \quad (\text{Policy gradient with baselines}) \\ &= \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi(a | s_t^i) \underbrace{(r(s_t, a_t) + \gamma V_\psi^\pi(s_{t+1}) - V_\psi^\pi(s_t))}_{Q^\pi(s_t, a_t)} \quad (\text{Remove MC estimates of } R^\gamma(\tau_{t:T})) \\ &= \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi(a | s_t^i) A(s_t, a_t) \quad (A(s_t, a_t) = Q^\pi(s_t, a_t) - V_\psi^\pi(s_t)) \end{aligned}$$

Because $Q^\pi(s_t, a_t) - V_\psi^\pi(s_t)$ can be interpreted as the “advantage” of the decision a_t at s_t over the average return of trajectories starting from s_t , this method is termed as advantage actor critic (A2C), where the actor and the critic are referred to the policy π and the value function V_ψ^π , respectively. Though A2C largely removes variance from MC estimation, the smoothed trajectory return estimates are biased estimates of the expected trajectory returns $\mathbb{E}[R^\gamma(\tau_{t:T})]$ because the value prediction $V_\psi^\pi(s_{t+1})$ is an approximation.

Before the formal argument, let’s build intuition about the variance and bias. MC estimation of $R^\gamma(\tau_{t:T})$ has high randomness since it is sampled from the environment, while the randomness of $r(s_t, a_t) + \gamma V_\psi^\pi(s_{t+1})$ only comes from $r(s_t, a_t)$ since $V_\psi^\pi(s_{t+1})$ is a deterministic function. Eliminating randomness from the objective reduces variance while increasing bias since the approximated value function V_ψ^π always has approximation error from the underlying true value.

With this intuitive reason in mind, let's try arguing it formally. In statistics, we define an estimate h is unbiased if

$$0 = \mathbf{Bias}(h) = \mathbb{E}[X - h(X)] = \mathbb{E}[X] - \mathbb{E}[h(X)]$$

where X denotes a random variable and $h(X)$ is our estimates to X . In this case, X corresponds to trajectory return $R^\gamma(\tau_{t:T})$ and $h(X)$ is our estimates to $R^\gamma(\tau_{t:T})$. To see if MC estimation $\frac{1}{N} \sum_{i=1}^N R^\gamma(\tau_{t:T}^i)$ is unbiased, we inspect its expectation

$$\mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N R^\gamma(\tau_{t:T}^i)\right] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[R^\gamma(\tau_{t:T}^i)] = \frac{1}{N} \cdot N \mathbb{E}[R^\gamma(\tau_{t:T}^i)] = \mathbb{E}[R^\gamma(\tau_{t:T}^i)].$$

As the expectation of MC estimation equals to the true expected trajectory returns $\mathbb{E}[R^\gamma(\tau_{t:T}^i)]$, MC estimation is unbiased. On the other hand, the smoothed estimate in A2C is biased

$$\mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N (r(s_t^i) + \gamma V_\psi^\pi(s_{t+1}^i))\right] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[r(s_t^i, a_t^i)] + \gamma \frac{1}{N} \sum_{i=1}^N \mathbb{E}[V_\psi^\pi(s_{t+1}^i)] \neq \mathbb{E}[R^\gamma(\tau_{t:T}^i)],$$

since approximation error $\|R^\gamma(\tau_{t:T}^i) - V_\psi^\pi(s_t)\|$ is unlikely to be 0 and will be especially large in the beginning of training.

4.4.2 Generalized Advantage Estimation (GAE)

The key idea of GAE is to balance the bias and variance of advantage estimates by mixing MC and approximated estimation. Consider the approximated estimation $r(s_t, a_t) + \gamma V_\psi^\pi(s_{t+1})$. One way to suppress variance of V_ψ^π in advantage estimation $A(s_t, a_t)$ is incorporating more steps of rewards sampled from the environment. Let's start by incorporating one more step to the approximated estimation.

$$A^1(s_t, a_t) = r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \gamma^2 V_\psi^\pi(s_{t+2}) - V_\psi^\pi(s_t).$$

The value predictions $V_\psi^\pi(s_{t+2})$ is down-weighted in $A^1(s_t, a_t)$ and thus, the the variance of A^1 is reduced by discounting (similar to the approach shown in Section 4.3). Similarly, we can add more steps of rewards and define n -step advantage as the follows

$$\begin{aligned} A^0(s_t, a_t) &= r(s_t, a_t) + \gamma V_\psi^\pi(s_{t+1}) - V_\psi^\pi(s_t) \\ A^1(s_t, a_t) &= r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \gamma^2 V_\psi^\pi(s_{t+2}) - V_\psi^\pi(s_t) \\ A^2(s_t, a_t) &= r(s_t, a_t) + \gamma r(s_{t+1}, a_{t+1}) + \gamma^2 r(s_{t+2}, a_{t+2}) + \gamma^3 V_\psi^\pi(s_{t+3}) - V_\psi^\pi(s_t) \\ &\dots \\ A^n(s_t, a_t) &= \sum_{i=1}^n \gamma^{i-1} r(s_{t+i}, a_{t+i}) + \gamma^{n+1} V_\psi^\pi(s_{t+n+1}) - V_\psi^\pi(s_t) \end{aligned}$$

The question coming along is how to select n . Instead of picking one n , GAE proposes to use an exponential weighted average to mix all $A^i(s_t, a_t)$ with a parameter λ and arrive at the following concise expression of mixed advantage

$$A^{GAE(\gamma, \lambda)}(s_t, a_t) = \sum_{i=1}^T (\gamma \lambda)^i A^i(s_t, a_t),$$

where λ controls the bias and variance. When $\lambda = 0$, GAE only uses 1-step advantage, while when $\lambda = 1$, GAE fully uses MC estimation. The derivation from weighted average to the expression of $A^{GAE(\gamma, \lambda)}(s_t, a_t)$ above is trivial and can be found in Equation (16) in [Schulman et al. \[2015a\]](#).

4.5 Exploration and Data Diversity

As we have discussed in Section 4.3, using a large number of trajectories is critical to obtaining a low-variance estimate of returns. Recent works typically employ (i) asynchronous sampling and (ii) entropy regularization to improve data diversity.

Asynchronous sampling The paper introducing A2C [Mnih et al., 2016] found that collecting trajectories using a single policy results in highly correlated data and consequently a biased estimate of the gradient. Therefore, they proposed to collect trajectories using multiple copies of the same policy operating asynchronously. Each copy of the policy is said to be run by a *worker*. A3C consists of one master and multiple workers collecting trajectories. This framework of multiple workers collecting data to compute the policy gradient was referred to as Asynchronous Advantage Actor-Critic (or **A3C**). The first obvious benefit of using multiple workers is increased sample size. In addition, the diversity of data is also improved due to asynchronous workers. Each worker fetches the master's policy model parameters asynchronously, computes the policy gradients using the collected trajectories, and sends the gradient back to the master thread. As each worker fetches policy models at different time points, the trajectories collected by each worker are different and avoid biased gradient estimation. The master policy asynchronously aggregates the gradients from the workers and updates the policy models. A natural question from readers might be the correctness of gradients estimated by policies at time different time points. In Recht et al. [2011], it has been shown that asynchronously estimated gradients can still lead to satisfactory performance under certain conditions.

Entropy regularization Mnih et al. [2016] proposed to regularize the policy gradients with the entropy⁴ of the policy to increase the diversity of actions executed by the agent. The regularized objective is:

$$\sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi(a|s_t^i) A(s_t, a_t) + \mathcal{H}(\pi(a|s_t)), \quad (12)$$

where $\mathcal{H}(\pi(a|s_t))$ is defined as

$$\mathcal{H}(\pi(a|s_t)) := - \sum_{a \in \mathcal{A}} \pi(a|s_t) \log \pi(a|s_t).$$

4.6 Conservative Policy Optimization

Recall the vicious cycle we discussed in the lecture. The challenge of learning with policy gradient is that the policy affects data collection. A suboptimal policy is unlikely to gather useful data for updating our policy. For example, in the modified landing page recommendation example, if page recommendation is terrible and users never spent money, there is no useful feedback to improve the policy. Once turning into a suboptimal policy, it is difficult for the agent to improve the policy due to lack of rewarding trajectories. One strategy to avoiding this is by being conservative in updating our policy such that our new policy is similar in performance to our old policy. The intuition behind this is that if our previous policy is able to collect rewarding trajectories, a small update is less likely to make the

⁴In information theory, entropy quantifies the randomness of a probability distribution.

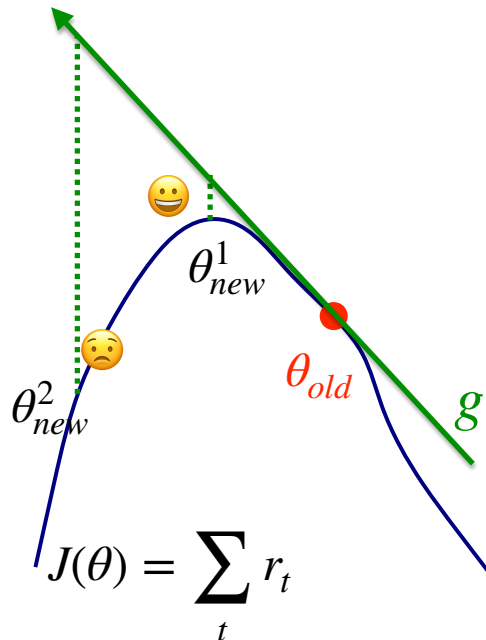


Figure 2: Step size is important. An excessively large step size can lead the policy update to low-return regimes. The black curve denote policy's returns.

policy suboptimal. Computationally, as shown in Figure 2, one view of conservative policy optimization is selecting a gradient step wisely to make sure the new policy's performance is close to the old policy. However, the question is *how do we choose such a gradient step?* We will start by motivating conservative policy optimization by a naive parameter space method in Section 4.6.1, and then cover more principled approaches in Section 4.6.2 and Section 4.6.3.

4.6.1 Parameter space constraint

On one hand, updating the policy conservatively (i.e., small gradient updates) can prevent overshooting but also substantially slows down the learning process making the algorithm inefficient, which in turn, makes the algorithm inefficient. Instead of updating parameters by an extremely small gradient step, one method that considers maximizing the objective value while prevents excessive updates is to reformulate policy gradient objective with a constraint on parameter changes (i.e., model parameters θ):

$$\begin{aligned} \max_{\theta} J(\theta) \\ \text{s.t.}, \|\theta\|_2^2 \leq \epsilon. \end{aligned}$$

Optimizing this objective results in a solution that maximizes the returns while not excessively changing the parameters. To solve this problem, we can recast it to

$$\begin{aligned} \max_d J(\theta + d) \\ \text{s.t.} \ \|d\|_2^2 \leq \epsilon, \end{aligned}$$

where d denotes the parameter difference after and before the update. As this is a nonlinear optimization problem, we use Lagrangian method [Bertsimas and Tsitsiklis, 1997] to relax the constraint and turn the problem to

$$\max_d J(\theta + d) - \lambda(\|d\|_2^2 - \epsilon).$$

Next, since $J(\theta + d)$ is unknown⁵, we use Taylor expansion to approximate $J(\theta + d)$

$$J(\theta + d) \approx J(\theta) + d^T \nabla J(\theta) + O(d^2),$$

where $O(d^2)$ is a second-order term that we will ignore. Using the first order approximation to $J(\theta + d, \lambda)$, we obtain

$$J(\theta + d, \lambda) \approx J(\theta) + d^T \nabla J(\theta) - \lambda(\|d\|_2^2 - \epsilon).$$

At the final step of solving for d , we set the derivative of $J(\theta + d, \lambda)$ with respect to d as 0

$$0 = \nabla J(\theta) - 2\lambda d,$$

and then we arrive at the optimal solution,

$$d^* = \frac{1}{2\lambda} \nabla J(\theta).$$

which is the same as performing vanilla gradient descent. In addition to optimality with respect to the objective function $J(\theta + d, \lambda)$, we also need to make sure the step size β of gradient update satisfies the constraint:

$$\|\beta d^*\|_2^2 \leq \epsilon,$$

which implies the largest $\beta = \sqrt{\epsilon / \|d^*\|_2^2}$. Therefore, the resulting gradient ascent update is:

$$\theta \leftarrow \theta + \sqrt{\frac{\epsilon}{\|d^*\|_2^2}} d^*.$$

However, picking a threshold ϵ to enforce policy output changes to be in a desired range can be tricky because the relationship between changes in parameters and policy output depends on the parameterization of θ . We show that with the same threshold ϵ , policy output changes are different with different parameterizations. Suppose we have a function $f(x, \theta) = \theta^T x$, $x \in \mathbb{R}^2$, which is parameterized in two different ways: $f(x, \theta^a) = \sum_{i=1}^{10} \theta_i^a x_1 + \theta_{11}^a x_2$ and $f(x, \theta^b) = \theta_1 x_1 + \theta_2 x_2$, where $\theta^a \in \mathbb{R}^{11}$ and $\theta^b \in \mathbb{R}^2$. The only difference in these two parameterizations is that feature x_1 is repeated ten times in the first case. Using the above derivation of d^* with $\lambda = 1$ and $x_1 = x_2 = c$, we obtain gradient update expressions for θ^a and θ^b :

$$\begin{aligned} \theta_{i+1}^a &\leftarrow \theta_i^a + \sqrt{\frac{\epsilon}{11c^2}}(c, c, \dots, c, c) \\ \theta_{i+1}^b &\leftarrow \theta_i^b + \sqrt{\frac{\epsilon}{2c^2}}(c, c), \end{aligned}$$

where i denotes the iterations of gradient updates. Let $\theta_1^a = (0, \dots, 0, 0)$ and $\theta_1^b = (0, 0)$, and thus $f(x, \theta_1^a) = f(x, \theta_1^b) = 0$. After one gradient update, we get $\theta_2^a = \sqrt{\frac{\epsilon}{11c^2}}(c, c, \dots, c, c)$ and $\theta_2^b = \sqrt{\frac{\epsilon}{2c^2}}(c, c)$. As a result, with $x_1 = x_2 = c$, the outputs $f(x, \theta_2^a) = \sqrt{\frac{\epsilon}{11c^2}}c^2$ and $f(x, \theta_2^b) = \sqrt{\frac{\epsilon}{2c^2}}c^2$ differ. Even though both gradient updates satisfy the constraint $\|\beta d^*\|_2^2 \leq \epsilon$, their output changes are different depending on the parameterizations.

⁵To evaluate $J(\theta + d)$, we would need to use the updated policy $\pi_{\theta+d}$ to collect trajectories, which is costly and undesired.

4.6.2 Parameterization-independent constraints on output space

To remove the parameterization dependency on the constraints, **Natural Policy Gradient (NPG)** [Kakade, 2001] employs the idea of natural gradient [Amari, 1998] to construct a parametrization-independent constraint on the expected outputs $\mathbb{E}[\log \pi_\theta(a|s)]$. We informally motivate the idea of NPG by inspecting the constraint of squared gradient norm again (in vector form and element form):

$$\begin{aligned} d^T d &\leq \epsilon && \text{(Vector form)} \\ \sum_{i,j} \mathbf{I}_{ij} d_i d_j &\leq \epsilon && \text{(Element form),} \end{aligned}$$

where $\mathbf{I}$ denotes an identity matrix. As gradient vector d has unit of infinitesimal changes on parameters $\partial\theta$ in each dimension, $\sum d_i d_j$ has unit $(\partial\theta)^2$. Thus, this constraint effectively says that “ $\|d\|_2^2$ must be smaller than ϵ ” instead of “ $\|\partial \log \pi_\theta(a|s)\|_2^2$ must be smaller than ϵ ”, where $\partial \log \pi_\theta(a|s)$ denotes infinitesimal changes of log-likelihood of the policy. If our target is to put constraint on $\log \pi_\theta(a|s)$ (i.e., output), then one strategy is to make the constraint have the unit $\partial \log \pi_\theta(a|s)$. Therefore, NPG modifies the constraint as:

$$\begin{aligned} d^T F d &\leq \epsilon && \text{(Vector form)} \\ \sum_{i,j} \frac{\partial \log \pi_\theta(a|s)}{\partial \theta_i} \frac{\partial \log \pi_\theta(a|s)}{\partial \theta_j} d_i d_j &\leq \epsilon && \text{(Element form),} \end{aligned}$$

where $\mathbf{F}(\theta)$ is a *Fisher information matrix*

$$\mathbf{F}(\theta) := \mathbb{E}_{\pi_\theta(a|s)} [\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^T].$$

As $\frac{\partial \log \pi_\theta(a|s)}{\partial \theta_i} \frac{\partial \log \pi_\theta(a|s)}{\partial \theta_j} d_i d_j$ has unit as shown below,

$$\frac{\partial \log \pi_\theta(a|s) \partial \log \pi_\theta(a|s) \partial \theta_i \partial \theta_j}{\partial \theta_i \partial \theta_j} = (\partial \log \pi_\theta(a|s))^2,$$

the constraint turns out to enforce the square of log-likelihood changes of the policy to be smaller than ϵ . While we motivate the use of Fisher information matrix from the units of quantities in the constraint, there is a rigorous analysis in information geometry that justifies Fisher information matrix as the only way to construct a parameterization-invariant metric between two probability distributions. We will not cover the details about information geometry in these lecture notes as they entails plenty of background beyond the scope of this class. For detailed discussion, please refer to Amari [1998], Kakade [2001] and Chentsov’s theorem.

Putting the constraint $d^T \mathbf{F}(\theta) d \leq \epsilon$ to policy optimization gives rise to NPG

$$\begin{aligned} \max_d \quad & J(\theta + d) \\ \text{s.t.} \quad & d^T \mathbf{F}(\theta) d \leq \epsilon. \end{aligned}$$

To solve this constrained optimization problem, similarly to the recipes in Section 4.6.1, we first relax this problem by Lagrangian method:

$$J(\theta + d, \lambda) \approx J(\theta) + d^T \nabla J(\theta) - \lambda \left(\frac{1}{2} d^T \mathbf{F}(\theta) d - \epsilon \right),$$

then set gradient w.r.t. d of $J(\theta + d, \lambda)$ to 0,

$$0 = \nabla J(\theta) - \lambda \mathbf{F}(\theta) d,$$

and obtain the optimal solution to d :

$$d^* = \lambda \mathbf{F}(\theta)^{-1} \nabla J(\theta).$$

The next step is to find the best step size β that make the constraint $d^T \mathbf{F}(\theta) d \leq \epsilon$ tighten. We solve β by first plugging d^* to the constraint

$$\begin{aligned} & \frac{\lambda^2}{2} (d^*)^T \mathbf{F}(\theta) d^* \\ &= \frac{\lambda^2}{2} (\beta \mathbf{F}(\theta)^{-1} \nabla J(\theta))^T \mathbf{F}(\theta) (\beta \mathbf{F}(\theta)^{-1} \nabla J(\theta)) \\ &= \frac{\lambda^2 \beta^2}{2} \nabla J(\theta)^T \mathbf{F}(\theta)^{-1} \nabla J(\theta) \\ &= \frac{\lambda^2 \beta^2}{2} \nabla J(\theta)^T \mathbf{F}(\theta)^{-1} \nabla J(\theta) \leq \epsilon, \end{aligned}$$

and finding a β that tightens the inequality for

$$\beta^* = \frac{1}{\lambda} \sqrt{\frac{2\epsilon}{\nabla J(\theta)^T \mathbf{F}(\theta)^{-1} \nabla J(\theta)}}.$$

Connection to Kullback–Leibler (KL) divergence As the second-order Taylor expansion of KL-divergence is Fisher information matrix, the optimization problem of NPG can be rewritten with KL-divergence as:

$$\begin{aligned} & \max_d J(\theta + d) \\ & s.t. \ D_{KL}(\pi_\theta || \pi_{\theta+d}) \leq \epsilon, \end{aligned} \tag{13}$$

where D_{KL} denotes KL-divergence. For second-order Taylor expansion of KL-divergence, please refer to Section B.

Connection to Newton method Readers familiar with Newton method might find natural gradient descent has the similar form in that

$$\begin{aligned} \theta &\leftarrow \theta + \beta \mathbf{H}^{-1}(\theta) \nabla J(\theta) && \text{(Newton method)} \\ \theta &\leftarrow \theta + \beta \mathbf{F}^{-1}(\theta) \nabla J(\theta) && \text{(Natural gradient),} \end{aligned}$$

where $\mathbf{H}$ is a Hessian matrix and $\mathbf{H}_{ij}(\theta) = \frac{\partial^2 J(\theta)}{\partial \theta_i \partial \theta_j}$. Despite the similar form, it should be noted that Fisher information matrix $\mathbf{F}(\theta)$ is not equivalent to Hessian matrix $\mathbf{H}(\theta)$ w.r.t. $J(\theta)$. The short reason is that in Newton method, Hessian matrix is a matrix of second-order derivative (i.e., curvature) of the objective function $J(\theta)$, while Fisher matrix is curvature of KL-divergence $D_{KL}(\pi_\theta || \pi_{\theta+d})$. For the detailed discussion, please refer to Kakade [2001].

4.6.3 Monotonic Policy Improvement

While NPG ensures the change in policy is bounded by ϵ , it does not guarantee that each policy update will improve the policy performance measured by the returns. **Trust Region Policy Optimization (TRPO)** [Schulman et al., 2015b] inspired by *Conservative Policy Iteration* [Kakade and Langford, 2002] mitigates this issue by maximizing policy improvement instead of just constraining changes in the policy outputs. If policy improvement is guaranteed, then the agent will not get stuck in poor local maxima due to an erroneous

updates and therefore will avoid the vicious cycle problem described in Section 4.6. The trust region approach solves the following optimization problem:

$$\max_d J(\theta + d) - J(\theta), \quad (14)$$

where $J(\theta + d) - J(\theta)$ quantifies the performance improvement of $\pi_{\theta+d}$ with respect to the old policy π_θ . $\theta + d$ and θ denote the parameters of new and old policies respectively. Note that for brevity, we will abbreviate π_θ as π . [Kakade and Langford \[2002\]](#) show that policy improvement $J(\theta + d) - J(\theta)$ can be rewritten as:

$$J(\theta + d) - J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta+d}} \left[\sum_{t=1} \gamma^{t-1} A^{\pi_\theta}(s_t, a_t) \right].$$

With this identity, we can restate the optimization problem in Equation 14 as:

$$\max_d \mathbb{E}_{\tau \sim \pi_{\theta+d}} \left[\sum_{t=1} \gamma^{t-1} A^{\pi_\theta}(s_t, a_t) \right]. \quad (15)$$

This objective says that as long as the new policy $\pi_{\theta+d}$ results in positive increase in expected sum of advantages (measured according to π), the performance of the new policy is guaranteed to be superior. To directly see the dependence on d , we rewrite Equation 15 as:

$$\begin{aligned} \mathbb{E}_{\tau \sim \pi_{\theta+d}} \left[\sum_t \gamma^{t-1} A^{\pi_\theta}(s_t, a_t) \right] &= \sum_{s,a} \sum_t P(s = s_t, a = a_t | \pi_{\theta+d}) [\gamma^{t-1} A^{\pi_\theta}(s, a)] \\ &= \sum_t \sum_{s,a} \gamma^{t-1} P(s = s_t, a = a_t | \pi_{\theta+d}) [A^{\pi_\theta}(s, a)] \\ &= \sum_t \sum_s \gamma^{t-1} P(s_t = s | \pi_{\theta+d}) \sum_a \pi_{\theta+d}(a_t = a | s_t) [A^{\pi_\theta}(s, a)] \\ &= \sum_s \rho_{\pi_{\theta+d}}^\gamma(s) \sum_a \pi_{\theta+d}(a | s) [A^{\pi_\theta}(s, a)] \\ &\text{(Define } \rho_{\pi_{\theta+d}}^\gamma(s) \text{ as } \sum_t \gamma^{t-1} P(s = s_t | \pi_{\theta+d})) \\ &= \sum_s \rho_{\pi_{\theta+d}}^\gamma(s) \sum_a \pi_\theta(a | s) \frac{\pi_{\theta+d}(a | s)}{\pi_\theta(a | s)} [A^{\pi_\theta}(s, a)] \\ &= \mathbb{E}_{s \sim \pi_{\theta+d}, a \sim \pi_\theta} \left[\frac{\pi_{\theta+d}(a | s)}{\pi_\theta(a | s)} A^{\pi_\theta}(s, a) \right] \end{aligned}$$

where $P(\cdot)$ denotes the probability of occurrence $(\cdot)$ in a trajectory τ and $\rho_{\pi_{\theta+d}}^\gamma$ ⁶ is the discounted state-visitation frequency of policy $\pi_{\theta+d}$. We can therefore express the objective function in Equation 15 as:

$$\max_d \mathbb{E}_{s \sim \pi_{\theta+d}, a \sim \pi} \left[\frac{\pi_{\theta+d}(a | s)}{\pi(a | s)} \gamma^{t-1} A^{\pi_\theta}(s, a) \right], \quad (16)$$

Unfortunately optimizing this objective is inefficient because it requires states s_t sampled from the new policy $\pi_{\theta+d}$. Sampling from the new policy is a costly operation requiring additional interaction with the environment. Moreover, since we are searching for d , new data will need to be collected for every d which makes the process even more expensive. Fortunately, Theorem 1 in [Schulman et al. \[2015b\]](#) suggests that the performance of the new policy $\pi_{\theta+d}$ can be approximated using the state distribution of the old policy π_θ . The approximated performance $\tilde{J}_\theta(\theta + d)$ bounds $J(\theta + d)$ as the follows:

$$J(\theta + d) \geq \hat{J}_\theta(\theta + d) - CD_{KL}^{max}(\pi_\theta(a | s) || \pi_{\theta+d}(a | s)),$$

⁶ $\rho_{\pi_{\theta+d}}^\gamma(s) = P(s_1 = s) + \gamma P(s_2 = s) + \gamma^2 P(s_3 = s) + \dots$

where $C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$, $\max_s D_{KL}^{max}(\pi_\theta(a|s)||\pi_{\theta+d}(a|s))$, and $\tilde{J}_\theta(\theta+d) = \mathbb{E}_{s \sim \pi_\theta, a \sim \pi_{\theta+d}}[A^{\pi_\theta}(s, a)]$. As maximizing the lower bound (right hand side) approximates $J(\theta+d)$, manipulating the objective $\hat{J}_\theta(\theta+d) - J(\theta)$ as above give rise to the following optimization problem:

$$\max_d \mathbb{E}_{s \sim \pi_\theta, a \sim \pi_\theta} \left[\frac{\pi_{\theta+d}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s, a) \right] - CD_{KL}^{max}(\pi_\theta(a|s)||\pi_{\theta+d}(a|s)). \quad (17)$$

Intuitively, TRPO made an approximation by maximizing the objective with states samples from the current policy, i.e., $s_t \sim \pi$. This assumption is valid if the state distributions induced by the new and old policies are similar. However, optimization problem 17 is hard to optimize in practice because the best step size is excessively small if the theoretically-justified penalty weight $C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$ is used.

We introduce practical algorithms to solve the constrained optimization problem in Equation 17 below.

Line search First, as evaluating $D_{KL}^{max}(\pi_\theta(a|s)||\pi_{\theta+d}(a|s))$ is intractable, TRPO use expected KL-divergence to constrain the optimization. Secondly, To enable larger step size, TRPO instead sets the KL-divergence constraint as a hard constraint as shown below

$$\begin{aligned} \max_d \mathbb{E}_{s \sim \pi_\theta, a \sim \pi_\theta} \left[\frac{\pi_{\theta+d}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s, a) \right] \\ s.t. \mathbb{E}_s [D_{KL}(\pi_\theta(a|s)||\pi_{\theta+d}(a|s))] \leq \epsilon, \end{aligned} \quad (18)$$

and searches for the optimal step size with line search. The TRPO paper solves the objective function described in Equation 18 using natural gradient ascent method covered in Section 4.6.2.⁷ In vanilla NPG, the gradient update that constraints the KL divergence, $D_{KL}(\pi_\theta(\cdot|s_t)||\pi_{\theta+d}(\cdot|s_t)) \leq \epsilon$, does not guarantee policy improvement $J(\theta+d) - J(\theta) \geq 0$. To guarantee policy improvement, when line search is performed to find a step size β , we check if the step size makes (i) performance improved and (ii) KL-divergence constraint satisfied. Let us call the (approximated) objective as $\hat{J}(\theta, d)$ written below:

$$\hat{J}(\theta, d) = \mathbb{E}_{s \sim \pi_\theta, a \sim \pi_\theta} \left[\frac{\pi_{\theta+d}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s, a) \right]$$

The procedure of line search is summarized as the follows

$$\beta \leftarrow \begin{cases} \beta * 0.5, & \text{if } D_{KL}(\pi_\theta(a|s)||\pi_{\theta+d}(a|s)) < \epsilon \\ \beta * 0.5, & \text{if } \hat{J}(\theta, d_{new}) - \hat{J}(\theta, d_{old}) < 0 \\ \beta, & \text{otherwise,} \end{cases}$$

where d_{new} and d_{old} denote the gradient vectors tested in the current and the last iteration of line search, respectively. β is initialized as the same as NPG: $\beta = \sqrt{\frac{2\epsilon}{\nabla J(\theta)^T \mathbf{F}(\theta)^{-1} \nabla J(\theta)}}$.

Adaptive Lagrangian method Nevertheless, performing line search may be expensive in determining a suitable step size, which in turn slows down the training progress in terms of wallclock time. Instead of using line search and fixed penalty weight, Schulman et al. [2017] replace D_{KL}^{max} with expected KL-divergence in the constraint, as shown below:

$$\max_d \mathbb{E}_{s \sim \pi_\theta, a \sim \pi_\theta} \left[\frac{\pi_{\theta+d}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s, a) - \lambda D_{KL}(\pi_\theta(\cdot|s)||\pi_{\theta+d}(\cdot|s)) \right], \quad (19)$$

⁷Instead of computing $F^{-1}\nabla J(\theta)$, Schulman et al. [2015b] employs conjugate gradient descent to avoid computing matrix inverse $\mathbf{F}^{-1}$.

and perform adaptive updates on λ :

$$\lambda \leftarrow \begin{cases} \lambda/2, & \delta_{KL} < \delta_{targ}/1.5 \\ \lambda \cdot 2, & \delta_{KL} > \delta_{targ} \cdot 1.5, \end{cases}$$

where $\delta_{KL} := D_{KL}(\pi_{\theta}(a|s) || \pi_{\theta+d}(a|s))$ and δ_{targ} is an arbitrary threshold value.

Clipping Removing the KL-divergence constraints, [Schulman et al. \[2017\]](#) propose to clip the objective function instead of enforcing the constraint in policy updates. This clipping method is so-called **Proximal Policy Optimization**. Let's look at the (non-approximated) objective function again

$$\mathbb{E}_{s \sim \pi_{\theta+d}, a \sim \pi_{\theta}} \left[\frac{\pi_{\theta+d}(a|s)}{\pi_{\theta}(a|s)} A^{\pi_{\theta}}(s, a) \right].$$

As $\frac{\pi_{\theta+d}(a|s)}{\pi_{\theta}(a|s)}$ weights the advantage $A^{\pi_{\theta}}(s, a)$, the idea of PPO is to penalize a new policy $\pi_{\theta+d}$ that has either excessively large or small ratio $c_t = \frac{\pi_{\theta+d}(a|s)}{\pi_{\theta}(a|s)}$. The penalization is realized by clipping c_t within pre-defined bound. The clipped objective function is stated as:

$$\mathbb{E}_{s \sim \pi_{\theta}, a \sim \pi_{\theta}} \left[\min \left\{ \frac{\pi_{\theta+d}(a|s)}{\pi_{\theta}(a|s)} A^{\pi_{\theta}}(s, a), \text{clip} \left(\frac{\pi_{\theta+d}(a|s)}{\pi_{\theta}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta}}(s, a) \right\} \right].$$

PPO empirically shows better performance than TRPO and adaptive Lagrangian method. Remarkably, PPO requires less computation (without the needs of line search). Though ϵ is still a problem specific hyperparameter to be tuned, $\epsilon = 0.2$ generally works well on standard benchmarks [[Schulman et al., 2017](#)].

4.7 Practical Considerations

Implementation matters in policy gradient methods, though it is often overlooked in textbooks. In this section, we will cover a range of critical implementation choices that largely affect performance. In the following sections, we mainly discuss the empirical results in robotic locomotion benchmarks, MuJoCo [[Todorov et al., 2012](#)], unless specified. For the details of MuJoCo benchmarks, please refer to [Duan et al. \[2016\]](#). Note that these findings could be limited to MuJoCo benchmarks and might not be applicable in other environments.

4.7.1 Code level optimization

[Engstrom et al. \[2020\]](#) shows that implementation tricks that are not presented in [Schulman et al. \[2017\]](#) is crucial for the performance of PPO. Section 3 in [Engstrom et al. \[2020\]](#) lists the implementation tricks (i.e., code-level optimization). In summary, PPO without code optimization cannot outperform TRPO, which puts doubt on the fundamental algorithmic improvements of PPO. The hidden contribution of this code-level optimization might explain the findings in, [Henderson et al. \[2018\]](#). [Henderson et al. \[2018\]](#) found that the performance of different open-sourced codebases are largely different even using the same hyperparameter settings.

4.7.2 Neural network architecture, Reward scale, and Random seed

[Henderson et al. \[2018\]](#) shows that the function class choices of policy π_θ and value function V_ψ^π largely influence the performance. Typically policy and value functions are parameterized as neural networks. However, the neural network architecture and activation function choices heavily influence the performance. Pessimistically, there is no single architecture to fit all environments, even in the MuJoCo benchmark. Additionally, they found that the performance is sensitive to the reward scale. Ideally, the optimal policy should be invariant when reward functions are constantly scaled (i.e., the optimal policies for reward functions r and $10r$ must be the same), while their findings show that policy gradient algorithms are sensitive to the scale of rewards. Also, they found that the average performance difference of different sets of random seeds⁸ are statistically significant. The sensitivity to these implementation choices could obscure the algorithmic improvements of these methods.

4.8 Rules of thumb in practice

[Andrychowicz et al. \[2020a\]](#) suggests several rules of thumb to implement policy gradient algorithms based on a large-scale empirical study. As [Andrychowicz et al. \[2020a\]](#) outlines these rules of thumbs clearly, we will not repeat them again in this lecture note. Please see [Andrychowicz et al. \[2020a\]](#) for details.

4.9 Debugging

In practice, we suggest debugging a policy gradient algorithm in the following ways

- **Visualize the policy performance at initialization:** Since the initial policy determines the first policy update, it is crucial for the policy updates and exploration in the later iterations.
- **Analyze returns at initialization:** Preferably, there should be some episodes with the non-zero return. Otherwise, the agent has no hope to improve its policy.
- **Increase batch size:** A large batch size is preferable.
- **Periodically visualize the agent behavior:** It helps identify the source of the issue – e.g., exploration or ill-designed reward function
- **Entropy of actions:** Entropy of policy indicates the randomness of the agent. It is helpful for detecting whether the agent prematurely converges to suboptimal policy or acts excessively randomly.

5 Reward Function Design

Reward function design is an open research question and important topic in SDM since reward structure largely affects the difficulty of the task in an environment. We first motivate

⁸Random seeds determines the randomness of simulation results of environments.

why reward design is a critical issue in SDM by comparing SDM and training a classifier with supervised learning

- **Supervised learning:** Given inputs x and labels y , we train a classifier f to predict y given x . That is, we solve an optimization problem shown below

$$\min_f \mathbb{E}[(y - f(x))^2],$$

where each pair of sample (x, y) generate training signal (i.e., gradients) for improving the classifier.

- **Policy gradient:** Given inputs s_t and return $R(\tau_{t:T})$, we train a policy π_θ to maximize trajectory return $R(\tau_{t:T})$:

$$\max_\theta \mathbb{E}_\tau \left[\sum_t^T \nabla_\theta \log \pi_\theta(a|s_t) R(\tau_{t:T}) \right].$$

The key difference to supervised learning is that $R(\tau_{t:T})$ is sampled by the policy. Unless the policy π_θ samples nonzero $R(\tau_{t:T})$, gradients $\nabla_\theta \log \pi_\theta(a|s_t)$ will be zeroed out. Reward design is influential in how difficult the agent can discover nonzero $R(\tau_{t:T})$ and how well the agent fit to our anticipated objective.

Nevertheless, how do we check if the designed reward function communicates the desired objective? In addition to the correctness, how do we make the agent learn the policy for our reward function easier? The following sections will discuss the challenges of designing a reward function that meets both requirements.

5.1 Challenges of Reward Function Design

Let's start from a simple reward design problem for a 2-dimensional navigation task illustrated in Figure 3. The state space $\mathcal{S}$ is (x, y) positions of the agent; the action space $\mathcal{A}$ is displacement $(\delta x, \delta y)$. The goal of the agent (white dot) is to reach the goal s_g (yellow dot). For this task, we consider the following possible reward functions.

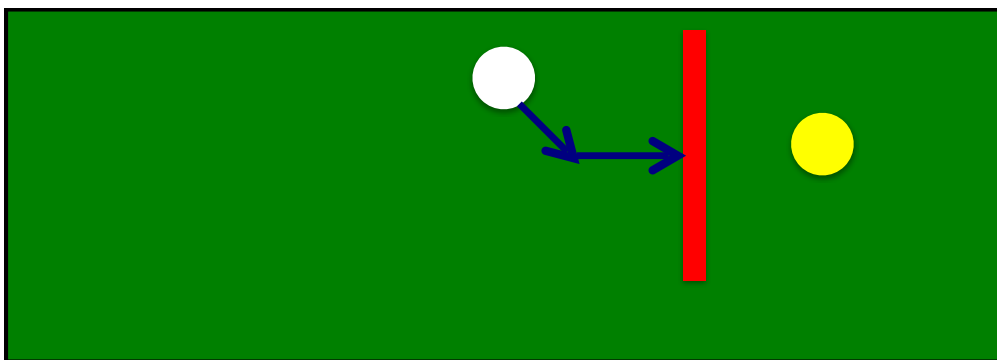


Figure 3: Illustration of 2D navigation problem. The white dot is our agent, and the yellow dot is the goal. The red bar is wall. The goal of this task is to reach the goal.

Sparse reward One option is to give the agent reward when it reaches the goal, otherwise zero. This option can be stated as the following reward function:

$$r(s, a) = \begin{cases} 1, & \mathcal{T}(s'|s, a) = s_g \\ 0, & \text{Otherwise} \end{cases}$$

This reward function correctly specifies our objective, *reaching the goal*, while the reward signals are too sparse to optimize for policy gradient. Unless the agent reaches the goal, the trajectory returns $R(\tau_{t:T})$ will constantly be zero.

Dense reward To address reward sparsity, an alternative is to use distance to the goal as reward for encouraging the agent to approach the goal. A commonly used distance-based reward function takes the negative Euclidean distance between the current state and the goal state as the rewards, which can be expressed as:

$$r(s, a) = \begin{cases} -\|s_g - s'\|_2 + 1, & s' = s_g, s' = \mathcal{T}(s'|s, a) \\ -\|s_g - s'\|_2, & \text{Otherwise} \end{cases}$$

It is easy to optimize the return of this reward function since trajectory returns $R(\tau_{t:T})$ are always nonzero values, which gives dense learning signals (i.e., most gradients are nonzero) for policy gradient. Nevertheless, this reward function is problematic since the traveling distance (more formally, geodesic distance) is greater than the euclidean distance between states because of the wall in our environment (red). For example, in Figure 3, it can be easily seen that traveling from the white dot to the yellow dot requires making a detour around the wall, where the length of detour is not equal to the euclidean distance between two points. However, if the agent maximizes this distance-based reward function, it will be guided to go straight toward the goal and hit the wall since going straight to the goal maximizes the reward fastest.

Oracle reward Using distance as reward solves the sparsity problem but raises the correctness problem. If we have geodesic distance between states, we can have both dense and correct reward functions

$$r(s, a) = \begin{cases} -D_g(s', s_g) + 1, & s' = s_g, s' = \mathcal{T}(s'|s, a) \\ -D_g(s', s_g), & \text{Otherwise} \end{cases}$$

where D_g denotes the geodesic distance between states. However, it is difficult to estimate geodesic distance between states since we require additional knowledge - either the state transition function or enough transition samples to estimate geodesic distance.

Summary We summarize our discussion of the above three reward function designs

- **Sparse reward:** It is easiest to define and gives the correct objective, while difficult to optimize due to sparsity.
- **Dense reward:** It is easy to optimize but gives the wrong objective to optimize. Also, crafting a dense reward function requires domain knowledge of the environment. In this example, we need to know this is a 2-dimensional environment and the states are x, y positions.
- **Oracle reward:** It is an ideal reward function since it gives the correct objective and is easy to optimize.

5.2 Reward Engineering for Multiple Objectives

In the last section, we consider designing a reward function that encourages the agent to reach the goal. However, in practice, we often hope the agent can achieve multiple objectives by a single reward function. What if we want the agent to reach the goal as soon as possible or avoid hitting walls? This section covers several common reward engineering techniques to design a reward function that specifies multiple objectives in a single reward function.

Achieve the goal as soon as possible Since the agent’s actions may incur costs, it is natural to hope the agent achieves the goal as soon as possible. Let’s consider again the example shown in Figure 3. In this environment, we hope the agent finds the shortest path to the goal rather than just a feasible path to the goal. Though discount factor γ can already specify this objective, discounting introduces biases in the objective (see Section 4.3). A common approach to account for this objective is introducing step penalty or action costs to the reward function, as shown below

$$r(s, a) = \begin{cases} 1, & \mathcal{T}(s'|s, a) = s_g \\ -0.1, & \text{Otherwise} \end{cases} \quad (\text{Step penalty})$$

$$r(s, a) = -\|a\|_2 \quad (\text{Action costs}).$$

A reward function with a step penalty says that the agent should reach the goal as soon as possible to stop receiving the penalty. Action costs have a straightforward connection to physical robots. If actions are motor torques, we, preferably, should prevent the robot from exerting large motor torques. Large torques could damage the robot or consume much energy.

Avoid danger In addition to efficiency of the policy, we might hope the agent to learn a safe policy that avoid danger. For example, in the example shown in Figure 3, walls could be dangerous since bumping into walls might damage robots. To account danger avoidance in the reward function, one typical approach is to add penalty on specific states:

$$r(s, a) = \begin{cases} 1, & \mathcal{T}(s'|s, a) = s_g \\ -0.3, & \mathcal{T}(s'|s, a) = \text{Walls} \\ -0.1, & \text{Otherwise.} \end{cases}$$

As the agent will receive -0.3 reward when hitting walls, the policy will be discouraged to approach walls during optimization. Note that the penalty of hitting walls has to be higher than the step penalty; otherwise, the agent will prefer hitting walls over walking on other free spaces. However, this raises the question of determining the penalty of hitting walls — how large should it be relative to the step penalty? Unfortunately, it requires experimenting with multiple values to find a suitable penalty scale. As a side note, the danger penalty will not “*guarantee*” the safety of the policy since it is impossible for the agent to know the penalty of hitting walls without hitting them.

5.3 Principles of Reward Shaping

In previous sections, we learned how to define a correct but difficult reward function (sparse reward) and shaped (i.e., reward engineering) it to a easier reward function. However, we

have not yet discussed how to verify if a shaped reward function directs to the correct optimal policy that aligns with the task's objective. To this end, [Ng et al. \[1999\]](#) suggest that as long as differences of state potential function construct a shaped reward function, the resultant optimal policy will be invariant to that from the unshaped reward function. Formally, the suggested reward function can be expressed as

$$r'(s_t, a_t) = r(s_t, a_t) + \phi(s_t) - \phi(s_{t-1}),$$

where ϕ denotes an arbitrary potential function of states (e.g., $\phi(s) = -\|s' - s\|_2$). The shaped reward function r' is unbiased and will give rise to an optimal policy invariant to the unshaped reward function since the potential difference $\phi(s') - \phi(s)$ will be canceled out when summing over time

$$\sum_{t=1}^T r'(s_t, a_t) = \sum_{t=1}^T r(s_t, a_t) - \phi(s_1) + \phi(s_T) = \sum_{t=1}^T r(s_t, a_t),$$

where we just need to ensure the initial state potential $\phi(s_1) = 0$ in the definition of ϕ . Nevertheless, this design principle has not shown practical benefits in recent research yet.

6 Learning from Demonstrations

Instead of crafting reward functions, an alternative method for task specification is to provide expert demonstrations. A naive method is to have the agent memorize the sequence of actions and repeat it. E.g., the first industrial robot, *Unimate* [[Gasparetto and Scalera, 2019](#)] was designed to memorize and repeat a sequence of motor commands. This simple method has obvious drawbacks: (a) agent cannot handle any changes in the “state”. E.g., a robot that has memorized to pick objects from a certain location will fail if the object is moved to a different position (i.e., change in state); (b) the policy is very specific to the given task.

To account for uncertainty and to adapt the policy in a state dependent manner, modern approaches of learning from demonstration (LfD) typically fit a state-conditioned policy $\pi : \mathcal{S} \mapsto \mathcal{A}$, similar to what we saw in [Section 3.2](#). The difference from policy gradients is that policy training in LfD is supervised by demonstrations instead of the reward function. It is typical to assume that agent is provided with multiple task demonstrations, $\{\tau_{1:T}^i\}_{i=1}^N$ (see [Section 3.4](#)), where τ_i denotes one demonstration trajectory. Often the demonstrations are assumed to be either optimal or nearly optimal. Before we delve into various LfD methods, we introduce some notation that is largely borrowed from [Section 3.2](#).

- **Training time:** The stage where we train the policy π_θ using demonstrations.
- **Testing (or deployment) time:** The phase where we fix the policy parameters (θ) and simply execute the predicted actions.
- **Expert policy π_D :** An expert policy that achieves optimal performance on the given task.
- **Demonstration dataset $\mathcal{D}$:** is constructed by rolling out trajectories from π_D . In some scenarios it is assumed that we have explicit access to π_D and in others only access to rollouts from the policy is assumed (e.g., a human expert only provides rollouts and not π_D itself). We define $\mathcal{D} := \{\tau_{1:T}^i\}_{i=1}^N$, where N and T denote the number of

trajectories and length of a trajectory, respectively. Note that T can be varied across trajectories, but we assume it to be a constant for convenience of notation.

The general outline of LfD framework is outlined in Algorithm 5 below. The TEST procedure

Algorithm 5 Learning from demonstration (LfD)

- 1: **Input:** Demonstration dataset $\mathcal{D}$ and/or expert policy π_D
 - 2: **procedure** TRAINANDTEST
 - 3: $\pi_{\theta^*} \leftarrow \text{TRAIN}(\pi_{\theta}, \mathcal{D}, \pi_D)$
 - 4: TEST(π_{θ^*}) ▷ Rollout some episodes in the environment
 - 5: **end procedure**
-

rollouts the policy π_{θ} and measures average return across multiple episodes.

6.1 Behavior cloning

One strategy to learn policy parameters from demonstrations is to match the distributions of trajectories generated by π_{θ} and π_D . This strategy known as **Behavior Cloning (BC)** can be formulated as minimizing KL-divergence⁹ from π_{θ} to π_D , as shown below:

$$\min_{\theta} L(\theta), \quad L(\theta) = D_{KL}(P_D || P_{\theta}),$$

where P_{θ} and P_D denote the distributions of trajectories of π_{θ} and π_D , respectively. This optimization problem can be again solved by gradient descent with the gradient $\nabla L(\theta)$. To see the connection of the gradient $\nabla L(\theta)$ to π_{θ} , we define the probability of a trajectory τ as $P(\tau)$ (P can be P_D or P_{θ}):

$$\begin{aligned} P(\tau) &= P(s_1, a_1, \dots, s_T) \\ &= P(s_1)P(a_1|s_1) \cdots P(a_{T-1}|s_1, a_1, \dots, s_{T-1})P(s_T|s_1, a_1, \dots, a_{T-1}) && \text{(By Bayes rule)} \\ &= P(s_1) \prod_{t=1}^T P(s_t|s_1, a_1, \dots, a_{t-1})P(a_t|s_1, a_1, \dots, s_t) \\ &= P(s_1) \prod_{t=1}^T P(s_t|s_{t-1}, a_{t-1})P(a_t|s_t) && \text{(Assume Markov property)} \\ &= \rho(s_1) \prod_{t=1}^T \mathcal{T}(s_t|s_{t-1}, a_{t-1})\pi(a_t|s_t) \end{aligned}$$

where ρ denotes an initial state distribution. $\nabla J(\theta)$ can then be expressed as (see Section C for details):

$$\begin{aligned} \nabla_{\theta} L(\theta) &= \nabla_{\theta} D_{KL}(P_D || P_{\theta}) \\ &= \nabla_{\theta} \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) \log \frac{P_D(s_1, a_1, \dots, s_T)}{P_{\theta}(s_1, a_1, \dots, s_T)} \right) \\ &= \nabla_{\theta} \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) (\log P_D(s_1, a_1, \dots, s_T) - \log P_{\theta}(s_1, a_1, \dots, s_T)) \right) \\ &= -\mathbb{E}_{\tau \sim D} \left[\nabla_{\theta} \log P_{\theta}(s_1, a_1, \dots, s_T) \right] \quad \text{(Hint: Applying log to } P(\tau) \text{ and take gradient w.r.t. } \theta) \\ &= -\mathbb{E}_{\tau \sim D} \left[\sum_{t=1}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right] \\ &= -\mathbb{E}_{s, a \sim D} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) \right]. \end{aligned}$$

⁹There are also other perspectives to motivate BC, but here we choose the one closer to what we discussed in previous sections.

Note that the objective $L(\theta)$ is equivalent to expected negative log-likelihood $-\mathbb{E}_{s,a \sim \mathcal{D}} [\nabla_{\theta} \log \pi_{\theta}(a|s)]$. Thus, minimizing $L(\theta)$ can also be interpreted as performing maximum likelihood estimation (MLE) (or supervised learning). From MLE perspective, BC can be interpreted as finding the optimal policy parameters θ^* defined as:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{s,a \in \mathcal{D}} [\log \pi_{\theta}(\hat{a} = a|s)]$$

where $\hat{a}$ is the predicted action. Depending on the action space $\mathcal{A}$, policy π_{θ} is parameterized in different distributions, where we detail the choices of policy parametrization in Section D. In summary, BC can be formulated as the following optimization problems:

$$\max_{\theta} J(\theta), \text{ where } J(\theta) := \mathbb{E}_{s,a \sim \mathcal{D}} [\log \pi_{\theta}(\hat{a} = a|s)], \quad (20)$$

where $\hat{a}$ denotes the predicted action of π_{θ} .

Though training a policy π_{θ} by BC accounts for state changes, deploying such a policy to the environment could be still problematic because the policy π_{θ} might “drift” from the experts’ trajectories. For example, consider we are training a self-driving car to stay at the centerline of the road, where the agent controls the steering direction. Because of approximation errors of π_{θ} , π_{θ} is unlikely to perform the same as the expert’s policy. Consequently, the agent could make mistakes and deviates from the centerline. Moreover, the errors resulting from this mistake will be compounded in future time steps. Since the expert never deviates from the centerline, there are no states s at the off-center line in the demonstration $\mathcal{D}$. As such, the agent’s policy π_{θ} unlikely makes an accurate action to return to the center due to lack of demonstration in such situations. In this example, the problem results from the distribution shift in inputs. The distribution of inputs (i.e., states) at training time is different from that at deployment time (or testing time), which degrades the performance of the learned policy at deployment time. This problem is called *co-variate shift* in machine learning.

To address the co-variate shift problem, [Ross et al. \[2011\]](#) proposed **DAGger (Dataset Aggregation)** where states s are collected by the agent’s policy π_{θ} and relabelled by the expert’s policy π_D . Collecting states by π_{θ} makes the distribution of states between training and testing time closer. However, as the agent’s policy does not perform well initially, the expert’s action labels on states collected by π_{θ} are unlikely to inform the agent about how to achieve the task. As a result, DAGger performs the training process iteratively. For each iteration of training, the first step is to collect trajectories of states by π_{θ} and label the action of each s_t by π_D . The resulting dataset is $\hat{\mathcal{D}} = \{(s, \pi_D(s))\}$. At the second step, DAGger simply performs gradient ascent to optimize the objective function $J(\theta)$. As the training progresses, the distribution of states will be closer to that from the expert’s policy while the learned policy can predict the correct action at states s_t absent in the expert’s demonstration $\mathcal{D}$ due to dataset aggregation. The objective of DAGger can be expressed as:

$$\max_{\theta} J(\theta), \text{ where } J(\theta) := \mathbb{E}_{s,a \sim \pi_{\theta}} [\log \pi_{\theta}(\hat{a} = a|s)].$$

We outline the pseudocode of DAGger in Algorithm 6.

Analysis The DAGger paper [[Ross et al., 2011](#)] shows that the learned policy of DAGger has lower cost bound than that of BC. The cost of DAGger is bounded by $O(\epsilon T)$ while that of BC has a higher cost bound $O(\epsilon T^2)$, where $\epsilon \in [0, 1]$. The analysis is detailed below. First, as the cost happens when the $\pi_{\theta}(s) \neq \pi_D(s)$, we construct the bound for the probability of

Algorithm 6 Learning from demonstration (LfD)

```

1: procedure TRAIN( $\pi_\theta, \mathcal{D}, \pi_E$ )                                 $\triangleright$  Expert demonstration  $\mathcal{D}$  is not used
2:   Initialized  $\mathcal{D}$  as  $\emptyset$ 
3:   for  $i = 1 \dots$  do
4:      $\hat{\mathcal{D}} = \{(s, \pi_D(s))\} \leftarrow \text{ROLLOUT}(\pi_\theta)$            $\triangleright$  Sample trajectories of states with  $\pi_\theta$ 
5:      $\mathcal{D} \leftarrow \mathcal{D} \cup \hat{\mathcal{D}}$                                       $\triangleright$  Aggregate
6:      $\theta \leftarrow \theta + \beta \nabla_\theta J(\theta)$                       $\triangleright \beta$  is step size
7:   end for
8: end procedure

```

occurrence of cost as below:

$$P(\pi_\theta(s) \neq \pi_D(s)) \leq \epsilon. \quad (21)$$

The cost c is defined as:

$$c(s, a) = \begin{cases} 0, & a = \pi_D(s) \\ 1, & a \neq \pi_D(s), \end{cases} \quad (22)$$

where the agent gets cost 1 when the predicted action mismatches the expert, otherwise 0. As the cost bound of DAgger will be used in constructing the cost bound of BC, we first construct the cost bound without co-variate shift (i.e., states are all sampled from $\mathcal{D}$). $J_{IN}(\theta)$ denotes the cost of θ in the absence of co-variate shift, where $J_{IN}(\theta)$ is bounded as the follows:

$$\begin{aligned}
J_{IN}(\theta) &= \sum_t \mathbb{E}_{s_t \sim D} \left[\mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \right] \\
&= \sum_t \sum_{s \in D} P_{\pi_D}(s_t = s) [(1 - \epsilon) \cdot 0 + (\epsilon) \cdot 1] \\
&= \sum_t \sum_{s \in D} P_{\pi_D}(s_t = s) \epsilon \leq O(\epsilon T).
\end{aligned}$$

Let us now consider the cost when the state-distribution is induced by rolling out the policy π_θ . In general, rolling out π_θ will lead to states that are not in $\mathcal{D}$. Let $J_{OUT}(\theta)$ be the cost of incurred by rolling out π_θ . We will now bound it as following:

$$\begin{aligned}
J_{OUT}(\theta) &= \sum_t \mathbb{E}_{s_t \sim \pi_\theta} \left[\mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \right] \\
&= \sum_t \sum_{s \in \mathcal{S}} P_{\pi_\theta}(s_t = s) \left[\mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \right] \\
&= \sum_t \sum_{s \in \mathcal{S}} (P_{\pi_D}(s_t = s) - P_{\pi_D}(s_t = s) + P_{\pi_\theta}(s_t = s)) \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \\
&= J_{IN}(\theta) + \sum_t \sum_{s \in \mathcal{S}} (P_{\pi_\theta}(s_t = s) - P_{\pi_D}(s_t = s)) \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)]
\end{aligned}$$

By total probability rule,

$$P_{\pi_\theta}(s_t = s) = \underbrace{P(s_t = s | a_{1:t} \in D)}_{P_{\pi_D}(s_t)} (1 - \epsilon)^t + \underbrace{P(s_t = s | \exists a_i \notin D, 1 \leq i \leq t)}_{P_O(s_t)} (1 - (1 - \epsilon)^t)$$

i.e., starting from an initial state $s_1 \in \mathcal{D}$ if the agent takes action according to D , then it will encounter the states in the same distribution as D . Therefore, $P(s_t = s | a_{1:t} \in D) = P_{\pi_D}(s_t)$.

If the agent takes atleast one action in the first t time steps that is inconsistent from D , it will end up in a different state-distribution than $P_{\pi_D}(s_t)$. We refer to this “other” state distribution as $P_O(s_t)$. $J_{OUT}(\theta)$ can therefore be written as:

$$\begin{aligned} &= J_{IN}(\theta) + \sum_t \sum_{s \in \mathcal{S}} (P_{\pi_D}(s_t = s)(1 - \epsilon)^t + P_O(s_t = s)(1 - (1 - \epsilon)^t) - P_{\pi_D}(s_t = s)) \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \\ &= J_{IN}(\theta) + \sum_t \sum_{s \in \mathcal{S}} [P_O(s_t = s)(1 - (1 - \epsilon)^t) - [P_{\pi_D}(s_t = s)(1 - (1 - \epsilon)^t)] \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \\ &= J_{IN}(\theta) + \sum_t \sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)](1 - (1 - \epsilon)^t) \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \end{aligned}$$

Using $(1 - \epsilon)^t \geq (1 - \epsilon t) \implies -(1 - \epsilon)^t \leq -(1 - \epsilon t)$ if $\epsilon \in [0, 1]$, we have the inequality:

$$\begin{aligned} &\leq J_{IN}(\theta) + \sum_t \sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)](1 - 1 + \epsilon t) \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \\ &= J_{IN}(\theta) + \sum_t \epsilon t \left(\sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)] \mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)] \right) \end{aligned}$$

Now lets bound $\mathbb{E}_{a_t \sim \pi_\theta(s_t)} [c(s_t, a_t)]$ by the worst-case cost of 1 to get,

$$\leq J_{IN}(\theta) + \sum_t \epsilon t \left(\sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)] \right)$$

Based on the above result we have,

$$\begin{aligned} |J_{OUT}(\theta) - J_{IN}(\theta)| &\leq \left| \sum_t \epsilon t \left(\sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)] \right) \right| \\ &= \sum_t \epsilon t \left| \sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)] \right| \end{aligned}$$

In the worst case, $\left| \sum_{s \in \mathcal{S}} [P_O(s_t = s) - P_{\pi_D}(s_t = s)] \right| \leq 2$ (lookup Total Variation Distance to see why). For a discrete distribution, an example where this worst case holds is: $P_O(s_t = s_1) = 1$ and $P_{\pi_D}(s_t = s_2) = 1$, where $s_1 \neq s_2$ and the probability of all other states is 0.) We therefore have:

$$|J_{OUT}(\theta) - J_{IN}(\theta)| \leq \sum_t 2\epsilon t \sim O(\epsilon T^2). \quad (23)$$

The upper bound in Equation 23 is not loose. An example that achieves the cost $O(\epsilon T^2)$ can be found in [Ross and Bagnell, 2010, Ross et al., 2011].

Based on the analysis above behavior cloning has a worst case cost of $O(\epsilon T^2)$ which stems from the difference in state distribution of D and π_θ . As the state distribution of the policy trained using DAgger eventually matches D , the cost of the policy is bounded by $O(\epsilon T)$. Therefore, if the expert is available to sample throughout the training process, using DAgger is more efficient than simple behavior cloning.

Summary Though DAgger solves the co-variate shift issue, the following challenging problems are still unsolved.

- **Tedious experts:** While BC requires the expert to label the dataset just once before training, DAgger requires querying the expert policy π_D multiple times during training.

This necessitates that human experts are available to provide action labels for the robot during the entire training time, which is tedious.

- **Multimodality:** The distribution of the expert’s action at a state might have multiple “peaks” (precisely, modes or modalities). For example, consider a robot controlling a steering angle to avoid the tree in the front. Both turning right and left can successfully avoid the tree. The distribution of the expert’s steering angle will likely have two modes where one is at left and another is at right. This leads to an issue since the policy is typically parameterized as a unimodal Gaussian distribution. To account for multimodalities, one can use a mixture of Gaussian models, conditional variational autoencoder (CVAE) [Sohn et al., 2015], or implicit behavior cloning [Florence et al., 2022]. Section 6.5 discusses these solutions in details.
- **Causal confusion:** The agent might learn “spurious” features for the policy. With self driving cars, for example, experts brake when pedestrians are standing on the road. However, BC or DAgger agents might correlate the brake indicator and the action of taking a brake. As a result, instead of learning to brake in response to a pedestrian, BC/DAgger may learn to brake when seeing the brake indicator light. Section 6.4 details the discussion on this issue.
- **Copy cat problem:** Though the Markov property is assumed in this section and thus the policy π_θ only takes the current state s_t as inputs, it is necessary to take history of states and actions $s_{1:t}, a : 1 : t - 1$ as inputs when the Markov property is invalid. However, as the expert’s actions are highly correlated in time, the learned policy with BC might predict the next action a_t by previous actions in most cases. This will result in a problem when the expert’s action changes sharply in a few but critical situations. We will detail this issue in Section 6.3.
- **Suboptimal demonstrations:** Demonstration might not be from a task expert. As BC/DAgger are trained to match the policy in the demonstration, the suboptimal demonstration is unlikely to train the policy to attain the optimal performance in the task. We will cover a workaround to this issue in Section 6.2.
- **Task-specific demonstration:** One set of demonstrations is only for a single task. As generating demonstrations is expensive, recent research [Ajay et al., 2020] have attempted to extract re-usable knowledge from the demonstration in hopes of accelerating learning in multiple tasks.

6.2 Improving over suboptimal demonstration

In the context of BC and DAgger, the fundamental challenge of learning a better policy beyond suboptimal demonstration is the lack of correct action label. As long as the demonstrator’s policy is suboptimal, the action labels in the demonstration dataset $\mathcal{D}$ are not the optimal actions for the task. By the first principle, to improve the policy over suboptimal demonstration, one requires additional learning signals other than demonstrators’ actions. This idea naturally leads to the combination of RL and BC. In the abstract sense, the policy optimization objective w.r.t. expected return offers additional learning signals beyond demonstrations. The following sections introduce two approaches of incorporating RL and BC.

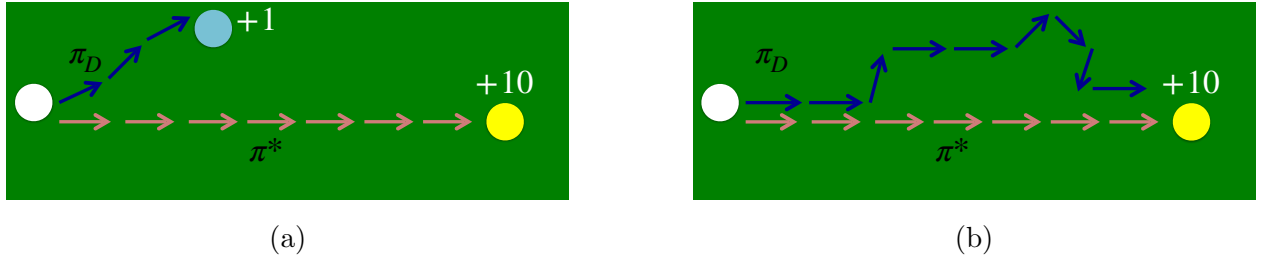


Figure 4: π_D and π^* denote the suboptimal and the optimal policies, respectively. The blue and pink trajectories are a suboptimal and an optimal trajectory. Hitting cyan and yellow dots give +1 and +10 rewards, respectively. If the agent imitates the π_D , the agent would unlikely reach the goal (yellow dot). (a) A policy π_θ trained by BC with this suboptimal demonstration impedes exploration since the pre-trained policy π_{θ} unlikely approaches the goal. (b) With this demonstration, pre-training the policy π_θ is beneficial since the exploration is not biased.

6.2.1 Pre-training the policy with BC

One strategy to learn a policy beyond suboptimal demonstration is combining RL and BC. Rajeswaran et al. [2017] pre-train the agent's policy π_θ by BC and continue training π_θ by RL. Intuitively, this method attempts to use BC to produce an initialized policy better than a randomly initialized one. However, initializing the policy by suboptimal demonstration might bias the exploration and hinder policy improvement. For example, in the example shown in Figure 4, if the policy of suboptimal demonstrator goes opposite to the goal, exploration of BC-initialized policy will unlikely hit the goal and thus not be able to obtain rewarding trajectories for policy optimization.

6.2.2 Combining BC and policy gradient in policy optimization

Alternatively, Rajeswaran et al. [2017] proposed to combine the objective of BC with policy gradient (PG), as shown below:

$$\nabla J_{PG+BC}(\theta) = \underbrace{\mathbb{E}_{\tau \in \mathcal{D}} \left[\sum_t \nabla_\theta \log \pi_\theta(a|s_t) w(s, a) \right]}_{BC} + \underbrace{\mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a|s_t) R(\tau_{t:T}) \right]}_{PG}, \quad (24)$$

where J_{PG+BC} denotes the combined objective function. The BC term in objective 24 is the gradient w.r.t. the policy parameter θ weighted by $w(s, a)$, where $w(s, a)$ measures the importance of gradients computed on demonstration dataset. Optimizing the policy π_θ by this objective can prevent the policy from being biased by demonstration since the $w(s, a)$ can modulate the weights of gradients of BC and PG objectives.

The next question is the choice of $w(s, a)$. As we want to measure the advantage of demonstrators' action $a \sim \mathcal{D}$ in a state $s \sim \mathcal{D}$, an ideal choice can be $w(s, a) = A^\pi(s, a)$. However, estimating $A^\pi(s, a)$ for $(s, a) \sim \mathcal{D}$ requires additional interaction data with the environment since (s, a) are not sampled from π_θ . Recall in Section 4.4.1, we compute the advantage by $A^{\pi_\theta}(s_t, a_t) = r(s_t, a_t) + \gamma V_\psi^{\pi_\theta}(s_{t+1})$, where s_t, a_t require to be sampled from π_θ . Another heuristic proposed by Rajeswaran et al. [2017] is decaying $w(s, a)$ over time. The proposed decaying scheme is formulated as below:

$$w(s, a) = \lambda_0 (\lambda_1)^m,$$

where $\lambda_0 = 0.1$ and $\lambda_1 = 0.95$, and m denotes the iterations of policy optimization (see [Rajeswaran et al. \[2017\]](#)) for details. Intuitively, this heuristic says that the BC objective is important at the beginning (i.e., m is small), while PG objective is relatively more important than BC objective in the later stage of training (i.e., m is large). The hypothesis behind this heuristic is that at the beginning, the BC term in the J_{PG+BC} can give more useful gradients than PG term since demonstrations are assumed to contain at least a few trajectories with non-zero returns, despite being suboptimal. Since the agent learns to obtain rewards as the training progresses, the gradients of PG terms are no longer zero and can potentially improve the policy better than BC. As such, the BC term should vanish over time to avoid biasing the policy optimization objective.

6.2.3 Summary

However, the heuristic $w(s, a) = \lambda_0(\lambda_1)^m$ is not guaranteed to overcome the biases of suboptimal demonstration as an excessively large λ_0 and λ_1 can cause BC term to dominate the objective all the time during training. The line of works on combining RL and suboptimal demonstration data mostly share the same idea of combining the policy optimization objective with trajectories from the agent and demonstration. Avoiding biases of suboptimal demonstration remains an open problem.

6.3 Copycat problem

The copycat problem mainly happens when it is necessary to take the expert's history of actions as inputs of the policy π_θ . Recall that in BC, the policy π_θ is trained to predict the expert's action a_t . As the expert's actions $a_{1:t}$ are highly correlated in time (i.e., a_{t-1} and a_t is correlated), it is possible to obtain a low expected loss by only conditioning on a_{t-1} and ignoring the state s_t . Formally, we can illustrate this problem by comparing the objective functions of two parameterizations of the policy π_θ for the optimization problem 20:

$$\begin{aligned} J_{SA}(\theta) &= \mathbb{E}_{s_t, a_t \in \mathcal{D}} [\nabla_\theta \pi_\theta(\hat{a}_t = a_t | s_{1:t}, a_{1:t-1})] \\ J_A(\theta) &= \mathbb{E}_{s_t, a_t \in \mathcal{D}} [\nabla_\theta \pi_\theta(\hat{a}_t = a_t | a_{1:t-1})], \end{aligned}$$

where note that we add timestep t to the sampled state-actions since this formulation requires the history of states-actions. If the history of actions $a_{1:t-1}$ highly correlate with a_t , optimizing J_{SA} and J_A likely obtains the similar objective value. As a result, the policy π_θ could ignore the state s_t during optimization and turns out to be a policy only conditioned on history actions $a_{1:t-1}$.

The policy ignoring the state results is unlikely to perform well in critical states. Consider training a policy with BC to match an expert car driver's policy. The expert will brake and stop for multiple timesteps until the light turns green upon seeing the red light. The transition from red and green light is the critical state. Once the green light shows up, the driver will step on the gas instantly, thus resulting in a sharp transition in the action sequence. Specifically, if the action denotes acceleration, it is likely to see the pattern $a_1 = 0$ and $a_2 = 10$ in the demonstration. Thus, the policy only using the history of actions $a_{1:t-1}$ cannot predict accurately in these critical states.

The fundamental problem of copycat is dataset imbalance. Critical states account for a

small portion of the dataset and will not largely affect the objective value. One strategy is to up-weight rare data in the dataset. [Wen et al. \[2021\]](#) proposed to weigh the objective as follows:

$$J(\theta) = \mathbb{E}_{s_t, a_t \in \mathcal{D}} [w \nabla_{\theta} \pi_{\theta}(\hat{a}_t = a_t | s_{1:t}, a_{1:t-1})],$$

where w denotes the weight of a sample (s_t, a_t) . One idea is to identify critical states and up-weight these states. However, how do we find critical states? Please see [Wen et al. \[2021\]](#) for details.

6.4 Causal confusion and spurious features

The fundamental issue of causal confusion in BC is that the learned policy correlates unwanted (spurious) features with action predictions. Recall in [Section 6.1](#), we mentioned that the agent might learn to brake when seeing the brake indicator lighting up, instead of braking when seeing pedestrians. Different from the copycat problem in [Section 6.3](#), the problem of causal confusion does not result from imbalanced (or biased) dataset. Despite being given a diverse dataset for car driving, this unwanted correlation could still happen because in the demonstration dataset, the brake indicator lights up whenever braking and thus results in a high correlation with actions of the brake. The causal confusion problem results from the ambiguity of “task specification”. We anticipate the agent learns to predict actions based on features of traffic on the road, while the demonstration dataset cannot specify our anticipation to BC algorithm.

As BC is equivalent to supervised learning, let’s take a simple example in classification to illustrate this concept. Consider training a classifier predicting digits on a colored MNIST dataset, where the images of each digit correspond to a unique color (e.g., 2 for red, 6 for blue), and testing the classifier on a dataset with different colors for each digit. It is known that the classifier can learn a perfect solution (i.e., nearly zero prediction error) on the training dataset, but performs poorly on the testing dataset with different colors. This observation indicates that the neural network does not learn the features of digits (e.g., shape), but likely correlates the color or other features with digit predictions.

The spurious feature correlation problem is central to task specification. In the above MNIST example, the task in the dataset designer is predicting digits, but the provided dataset does not communicate this intention. The colored MNIST can also be interpreted as predicting the color (e.g., red images for label 2 and blue images for label 6). This ambiguity results in spurious features problems since the neural network can learn to either correlate colors or shapes to predictions. In turn, there are an infinite number of valid solutions (i.e., neural network weights) for a training dataset, while our anticipated solution is dependent on the testing dataset inaccessible during training. Moreover, this is not a problem that deep neural network finds a bad local optimum solution since, in this MNIST example, the prediction errors are extremely low. As the details of this problem are beyond the scope of this lecture note, interested readers may refer to [Zhang et al. \[2016\]](#), [Ilyas et al. \[2019\]](#).

6.5 Multimodality problem

Intuitively, the multimodality in BC results from that the demonstrator has multiple strategies to accomplish the task or that demonstration is collected from many demonstrators with different policies. Quantitatively speaking, there are two “modes” (in statistics) in the action distribution $\pi_D(a|s)$.

Steering angle example In the following, we will use the example of the steering angle mentioned in Section 6.5, where actions correspond to the angular velocity. Both $a = -1$ and $a = 1$ could be the modes of demonstrator’s action since both turning left and turning right are the optimal actions avoiding obstacles in the front.

The multimodality nature can be easily accounted by Boltzmann distribution if the action space is a discrete set.

$$\pi_\theta(\hat{a}|s) = \frac{\exp(\phi_{\theta\hat{a}}(s))}{\sum_{a \in \mathcal{A}} \exp(\phi_{\theta a}(s))},$$

where $\phi_\theta(s) = [\phi_{\theta_{a_1}}(s), \phi_{\theta_{a_2}}(s), \dots]$ denotes a vector of the the output of a neural network parameterized by θ conditioned on the state s . Each $\phi_{\theta_{a_i}}(s)$ represents the logits for a_i output by the neural network. Consider the action space is $\{-1, 0, 1\}$ and -1 and 1 are the modes of expert actions. Two actions -1 and 1 will be sampled equally often, where $\pi_\theta(\hat{a} = 1|s) = 0.5$ and $\pi_\theta(\hat{a} = -1|s) = 0.5$.

If the action space is a continuous space, capturing multimodality in the dataset is not trivial. In practice, most of the algorithms assume the expert actions are drawn from a unimodal Gaussian distribution and approximate the mean of this distribution by π_θ . Maximizing the log-likelihood of a unimodal Gaussian assumption is equivalent to minimizing the mean squared errors between $\pi_\theta(s)$ and the expert’s actions a (see Section D for details). However, this approach cannot account for multiple modes of action. Minimizing the mean squared error between the unimodal policy π_θ and a multimodal expert policy leads to a π_θ predicting average action between modes, while the average action is not optimal. Consider the above example of controlling the steering angle to avoid the obstacle. If the modes of expert actions are -1 and 1 , the average action between two modes is 0 and thus cannot make the agent avoid the obstacle in the front.

6.5.1 Mixture of Gaussian (MoG)

One way to account for multimodality is using a mixture of Gaussian models to fit the distribution of the expert’s actions. A mixture of Gaussian models consists of multiple unimodal Gaussian distributions, where each distribution corresponds to a mode of expert actions. The action likelihood $\log \pi_\theta(a|s)$ turns into the weighted sum over Gaussian distributions, as shown in the follows:

$$\log \pi_\theta(a|s) = \sum_{k=1}^K w_k(\theta_k) \mathcal{N}(a|\mu_{\theta_k}, \Sigma_{\theta_k}, s),$$

where we slightly abuse the notation $\theta = \{\theta_1, \theta_2, \dots\}$ for denoting a set of parameters, $\mathcal{N}$ denotes a Gaussian distribution, and μ_{θ_k} , Σ_{θ_k} , and w_k stand for the mean, the covariance matrix, and the weight of k -th Gaussian distribution.

6.5.2 Discretized Autoregressive models

As a discrete action space can account for multimodality, one idea is to discretize the continuous action space and fit a discrete distribution as we show above in the beginning of Section 6.5. Suppose the action space $\mathcal{A} = \mathbb{R}^M$ (i.e., M -dimensional vector) and each action is denoted as an M -dimensional vector

$$\mathbf{a} := (a^1, \dots, a^M),$$

where a^i denotes a dimension of an action vector.

The values of each dimension of $\mathcal{A}$ is assumed to be within range $[a_{MIN}, a_{MAX}]$. As shown in Figure 5a, partitioning each dimension to N bins, one can obtain an action space with dimension $\underbrace{N \times N \times N \cdots N}_M = N^M$. The policy's output turns out to be a Boltzmann distribution over N^M bins, where each bin denotes a combination of action values in each dimension (e.g., $(0.25, 0.5, 0.5, \dots, 0.75)$) and the $\pi_\theta(a = (0.25, 0.5, 0.5, \dots, 0.75)|s)$ corresponds to the joint probability of action $a = (0.25, 0.5, 0.5, \dots, 0.75)$. Despite being able to account for multimodality, this approach makes optimization difficult due to the huge output dimension. The dimension of the output space (i.e., the dimension of the policy's output) grows exponentially with the number of bins N with this discretization scheme. The resulting large output space makes optimization difficult.

One option is to assume each dimension of $\mathbf{a}$ is independent. Thus the joint probability $P(a^1 \cdots a^M|s, \theta)$ can be rewritten as:

$$P(a^1 \cdots a^M|s, \theta) = P(a^1|s, \theta)P(a^2|s, \theta) \cdots P(a^M|s, \theta).$$

The policy's output can be structured as Figure 5b, where the output dimension is reduced to $M \times N$. However, this independence assumption might not always hold. To account for dependence among each dimension of $\mathbf{a}$ and reduce the output dimension, one can use the Bayes rule to decompose the joint probability as follows:

$$P(a^1, \dots, a_M|s, \pi_\theta) = P(a^1|s, \pi_\theta)P(a^2|a^1, s, \pi_\theta)P(a^3|a^1, a^2, s, \pi_\theta) \cdots P(a^M|a^{1:M-1}, s, \pi_\theta).$$

This decomposition gives rise to an autoregressive architecture shown in Figure 5c. Though this autoregressive model can account for dependency between each dimension of $\mathbf{a}$, one requires domain knowledge to determine the ordering of the dependency. For example, one can write $P(a^1, \dots, a_M|s, \pi_\theta)$ in a different ordering (i.e., a^2 comes before a^1) as the follows:

$$P(a^1, \dots, a_M|s, \pi_\theta) = P(a^2|s, \pi_\theta)P(a^1|a^2, s, \pi_\theta) \cdots P(a^M|a^{1:M-1}, s, \pi_\theta).$$

6.5.3 Latent variable models

The problem of multimodality in unimodal Gaussian distributions results from a single mean cannot express multimodal behavior. Thus, one workaround is augmenting the policy π_θ with latent variables in the inputs. The outputs of this augmented policy can be expressed as $\pi_\theta(\cdot|s, \epsilon)$, where ϵ denotes noises drawn from an arbitrary distribution. Since the implementation of the latent variable models is beyond the scope of this lecture note, we refer the reader to conditional variational autoencoder (CVAE) [Sohn et al., 2015] for details.

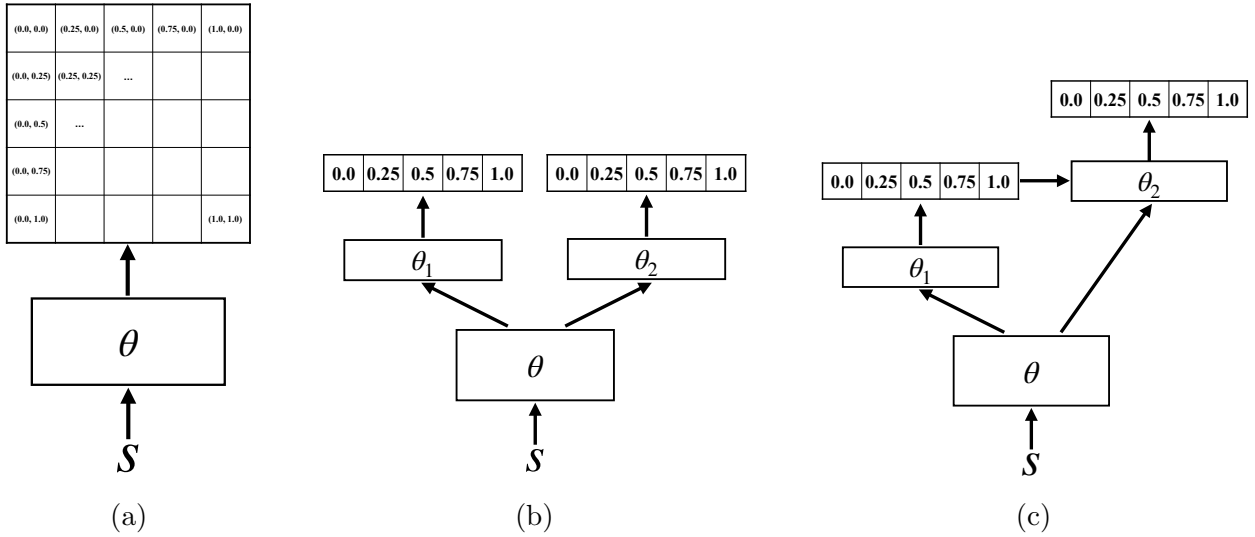


Figure 5: Three approaches parametrizing the discretized action space with $\mathcal{A} = \mathbb{R}^2$ and 5 bins in each dimension. We denote an action $\mathbf{a} := (a^1, a^2)$ (a) Parameterizing the joint probability by θ directly (e.g., $P(a^1 = 0.25, a^2 = 0.5 | s, \theta)$). (b) Assuming each dimension of action $\mathbf{a}$ is independent, the joint probability can be decomposed to $P(a^1, a^2 | s, \theta) = P(a^1 | s, \theta)P(a^2 | s, \theta)$. (c) Using Bayes rule, the joint probability can be decomposed to $P(a^1, a^2 | s, \theta) = P(a^1 | s, \theta)P(a^2 | a^1, s, \theta)$. The option (a) requires outputting a large vector with shape 5×5 , while the options (b) and (c) output a smaller vector with shape 2×5 . The option (c) accounts for dependency between each dimension of $\mathbf{a}$ while option (b) requires each dimension to be independent.

6.5.4 Implicit models

Instead of modeling the likelihood of action directly, recent works [Florence et al. \[2022\]](#) proposed to model the action prediction with an implicit model $E_\theta(s, a)$ shown below:

$$\pi_\theta(s) = \arg \min_a E_\theta(s, a),$$

where the function $E_\theta(s, a)$ can be optimized using contrastive learning [Van den Oord et al. \[2018\]](#). This implicit method can overcome multimodality problem since the function E_θ does not require assumptions on types of distributions.

6.6 Beyond task specific demonstration

In addition to learning to achieve a task from demonstration, one can also learn reusable “skills” from the demonstration. A skill is a common pattern of action sequences among demonstrations. The learned skills can be used to accelerate the progress of RL training since skills can be composed to achieve a new task and thus save time on exploring the combination of primitive actions. For example, in robotic applications, a robot has to try a number of different combinations of motor commands (i.e., joint torques) to achieve “pick” and “place.” In contrast, it is easy for the robot to learn pick-and-place with the learned skills. The robot requires sequencing skills “reach” and “pick” for a few steps since the action sequence abstracts the time, where one “reach” might consist of 10 motor commands. As a result, the total number of decisions is reduced.

Formally, a skill can be defined as a distribution of action sequence $a_{t:t+K}$ that the expert usually takes in a state s_t , where K is the length of the action sequence. Denoting the skill type

by variable z , one can represent an expert skill as a conditional distribution $\pi_D(a_{t:t+K}|s, z)$. One way to discover skills is performing K -means clustering over action sequences in the demonstration. Nevertheless, this approach requires pre-defining the number skills K beforehand, and the distance metric (for K -means algorithm) between action sequences $a_{t:t+K}$ is not well-defined (e.g., Euclidean distance between two action sequences do not have clear meaning on the similarity between action sequences).

One way to extract skills from demonstration is using auto-encoder. The idea is to use auto-encoder to encode action sequences as latent variables of skills and take these skills as primitive actions for an RL agent’s policy. [Ajay et al. \[2020\]](#) train an encoder f_{enc} that takes in a state s_t and an action sequence $a_{t:t+K}$ and outputs a latent skill variable z . Another decoder f_{dec} taking z is trained to output (i.e., reconstruct) the encoder’s input $a_{t:t+K}$. Freezing the decoder f_{dec} , [Ajay and Agrawal \[2020\]](#) train a policy π_θ taking state s and predicting skill variables z . The agent then feeds the predicted z to f_{dec} , and takes the decoded action sequence $a_{1:K}$ to interact with the environment.

A limitation of learning a policy based on skills is that the diversity of the set of skills can restrict the capability of a policy. For example, if the demonstration does not contain a “jump” action, a robot learning a policy over skills is unable to learn to jump over obstacles.

6.7 Inverse reinforcement learning (draft)

Inverse Reinforcement Learning (IRL) [[Ng and Russell, 2000](#), [Russell, 1998](#)] offers an alternative lens to LfD. Intuitively, the idea of IRL is to infer the expert’s “intent” by observing the expert’s demonstration (e.g., trajectories). The question is: *how do we define the intent?* One classic view proposed in [Ng and Russell \[2000\]](#) suggest that a succinct summary of the demonstrator’s intent is an (unknown) reward function r_D . [Ng and Russell \[2000\]](#) formalizes IRL as a problem of inferring the demonstrator’s reward function r_D from demonstration. The idea of inferring the demonstrator’s reward function gives rise a new aspect to LfD. Assuming the demonstrator is maximizing the expected return of the reward function, one can train a policy maximizing the expected return w.r.t. the inferred reward function to match the demonstrator’s policy.

In general, inferring the demonstrator’s reward function is an iterative process. Most of IRL algorithms start from a randomly guess reward function $\hat{r}$ and iteratively improve $\hat{r}$ such that $\hat{r} \approx r_D$. Improving the guess reward function $\hat{r}$ often requires training a policy π (we term it as the agent’s or the learner’s policy) maximizing the return w.r.t. $\hat{r}$. The intuitive reason for the needs of training a policy to improve $\hat{r}$ is that we need to “test” if the guess reward function $\hat{r}$ approximates the demonstrator’s reward function r_D . We will detail how the policy is used for approximating r_D in the following sections. In summary, IRL can be outlined as the following steps:

1. Initialize a policy π and a guess reward function $\hat{r}$
2. **Outer loop:** Update the guess reward function $\hat{r}$ using π
3. **Inner loop:** Update policy π with the guess reward function $\hat{r}$
4. Go to step 2 (Outer loop).

The inner loop is simply a canonical policy optimization procedure. The challenge of IRL is central at outer loop: updating the guess reward function. Let us try defining a reward function that best fit to the demonstrator's intent. As the demonstrator's policy is assumed to be optimal in the task, the reward function r_D is commonly [Ng and Russell, 2000, Abbeel and Ng, 2004, Ziebart et al., 2008] characterized by the following inequality:

$$\mathbb{E}_{\pi_D, s_1 \sim \rho(s_1)} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_D(s_t, a_t) \right] \geq \mathbb{E}_{\pi, s_1 \sim \rho(s_1)} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_D(s_t, a_t) \right] \forall \pi, \quad (25)$$

where $\rho(s_1)$ denotes an arbitrary initial state distribution and for any π , $\mathbb{E}_{\pi, s_1 \sim \rho(s_1)}$ means s_1 is sampled from $\rho(s_1)$ and rest of states and actions are sampled by the interaction between π and the environment. Equation 25 states that the expert's policy π_D must result in higher expected return than all the other learner's policy π .

The problem is that Equation 25 affords a lot of feasible solutions and even a trivial feasible solution: $\hat{r}(s, a) = 0 \forall s, a$. While this trivial reward function satisfies the inequality in Equation 25, but cannot be used to learn the policy π_θ . Trivial solutions exist because there are many choices of reward functions under which the expert demonstrations are optimal. In other words, given only the demonstrations, the intent is *under-specified*. Resolving the under-specification, requires making additional assumptions, akin to the use of *regularization* in machine learning or other constraints.

In the following sections, we introduce classic and the state-of-the-art methods to address this under-specification problem in IRL. Section 6.7.1 covers the max-margin method restricting the solution set of $\hat{r}$ by maximizing the separation between the expected return of the demonstrator's policy π_D and the learner's policy π . Section 6.7.2 introduces maximum entropy method that regularize the solution of $\hat{r}$. Section 6.7.3 presents the state-of-the-art IRL method using the adversarial formulation.

6.7.1 Max margin inverse reinforcement learning

To mitigate under-specification, Ng and Russell [2000] proposed to maximize the gap between the expert's and the learned policy's value functions. The rationale is among all that the reward function that maximizes separation between the expert policy π_D from all other policies π is preferred among the set of $\hat{r}$ feasible to the constraint in Equation ???. For brevity, we denote the expected return of a policy π as the expected value over distribution of initial states, as shown below:

$$\mathbb{E}_{s_1 \sim \rho(s_1)} [V_{\hat{r}}^\pi(s_1)] = \mathbb{E}_\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} \hat{r}(s_t, a_t) \right].$$

Note that to highlight the dependency on the reward function $\hat{r}$, we add $\hat{r}$ to the subscript of the value function. The expert's reward function can then be obtained by solving for $\hat{r}$ of the optimization as shown below:

$$\begin{aligned} \max_{\hat{r}, \epsilon} \quad & \epsilon \\ \text{s.t.} \quad & \mathbb{E}_{s_1 \sim \rho(s_1)} [V_{\hat{r}}^{\pi_D}(s_1)] \geq \mathbb{E}_{s_1 \sim \rho(s_1)} [V_{\hat{r}}^\pi(s_1)] + \epsilon \quad \forall \pi, \end{aligned} \quad (26)$$

where ϵ denotes a margin separating the expected return of π_D and π .

One simple implementation of max margin method proposed in [Ng and Russell \[2000\]](#) assumes the reward function $\hat{r}$ can be parameterized as a linear function shown below

$$\hat{r}(s, a) = \mathbf{w}^T \phi(s, a) = \sum_{i=1}^d \mathbf{w}_i \phi_i(s, a), \quad (27)$$

where $\phi(s, a)$ denotes a d -dimensional feature vector of a state-action pair (s, a) and $\mathbf{w}$ denotes the coefficients of a linear function. Note that ϕ_i denotes i -th dimension of vector ϕ . In this section, we will denote the element of a vector by subscript. Since the reward function is assumed linear, the value $V_{\hat{r}}^{\pi}(s), \forall s \sim \rho(s)$ of a policy π can then be related to $\mathbf{w}$ as the follows:

$$\begin{aligned} \mathbb{E}_{s_1 \sim \rho(s_1)} [V_{\hat{r}}^{\pi_D}(s_1)] &= \mathbb{E}_{\pi} \left[\sum_{t=1}^{\infty} \gamma^{t-1} \hat{r}(s_t, a_t) \right] \\ &= \mathbb{E}_{\pi} \left[\sum_{j=t}^{\infty} \gamma^{j-t} \sum_{i=1}^d \mathbf{w}_i \phi_i(s_t, a_t) \right] \\ &= \sum_{i=1}^d \mathbb{E}_{\pi} \left[\sum_{t=1}^{\infty} \gamma^{j-t} \mathbf{w}_i \phi_i(s_t, a_t) \right] \\ &= \sum_{i=1}^d \mathbf{w}_i \mathbb{E}_{\pi} \left[\sum_{t=1}^{\infty} \gamma^{t-1} \phi_i(s_t, a_t) \right] \\ &= \sum_{i=1}^d \mathbf{w}_i \mu_i(\pi) \\ &= \mathbf{w}^T \mu(\pi), \end{aligned}$$

where we abbreviate $\mathbb{E}_{\pi} \left[\sum_{t=1}^{\infty} \gamma^{t-1} \phi_i(s_t, a_t) \right]$ as $\mu(\pi)$ for any π . Thus, the optimization problem in Equation 26 can be formulated as a linear program as shown below¹⁰:

$$\begin{aligned} \max_{\mathbf{w}, \epsilon} \quad & \epsilon \\ \text{s.t.} \quad & \mathbf{w}^T \mu(\pi_D) \geq \mathbf{w}^T \mu(\pi_{\theta^k}) + \epsilon \quad \forall k = 1 \dots K \\ & |\mathbf{w}_i| \leq 1 \quad \forall i = 1 \dots d, \end{aligned} \quad (28)$$

where note that the constraint $\mathbf{w}^T \mu(\pi_D) \geq \mathbf{w}^T \mu(\pi_{\theta^k}) + \epsilon$ is enforced over K policies since it is intractable to enumerate all policies π . θ^k denotes the parameter vector of k -th sampled policy. The constraint $|\mathbf{w}_i| \leq 1$ is a regularization to prevent overfitting [\[Ng and Russell, 2000\]](#). We summarize the max-margin approach using the outer-inner loop framework outlined in the beginning of Section 6.7.

1. Given $\mu(\pi_D)$ (it can be estimated from demonstration)
2. Initialize a policy parameter θ^1 , the reward function coefficients $\mathbf{w}$, and $K = 1$
3. **Outer loop:**
 - (a) Estimate $\mu(\pi_{\theta^k})$ by rolling out trajectories in the environment
 - (b) Update $\mathbf{w}$ by solving the optimization problem in Equation 28
4. **Inner loop:** Learn a set of new policy parameters: $\theta^{k+1} \leftarrow \arg \max_{\theta} \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} \hat{r}(s_t, a_t) \right]$

¹⁰Readers might find our equations different from [Ng and Russell \[2000\]](#) because we unify the formulation of [Ng and Russell \[2000\]](#) and [Abbeel and Ng \[2004\]](#). Both formulations use the same linear reward assumption (Equation 27) and can be unified in the same formulation.

5. Set $K \leftarrow K + 1$ and go to step 3 (Outer loop).

Remark 6.1 (Feature matching is equivalent to matching return). Notably, [Abbeel and Ng \[2004\]](#) showed a useful property that will be used in the later sections. This property gives rise a new LfD interpretation formalized as finding a policy π such that:

$$|\mathbb{E}_{\pi_D, s_1 \sim \rho(s_1)} [\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t)] - \mathbb{E}_{\pi, s_1 \sim \rho(s_1)} [\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t)]| \leq \epsilon, \quad (29)$$

where ϵ denotes a small constant that can be interpreted a margin between π_D and π . Changing the regularization constraint on $\mathbf{w}$ in Equation 28, we can obtain an optimization problem:

$$\begin{aligned} \max_{\mathbf{w}, \epsilon} \quad & \epsilon \\ \text{s.t. } \quad & \mathbf{w}^T \mu(\pi_D) \geq \mathbf{w}^T \mu(\pi_{\theta^k}) + \epsilon \quad \forall k = 1 \cdots K \\ & \|\mathbf{w}\| \leq 1, \end{aligned} \quad (30)$$

With the linear reward assumption (Equation 27), we can obtain the following relations:

$$\begin{aligned} & |\mathbb{E}_{\pi_D, s_1 \sim \rho(s_1)} [\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t)] - \mathbb{E}_{\pi, s_1 \sim \rho(s_1)} [\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t)]| \\ &= |\mathbf{w}^T (\mu(\pi_D) - \mu(\pi))| \\ &\leq \|\mathbf{w}\| \|\mu(\pi_D) - \mu(\pi)\| \quad (\text{by } \|\mathbf{w}\| \leq 1) \\ &\leq 1 \cdot \|\mu(\pi_D) - \mu(\pi)\| \leq \epsilon. \end{aligned}$$

Thus, matching the expert's expected return can be reformulated as finding a policy such that:

$$\|\mu(\pi_D) - \mu(\pi)\| \leq \epsilon.$$

6.7.2 Maximum entropy inverse reinforcement learning

[Ziebart et al. \[2008\]](#) proposed an alternative to address under-specification problem by the entropy of the distribution of trajectories generated by π_D . The fundamental problem of under-specification is ambiguity among guessed reward functions $\hat{r}_{\mathbf{w}}$, where multiple $\hat{r}_{\mathbf{w}}$ can make the expert's policy π_D to be the optimal policy (i.e., explain the expert's intent). [Ziebart et al. \[2008\]](#) disambiguate the guessed reward functions $\hat{r}_{\mathbf{w}}$ based on the principle of maximum entropy (cite). The rationale of using this principle is that the expert's policy is assumed to:

- Prefer trajectories of high returns
- Have equal preference to other trajectories.

The preference to high-return trajectories can be modeled by Equation 25. Maximizing the entropy of expert's trajectories accounts for equal preference to other trajectories because maximum entropy implies an uniform distribution over trajectories (i.e., equal preference). In short, we aim for a reward function $\hat{r}_{\mathbf{w}}$ satisfying the following conditions:

- **Optimality:** Let $P_{\mathbf{w}}$ be the distribution of expert's trajectories induced by the reward function $\hat{r}_{\mathbf{w}}$. We define the optimal reward function (i.e., preference on high-entropy trajectories) as the one resulting maximum entropy:

$$\max_{\mathbf{w}} \mathcal{H}(P_{\mathbf{w}}), \quad \mathcal{H}(P_{\mathbf{w}}) = - \sum_{\tau} P_{\mathbf{w}}(\tau) \log P_{\mathbf{w}}(\tau).$$

Maximizing this objective gives rise to an uniform distribution over trajectories, thus giving equal preference to each trajectory. Reader might confuse that how the guessed reward function relates the expert's trajectories since the expert's trajectories are given in the demonstration dataset $\mathcal{D}$. Each trajectory τ determines a sequence of states and actions (see Section 3.4), while note that the probability of a trajectory τ can be different due to $\hat{r}_{\mathbf{w}}$. Recall that a probability of a trajectory can be expressed as the product of state transition function and the policy. As the action probability output by the policy is affected by the reward function, we can write $P_{\mathbf{w}}(\tau)$ as the follows:

$$P_{\mathbf{w}}(\tau) = \rho(s_1) \prod_{t=1}^T \mathcal{T}(s_t | s_{t-1}, a_{t-1}) \pi_D(a_t | s_t, \hat{r}_{\mathbf{w}}).$$

Though the above expression has not yet yield clear connection to the reward function $\hat{r}_{\mathbf{w}}$, we will soon show the relation in the following.

- **Feasibility:** In addition to the preference on maximum entropy solution, we need to ensure the found reward function $\hat{r}_{\mathbf{w}}$ makes the expert's policy better than all the other policies since the expert's policy is assumed optimal, as shown in Equation 25. Since MaxEnt IRL does not need to maximize the margin ϵ to all policies, we only need to focus on the optimal policy closest to the expert's policy. Suppose the reward function guessed $\hat{r}_{\mathbf{w}}$ equals to the expert's reward function r_D and the expert's policy is optimal w.r.t. r_D . It implies that the optimal policy π^* w.r.t. $\hat{r}_{\mathbf{w}}$ must perform as well as the expert's policy π_D . As the reward is assumed linear (see Equation 27) in Ziebart et al. [2008] and thus the expected return of any policy π can be expressed as the follows:

$$\mathbf{w}^T \mu(\pi) = \mathbb{E}_{\pi, s_1 \sim \rho(s_1)} \left[\sum_{t=1}^{\infty} \gamma^{t-1} \hat{r}_{\mathbf{w}}(s_t, a_t) \right].$$

Hence, we can construct an equality constraint as shown below to enforce the expected returns of the expert and the learner's optimal policy to be equal:

$$\mathbf{w}^T (\mu(\pi_D) - \mu(\pi^*)) = 0.$$

Based on Remark 6.1, the above constraint implies that

$$\mu(\pi_D) = \mu(\pi^*), \text{ where } \mathbf{w} \neq \mathbf{0}.$$

Derivation Combining the optimality and feasibility conditions gives rise to the following constrained optimization problem:

$$\begin{aligned} \max_{P_{\mathbf{w}}} \quad & \mathcal{H}(P_{\mathbf{w}}) \\ \text{s.t.} \quad & \mu(\pi_D) = \mu(\pi^*) \\ & \sum_{\tau} P_{\mathbf{w}}(\tau) = 1, \end{aligned} \tag{31}$$

where the constraint $\sum_{\tau} P_{\mathbf{w}}(\tau) = 1$ is needed for ensuring $P_{\mathbf{w}}$ is a probability distribution. So far, we still cannot solve the optimization problem in Equation 31 since the expression of $P_{\mathbf{w}}$ contains π_D , while the expression of π_D is unknown.

Ziebart et al. [2008] solves the expression of $P_{\mathbf{w}}$ for the optimization problem in Equation 31 by Lagrangian duality method. Surprisingly, the resulting expression of $P_{\mathbf{w}}$ is independent of π_D and the optimization problem in Equation 31 is equivalent to maximum likelihood estimation. We construct the Lagrangian dual problem in Equation 31 as the follows:

$$\min_{\mathbf{w}, \lambda} \mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda] \quad (32)$$

$$\text{where } \mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda] = \mathcal{H}(P_{\mathbf{w}}) - \mathbf{w}^T(\mu(\pi_D) - \mu(\pi^*)) - \lambda(\sum_{\tau} P_{\mathbf{w}}(\tau) - 1).$$

The dual problem allows us to express the constrained optimization problem in Equation 31 to Equation 33

$$\min_{\mathbf{w}, \lambda} \max_{P_{\mathbf{w}}} \mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda] \quad (33)$$

Note that we denote $\mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda]$ as a functional. We solve the optimal $\mathbf{w}$, λ , and $P_{\mathbf{w}}$ below. Let's first expand the expression of $\mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda]$ as below:

$$\begin{aligned} \mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda] &= \int_{\tau} -P_{\mathbf{w}}(\tau) \log P_{\mathbf{w}}(\tau) d\tau \\ &\quad - \mathbf{w}^T \mu(\pi_D) + \int_{\tau} \mathbf{w}^T P_{\mathbf{w}}(\tau) \phi(\tau) d\tau - \lambda(\int_{\tau} P_{\mathbf{w}}(\tau) d\tau - 1), \end{aligned} \quad (34)$$

where we slightly abuse the notation $\phi(\tau)$ to denote the feature summation over trajectory¹¹.

First, we solve inner problem $\max_{P_{\mathbf{w}}} \mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda]$ in Equation 33. The optimal $P_{\mathbf{w}}$ to the relaxed optimization problem can be obtained by setting $\partial \mathcal{L} / \partial P_{\mathbf{w}} = 0$ (how can we differentiate w.r.t. a function? see calculus of variations). Using Euler-Lagrange equation to $\mathcal{L}$, we arrive at the following expressions:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial P_{\mathbf{w}}} &= -\log P_{\mathbf{w}}(\tau) - 1 + \mathbf{w}^T \phi(\tau) - \lambda = 0 \\ \log P_{\mathbf{w}}^*(\tau) &= -1 + \mathbf{w}^T \phi(\tau) - \lambda \\ P_{\mathbf{w}}^*(\tau) &= \exp(-1 + \mathbf{w}^T \phi(\tau) - \lambda), \end{aligned}$$

where $P_{\mathbf{w}}^*$ denotes the optimal solution to $\max_{P_{\mathbf{w}}} \mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda]$.

Given $P_{\mathbf{w}}^*$, we solve the outer problem $\min_{\mathbf{w}, \lambda} \mathcal{L}[P_{\mathbf{w}}^*, \mathbf{w}, \lambda]$ by plugging $\log P_{\mathbf{w}}(\tau) = -1 + \mathbf{w}^T \phi(\tau) - \lambda$ to Equation 34 and setting $\partial \mathcal{L} / \partial \lambda = 0$. First, we plug the expression of $\log P_{\mathbf{w}}$ back to Equation 34:

$$\begin{aligned} \mathcal{L}[P_{\mathbf{w}}^*, \mathbf{w}, \lambda] &= \int_{\tau} -P_{\mathbf{w}}(\tau)(-1 + \mathbf{w}^T \phi(\tau) - \lambda) d\tau \\ &\quad - \mathbf{w}^T \mu(\pi_D) + \int_{\tau} \mathbf{w}^T P_{\mathbf{w}}(\tau) \phi(\tau) d\tau - \lambda(\int_{\tau} P_{\mathbf{w}}(\tau) d\tau - 1) \\ &= \int_{\tau} P_{\mathbf{w}}(\tau) d\tau - \mathbf{w}^T \mu(\pi_D) + \lambda \\ &= \underbrace{\int_{\tau} \exp(\mathbf{w}^T \phi(\tau)) d\tau}_Z \exp(-1 - \lambda) - \mathbf{w}^T \mu(\pi_D) + \lambda \\ &= Z \exp(-1 - \lambda) - \mathbf{w}^T \mu(\pi_D) + \lambda. \end{aligned}$$

Setting $\partial \mathcal{L} / \partial \lambda = 0$, we solve the optimal λ as shown below:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \lambda} &= 1 - Z \exp(-1 - \lambda) = 0 \\ \exp(-1 - \lambda) &= \frac{1}{Z}. \end{aligned}$$

¹¹ $\phi(\tau) = \sum_{s, a \in \tau} \phi(s, a)$

Plugging the $\exp(-1 - \lambda) = \frac{1}{Z}$ back to $\mathcal{L}[P_{\mathbf{w}}, \mathbf{w}, \lambda]$, we obtain

$$\begin{aligned}\mathcal{L}[P_{\mathbf{w}}^*, \mathbf{w}, \lambda^*] &= 1 + \lambda - \mathbf{w}^T \mu(\pi_D) \\ &= \log Z - \mathbf{w}^T \mu(\pi_D).\end{aligned}$$

As a result, the optimization problem in Equation 33 can be boiled down to:

$$\min_{\mathbf{w}} \log Z - \mathbf{w}^T \mu(\pi_D) = \max_{\mathbf{w}} \mathbf{w}^T \mu(\pi_D) - \log Z.$$

The above expression can be connected to maximum likelihood estimation problem as the follows:

$$\begin{aligned}& \max_{\mathbf{w}} \mathbf{w}^T \mu(\pi_D) - \log Z \\ &= \max_{\mathbf{w}} \log \exp(\mathbf{w}^T \mu(\pi_D)) - \log Z \\ &= \max_{\mathbf{w}} \log \frac{\exp(\mathbf{w}^T \mu(\pi_D))}{Z} \\ &= \max_{\mathbf{w}} \log \frac{\exp(\mathbf{w}^T \mu(\pi_D))}{Z} \\ &= \max_{\mathbf{w}} \log \frac{\int_{\tau} P_{\mathbf{w}}(\tau) \exp(\mathbf{w}^T \phi(\tau)) d\tau}{Z} \\ &= \max_{\mathbf{w}} \int_{\tau} \log P_{\mathbf{w}}(\tau) \frac{\exp(\mathbf{w}^T \phi(\tau))}{Z} d\tau \\ &= \max_{\mathbf{w}} \mathbb{E}_{\tau \sim D} \left[\frac{\exp(\mathbf{w}^T \phi(\tau))}{Z} \right],\end{aligned}$$

which indicates that the reward parameters $\mathbf{w}$ can be estimated by maximizing the likelihood of the reward distribution $\exp(\mathbf{w}^T \phi(\tau))/Z$. In summary, we bridge the maximum entropy objective and the maximum likelihood objective:

$$\begin{aligned}& \max_{P_{\mathbf{w}}} \mathcal{H}(P_{\mathbf{w}}) \\ & s.t. \mu(\pi_D) = \mu(\pi^*) \quad \iff \quad \max_{\mathbf{w}} \mathbb{E}_{\tau \sim D} \left[\frac{\exp(\mathbf{w}^T \phi(\tau))}{Z} \right] \\ & \sum_{\tau} P_{\mathbf{w}}(\tau) = 1,\end{aligned}$$

Algorithm Let the reward distribution over trajectories be

$$P(\tau|\mathbf{w}) = \frac{\exp(\mathbf{w}^T \phi(\tau))}{Z}.$$

To solve the optimization problem below

$$\max_{\mathbf{w}} J(\mathbf{w}), \quad J(\mathbf{w}) = \mathbf{w}^T \mu(\pi_D) - \log Z \quad (35)$$

Ziebart et al. [2008] suggested optimizing the objective in Equation 35 using gradient ascent, where the gradient of the objective $\nabla J(\mathbf{w})$ is expressed as:

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= \mu(\pi_D) - \nabla_{\mathbf{w}} \log \int_{\tau} \exp(\mathbf{w}^T \phi(\tau)) d\tau \\ &= \mu(\pi_D) - \frac{1}{\int_{\tau} \exp(\mathbf{w}^T \phi(\tau)) d\tau} \int_{\tau} \exp(\mathbf{w}^T \phi(\tau)) \phi(\tau) d\tau \\ &= \mu(\pi_D) - \int_{\tau} P(\tau|\mathbf{w}) \phi(\tau) d\tau \\ &= \mu(\pi_D) - \int_s D(s|\mathbf{w}) \phi(s) ds \quad (\text{write in expressions of states}),\end{aligned}$$

where $D(s|\mathbf{w})$ denotes the expected visitation frequency of a state s conditioned on the reward function parameter $\mathbf{w}$. For the calculation of state frequency $D(s|\mathbf{w})$, please refer to Algorithm 1 in Ziebart et al. [2008].

6.7.3 Generative adversarial imitation learning

The limitations of [Abbeel and Ng \[2004\]](#) and [Ziebart et al. \[2008\]](#) is the assumption of linear rewards. Drawing inspirations from generative adversarial training [Goodfellow et al. \[2014\]](#), [Ho and Ermon \[2016\]](#) proposed **Generative Adversarial Imitation Learning (GAIL)** to remove the reliance on the linear rewards assumption. We will first layout the intuition of GAIL and analyze GAIL with MaxEnt IRL framework.

Intuitively, GAIL treats IRL as a adversarial game, where a classifier tries telling if the policy is the expert or not and a learner attempts to fool the classifier's predictions. If the policy successfully fools the classifier, it indicates that the policy performs closely to the expert's policy does. Succinctly, GAIL can be formulated as the following optimization problem:

$$\max_{\theta} \min_{\mathbf{w}} \mathbb{E}_{(s,a) \sim \pi_{\theta}} [f_{\mathbf{w}}(s, a)] + \mathbb{E}_{(s,a) \sim \pi_D} [1 - f_{\mathbf{w}}(s, a)], \quad (36)$$

where $f_{\mathbf{w}} : \mathcal{S} \times \mathcal{A} \mapsto [0, 1]$ denotes the binary classifier predicting the probability of that a state-action pair (s, a) comes from the expert's policy π_D , and $\mathbf{w}$ denotes the parameters of the classifier f . The optimization problem in Equation 36 can be optimized using the following two alternating procedures

1. Train the classifier f

$$\mathbf{w} \leftarrow \mathbf{w} + \beta \left(\mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\mathbf{w}} \log f_{\mathbf{w}}(s_t, a_t)] + \mathbb{E}_{\tau_D \sim D} [\nabla_{\mathbf{w}} \log(1 - f_{\mathbf{w}}(s_t, a_t))] \right)$$

2. Train the policy π using policy gradient (e.g., TRPO [Schulman et al. \[2015b\]](#))

$$\theta \leftarrow \theta + \beta \left(\mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log Q(s_t, a_t)] \right),$$

$$\text{where } Q(s, a) = \mathbb{E}_{\tau \sim \pi_{\theta}} [f_{\mathbf{w}}(s_t, a_t) | s_1 = s, a_1 = a].$$

Connection to MaxEnt IRL and Apprenticeship Learning (TODO)

7 Learning from simulator

A central issue of reinforcement learning and imitation learning is tremendous data requirement. Unfortunately, it is expensive to collect interaction data and demonstration in the real world. One alternative of collecting data to sidestep the huge demand on data is using simulators. Collecting interaction data or demonstration in simulators is cheap since physical robots would not damage and simulation speed can be faster than the real time. In addition, the “privilege information” in simulators makes policy training easier. Privilege information are the information easily accessible in simulators, while inaccessible in the real world. For example, in simulators, one can easily get the ground truth pose of robots and take it as inputs of the policy. Ground truth poses make policy training more efficient because poses allow Markov property (see Section E) to hold. For example, [Andrychowicz et al. \[2020b\]](#) use privilege information to improve policy training for in-hand manipulation tasks. [Andrychowicz et al. \[2020b\]](#) train a policy taking object poses as inputs in simulators, and train a pose estimation network to infer the pose information for the policy in the real world.

Though simulators could produce a large amount of cheap data and useful privilege information, it is unclear whether the data from simulators match the real world. Consider training a policy using images. The images rendered in simulators are unlikely to look like natural images in the real world. Not only images, but also the physical parameters such as friction coefficients in the simulators could be different from that in the real world. Such a difference prevents the policy learned in simulators from performing well in the real world. The discrepancy between the real world and the simulators is so-called *sim-to-real* gap. In the following sections, we will focus on discussing the solutions for sim-to-real gap.

7.1 Domain randomization

As the environment parameters (hereinafter, we will use environment parameters and dynamics interchangeably) are unknown, one approach to prepare the learned policy for transferring from simulators to the real world is training a policy to cope with a range of different dynamics. This line of methods is **domain randomization** [Tobin et al., 2017]. Since the policy is trained to optimize the performance (e.g., maximizing expected return or minimizing prediction errors to the expert’s actions) over a distribution of dynamics, the policy can perform well as long as the dynamics in the real world is inside this distribution. Formally, the domain randomization technique can be summarized as the follows:

$$\max_{\pi} \mathbb{E}_{z \sim P(z)} [J_z(\pi)],$$

where z and $P(z)$ denotes the environment parameters and the distribution of the dynamics.

What is being randomized? In general, the variable z can correspond to more than environment dynamics, but anything in the environment. For example, if we train a policy taking images as inputs, the textures and lighting conditions of images can variables z being randomized. Also, if the policy outputs the reference signals for the low-level PD controller, the gain of the PD controller can also be randomized to tackle various control gains of PD controllers.

7.1.1 Performance-robustness Trade-off

As domain randomization trains the policy to optimize the average performance under a distribution of environment parameters, the resulting policy would be more “conservative” than the policy trained under a single environment dynamics. The conservative actions makes the policy robust against a variety of dynamics, while the performance would degrade. Intuitively, consider we want to train a policy for a robot to run as fast as possible on ice and rocky terrains. Given the same pose of robot (i.e., state), the policy has to output a slow velocity such that the robot will not slip on ice, while such an action slows down the robot on rocky terrains. As a result, the optimal performance of the policy trained by domain randomization will be lower than that of the policy trained under the target dynamics. The target dynamics here means the dynamics in the environment where the robot will be deployed. We illustrate this performance-robustness trade-off in Figure (TODO).

7.1.2 Advanced Domain Randomization

One strategy to ease the detrimental effect of domain randomization mentioned in Section 7.1.1 is to narrow down the range of randomization. How to narrow down the range of randomization is a central issue in domain randomization methods.

Learning to simulate Ruiz et al. [2018] uses the performance of the policy (or any predictive model) to guide the domain randomization. Consider training a predictive model h_θ to predict y given x using a dataset generated by the environment parameters (i.e., parameters of the simulator) ψ . This approach can be formulated as a bi-level optimization problem shown below:

$$\begin{aligned} \psi^* &= \arg \min_{\psi} \sum_{(x,y) \in D_{real}} \mathcal{L}(h_\theta(x), y) \\ s.t. \theta^*(\psi) &= \arg \min_{\theta} \sum_{(x,y) \in D_\psi} \mathcal{L}(h_\theta(x), y), \end{aligned}$$

where D_{real} and D_ψ correspond to the data from the real world and that from ψ . This bi-level optimization program alternate between optimizing the simulator ψ and the policy (or any predictive model) θ . The resulting ψ is a simulator maximizing the performance of h in the real dataset.

Automatic image augmentation Raileanu et al. [2021] consider training a policy in an image state space. To be robust against variations of images, Raileanu et al. [2021] employed a data augmentation technique to randomize the training images (i.e., states) by a variety of image operations (e.g., scaling, transformation, or flipping). Image augmentation can be viewed as performing domain randomization over the image state space, where the set of image operations can be viewed as the range of randomization. Formulating operations selection as a multi-arm bandit (MAB) problem, Raileanu et al. [2021] employ upper-confidence bound (UCB) (see Section 2) to choose image operations to augment the dataset.

Canonical representations James et al. [2019] considers transferring a policy taking images as inputs from simulators to the real world. The sim-to-real gap is bridged by training the model using canonical intermediate representation as inputs. James et al. [2019] train a transformation network converting perturbed images back to the canonical ones. Formally, let the canonical image s as the image rendered by the simulator, and the perturbed images $\hat{s} = g(s)$, where g denotes the operation of perturbation. A transformation network f is trained to convert perturbed images $\hat{s}$ back to s , as shown below:

$$\min_f \sum_{s \in \mathcal{S}} \mathcal{D}(s, g(s)),$$

where $\mathcal{D}$ denotes an arbitrary distance metric between images. The policy π is then trained using canonical representations s . At the deployment time in the real world, a real world image can be treated as perturbed image $\hat{s}$. Converting the real world image $\hat{s}$ back to canonical representations s , we enable the policy π to take inputs identical to that used in the training time.

7.2 System identification and domain randomization

As we mentioned in Section 7.1.1, one caveat of domain randomization is that the learned policy is overly conservative and consequently limits the best performance that a learning algorithm can attain. For example, intuitively, if a policy is trained to move on “all” terrains, the policy is forced to learn a behavior that can walk smoothly in both rugged hills and flat ground. It’s because the policy π only perceives observation o from the environment while both terrains appear to be indistinguishable in o (e.g., image observation doesn’t reveal the friction of grounds). As such, the policy will not make the robot spring even on flat ground since the policy has to ensure the robot won’t fall on rugged terrains. The policy turns out to be “overly pessimistic.”

This over-pessimism issue stems from the policy being blind to the type of terrain or, more generally, the “system parameters” (e.g., terrain type) during training with domain randomization. Can we address the over-pessimism issue if we know the system parameter? One strategy to tackle this issue is training a policy conditioned on the system parameter with domain randomization. For example, Lee et al. [2020] trains a policy taking a quadruped state (e.g., proprioception) and the terrain’s friction as inputs, on randomly generated terrains with different friction. Taking the system parameter (i.e., friction in this example) as inputs, the policy can act differently on different terrains, while the remaining question is the system parameters are only available in simulators yet unknown in the real world. How do we know the system parameter such as friction in the real world when deploying robots? For instance, how do you know the friction of floor you are walking on? The short answer is: inferring the system parameter from the observations, which is also known as *system identification*. The intuition is that the friction of the ground can be inferred when we walk. If exerting a small force leads to large movement, the ground is likely to be slippery. In the following, we formalize the idea of a training policy conditioned on system parameters and system identification.

7.2.1 Training system parameter conditioned policy

Since system parameters are part of inputs to the policy when training policy, the policy optimization objective can be written as follows:

$$\max_{\pi \in \Pi} \mathbb{E}_{z \sim p_z, a_t \sim \pi(\cdot | o_t, z)} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, z) \right], \quad (37)$$

where p_z denotes the parameters of system parameters that are randomized by domain randomization. Optimizing policy with Equation 37 gives rise to a policy conditioned on system parameters z . This conditioned policy can be viewed as a set of nearly optimal policies, where each z determines an optimal policy in the set.

7.2.2 System identification

Now, as long as we know the system parameters z in the real world (or deployment environment), we are able to retrieve the optimal policy corresponding to the real world from this set of policies. As the system parameters cannot be inferred from one observation (e.g., one cannot infer friction by standing around the same place), the line of works on system identification builds a model based on the history of observations.

Explicit parameter inference. One approach to approximate the system parameters is solving the empirical risk minimization problem shown below:

$$f^* \in \arg \min_{f \in \mathcal{F}} \mathbb{E}_h [(z - f(h))^2], \quad (38)$$

where z is the ground truth of environmental dynamics, $h := [o_1, \dots, o_H]$ denotes a sequence of observations with length H , and $\mathcal{F}$ denotes the model class (e.g., neural network architecture). Note that h doesn't have to be sampled from trajectories generated by a specific policy (in other words, the training data for f are not required to be on-policy). Putting them all together gives rise to the following algorithm:

1. Solve the system parameter z conditioned policy π for Equation 37 using domain randomization in simulators
2. Infer the system parameter z^* in the real world
3. Execute the policy with the inferred system parameter z^*

Implicit parameter inference (teacher-student learning). One caveat of explicit parameter inference is that some of the system parameters may be irrelevant to the current task. For example, supposing the lighting condition is part of system parameter z , we may randomize lighting conditions for training a quadruped walking policy (presume the policy does not use vision inputs) but the lighting condition is irrelevant to the walking task. Apparently, inferring lighting conditions is not necessary and may even result in noisy signals explicit parameter inference. In this simple example, it's obvious that lighting conditions can be ignored during domain randomization. However, what if we don't know what system parameters may be relevant to the task apriori? It may be difficult for humans to determine the relevance to the task. A well-known paradigm to tackle this problem is: teacher-student learning. The idea is simple: *inferring what's necessary to learn a good policy in the task*. How do we know if the inferred system parameters are sufficient for learning a good policy for the task? One possible criterion is to check if we can match the actions generated by a policy with access to true system parameters in every observation. Intuitively, inferring the system parameter matching this criterion is like an action-guessing game between two policies where only one policy has the access to the true system parameters. Let $\pi(\cdot, z)$ be the policy that is trained with system parameters domain randomization and has the access to the true system parameters z , and $\pi(\cdot, h)$ be the policy accessing to the history of observations and actions. Since $\pi(\cdot, z)$ has the access to true system parameter z , we name $\pi(\cdot, z)$ as a teacher policy (i.e., $\pi_{teacher}$) and $\pi(\cdot, h)$ (i.e., $\pi_{student}$) as a student policy. This action-guessing game runs as follows: given an observation o_t at timestep t , the system parameters z , and history up to the current timestep h_t , we sample an action $a_{teacher} \sim \pi_{teacher}(\cdot | o_t, z_t)$ and ask $\pi_{student}(\cdot | o_t, h_t)$ to guess $a_{teacher}$ based on history. If $\pi_{student}$ ends up being able to match the teacher's action at most of the observations and system parameters, it implies that the student policy can copy the expert's policy using history without access to the true system parameters. As such, we obtain a policy that can be deployed to the real world since it only requires history. Formally, the generic teacher-student learning algorithm can be described as follows:

1. Solve the system parameter z conditioned policy $\pi_{teacher}$ for Equation 37 using domain randomization in simulators

2. Solve the history conditioned policy $\pi_{student}$ by

$$\pi_{student} \in \arg \min_{\pi \in \Pi} \mathbb{E}_{(o_t, h_t, z)} [D(\pi_{student}(\cdot | o_t, h_t), \pi_{teacher}(\cdot | o_t, z))], \quad (39)$$

where D can be any distance or divergence measure between two probability distributions over actions. For example, if both $\pi_{teacher}$ and $\pi_{student}$ are Gaussian distributions, defining D as KL-divergence leads to mean-squared error.

3. Execute the student policy in the real world (or the target simulator)

8 Model-based control (in progress)

The lecture note so far mainly considers finding a policy maximizing the expected return reinforcement learning (RL) and imitation learning (IL). While showing great performance in many tasks and being easy to implement, the caveat is that RL and IL both require an intensive amount of data from the environment interaction or the expert demonstration, which are not always available. For example, deploying a policy and collecting data for RL in the real world is not preferable due to potential safety risks and also takes a lot of time. Even deploying in simulators could have efficiency concerns since some simulators take much time to complete one trajectory simulation (e.g., science applications). Also, it's likely that no enough expert demonstration is available for IL if the task is also difficult for humans. Overall, the cost of collecting data, either from simulators or the real world, can be a bottleneck of RL and IL. Formally speaking, the cost of collecting data is related to the cost of querying expert and the cost of querying the transition function (also known as the forward dynamics model) $\mathcal{T}$. In this section, we will present a paradigm that reduces the demands on querying the forward dynamics model.

Querying the forward dynamics model $\mathcal{T}$ is often costly, but do we really need a perfect simulation or interaction with the real world all the time? Possibly not really. The established literature in model-based control in robotics [cite] and process control [cite] shows that approximated forward dynamics models can produce a satisfactory performance on a variety of control tasks. Also, learning an approximated forward dynamics model is o

As estimating an approximation of the forward dynamics model

8.1 Model Predictive Control

The general problem formulation of receding horizon control is defined as:

$$\begin{aligned} \max_{a_{1:H}} \sum_{t=1}^H r(s_t, a_t) \\ s.t. \ s_{t+1} = \mathcal{T}(s_t, a_t) \end{aligned} \quad (40)$$

where s_1 is given.

8.1.1 Cross entropy method (CEM)

Notations

- N : Population size
- M : Number of CEM iterations
- H : Planning horizon
- E : Number of elites
- α : Step size for updating the CEM parameters
- $\mathcal{T}$: State transition function (i.e., forward dynamics model)
- r_f : Terminal rewards

Algorithm 7 Cross entropy method (CEM)

```

1: procedure CEM( $s_1, N, M, H, E, \alpha, \mathcal{T}$ )
2:   for  $m = 1 \cdots M$  do
3:     Initialize the action distributions over  $H$  steps:  $\mathcal{N}(\mu_{1:H}, \Sigma_{1:H})$ 
4:     Sample action sequences:  $a_{1:H}^i \sim \mathcal{N}(\mu_{1:H}, \Sigma_{1:H}) \quad \forall i \in [1, N]$ 
5:     Rollout the trajectories:  $s_{1:H}^i \quad \forall i \in [1, N]$ , where  $s_{t+1}^i = \mathcal{T}(s_t^i, a_t^i) \quad \forall t \in [1, H]$ 
6:     Compute the trajectory return:  $R(a_{1:H}^i) = \sum_{t=1}^H r(s_t^i, a_t^i) + r_f(s_{H+1}^i)$ 
7:     Sort action sequences by returns:
8:        $\tilde{a}_{1:H}^1, \dots, \tilde{a}_{1:H}^N = \text{DESCENDARGSORT}(R(a_{1:H}^1), \dots, R(a_{1:H}^N))$ 
9:     Update mean:  $\mu_{1:H} \leftarrow \alpha \text{MEAN}(\tilde{a}_{1:H}^{1:E}) + (1 - \alpha) \mu_{1:H}$ 
10:    Update co-variance:  $\Sigma_{1:H} \leftarrow \alpha \text{VARIANCE}(\tilde{a}_{1:H}^{1:E}) + (1 - \alpha) \Sigma_{1:H}$ 
11:  end for
12:  Return  $\mu_{1:H}, \Sigma_{1:H}$ 
13: end procedure

```

Algorithm 8 Conservative Cross entropy method (CCEM)

```

1: procedure CONSERVATIVE_CEM( $s_1, N, M, H, E, \alpha, \mathcal{T}_{1:B}$ )
2:   for  $m = 1 \cdots M$  do
3:     Initialize the action distributions over  $H$  steps:  $\mathcal{N}(\mu_{1:H}, \Sigma_{1:H})$ 
4:     Sample action sequences:  $a_{1:H}^i \sim \mathcal{N}(\mu_{1:H}, \Sigma_{1:H}) \quad \forall i \in [1, N]$ 
5:     for  $b = 1 \cdots B$  do
6:       Rollout the trajectories:  $s_{1:H}^{i,b} \quad \forall i \in [1, N]$ , where  $s_{t+1}^{i,b} = \mathcal{T}_b(s_t^{i,b}, a_t^{i,b}) \quad \forall t \in [1, H]$ 
7:       Compute the trajectory return:  $R_b(a_{1:H}^i) = \sum_{t=1}^H r(s_t^{i,b}, a_t^{i,b}) + r(s_{H+1}^{i,b}, a_{H+1}^{i,b})$ 
8:       Sort action sequences by returns:
9:          $\tilde{a}_{1:H}^{1,b}, \dots, \tilde{a}_{1:H}^{N,b} = \text{DESCENDARGSORT}(R_b(a_{1:H}^1), \dots, R_b(a_{1:H}^N))$ 
10:      Sort returns:  $\tilde{R}_b(a_{1:H}^1), \dots, \tilde{R}_b(a_{1:H}^N) = \text{SORT}(R_b(a_{1:H}^1), \dots, R_b(a_{1:H}^N))$ 
11:      Compute the average returns of elites:  $\bar{R}_b = \frac{1}{N} \sum_{i=1}^N \tilde{R}_b(a_{1:H}^i)$ 
12:    end for
13:    Select the worst member:  $b^* \leftarrow \arg \min_b \bar{R}_b$ 
14:    Update mean:  $\mu_{1:H} \leftarrow \alpha \text{MEAN}(\tilde{a}_{1:H}^{1:E,b^*}) + (1 - \alpha) \mu_{1:H}$ 
15:    Update co-variance:  $\Sigma_{1:H} \leftarrow \alpha \text{VARIANCE}(\tilde{a}_{1:H}^{1:E,b^*}) + (1 - \alpha) \Sigma_{1:H}$ 
16:  end for
17:  Return  $\mu_{1:H}, \Sigma_{1:H}$ 
18: end procedure

```

8.2 Inverse model approach

9 Dynamic Programming Method (draft)

This section will introduce dynamic programming (DP) method for sequential decision making (SDM) problem. Using the optimal substructure of value function, DP performs more sample efficiently than policy gradient methods (see Section 4) under mild assumptions (we will discuss its limitations later). We motivate why DP methods are more sample efficient in the following example.

Motivating example: shortest path: Consider an SDM problem, finding a single-source-single-destination shortest path on a graph, where this graph is a directed acyclic graph (DAG) illustrated in Figure 6. This DAG consists of 4 chains of states where each chain is of length T . The agent is initialized at the starting state (blue node). Visiting each edge incurs -1 reward (i.e., 1 cost). An episode terminates when the agent reaches the goal state (goal).

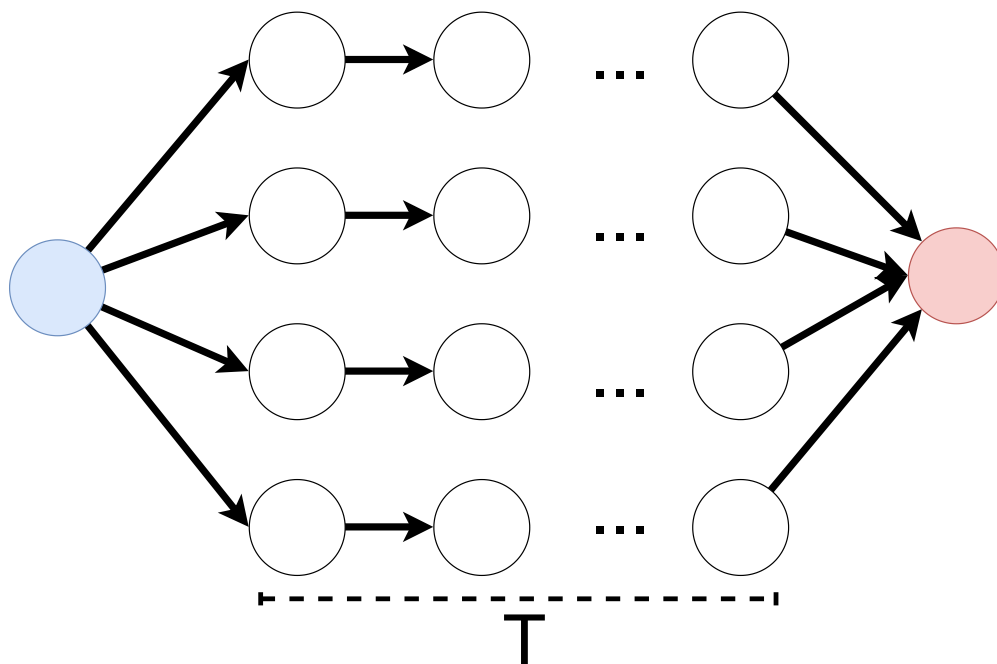


Figure 6: The environment for the example of shortest path finding. This environment is a directed acyclic graph (DAG), where the blue and the red nodes denote starting and goal states, respectively. The reward of visiting each edge is -1 .

Why policy gradient methods are inefficient? Let's first see how policy gradient methods can be inefficient in this example. Recall that policy gradient methods maximize the expected return over the trajectories (i.e., paths) sampled by the policy. Policy gradient methods can be viewed as searching for the shortest path over all possible paths on the graph. At each state on the chain, the agent has 4 choices of next states. Thus, there are 4^T possible paths where each path has length T . Roughly speaking, policy gradient methods can be viewed as performing random shooting over possible paths, which means that the number of steps required for finding the shortest path can exponentially grow.

Optimal substructure is useful The fundamental reason that the policy gradient method is inefficient is that the optimal substructure of the subproblems is overlooked. Loosely speak-

ing, a problem has an optimal substructure if the solution to the problem can be constructed using the solutions to the sub-problems of the original problem. Shortest path finding is a classic example of DP to illustrate the optimal substructure since the distance to the goal from a state can be divided into the follows by recursion:

$$\text{Dist}(s) = \begin{cases} 1 + \min_{s' \in \text{Suc}(s)} \underbrace{\text{Dist}(s')}_{\text{Solutions to sub-problems}}, & s \neq \text{goal state} \\ 0, & s = \text{goal state.} \end{cases}$$

where $\text{Dist}(s)$ denotes the distance to the goal state from state s and $\text{Suc}(s)$ is defined as $\{s' \mid \text{There is an edge from } s \text{ to } s'\}$. By this formulation, one can get a Dist function for all states $s \in \mathcal{S}$ by solving $\text{Dist}(s)$ backward from the goal state. In turn, one simply requires $4T$ steps to find the shortest path.

In summary, DP approaches can be more efficient than policy gradient methods because DP approaches just requires solving the subproblems (i.e., updating the Dist function) over each state in the state space, instead of sampling trajectories (i.e., all the combinations of states) over and over again for updating the policy. This nature makes DP approaches *off-policy*. Off-policy algorithms can learn the policy from data generated by an arbitrary policy, while on-policy algorithms such as policy gradient require data sampled from the current policy. As such, off-policy algorithm can reuse data collected by previous policies during training.

In the following sections, we will introduce how DP is used for solving SDM. In DP approach, the work horse is value function introduced in Section 4.3. The difference to previous sections is that we break down the value function using the optimal substructure as shown below

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi \left[\sum_{i=t}^T \gamma^{i-t} r(s_i, a_i) \mid s_t = s \right] \\ &= \mathbb{E}_\pi \left[r(s_t, a_t) + \gamma \sum_{i=t+1}^T \gamma^{i-t} r(s_i, a_i) \mid s_t = s \right] \\ &= \mathbb{E}_\pi \left[r(s_t, a_t) + \gamma V^\pi(s_{t+1}) \mid s_t = s \right] \end{aligned} \tag{41}$$

9.1 Policy iteration & Value Iteration

This section introduces two classic DP methods, policy iteration (PI) and value iteration (VI), for solving SDM. In contrast to policy gradient methods estimating the policy directly, PI and VI are using value function to derive the policy. Note that PI and VI methods assume the access to the state transition function $\mathcal{T}$ and the reward function r . This assumption is impractical in most of environments (e.g., video game playing or humanoid robot control). We introduce PI and VI in this section because they are the foundations to establish state-of-the-art DP methods in computational sensorimotor learning.

9.1.1 Policy iteration

Policy iteration approaches the optimal policy by two steps: policy evaluation and policy improvement. Policy evaluation step evaluates the performance of the current policy; policy improvement step derives a new policy improving beyond the previous policy. Overall, policy iteration can be summarized as Algorithm 9.

Algorithm 9 Policy Iteration

```

1: procedure POLICYITERATION( $\epsilon$ )
2:   Initialize policy  $\pi_1$  arbitrarily [Sutton and Barto, 2018]
3:   for  $i = 1 \dots$  do
4:      $V^{\pi_i} \leftarrow \text{POLICYEVALUATION}(\pi_i, \epsilon)$ 
5:      $\pi_{i+1} \leftarrow \text{POLICYIMPROVEMENT}(V^{\pi_i})$ 
6:     if  $\pi_i(s) = \pi_{i+1}(s) \forall s \in \mathcal{S}$  then
7:       break
8:     end if
9:   end for
10: end procedure

```

Policy evaluation The goal of policy evaluation is to find a V^π such that

$$V^\pi(s) = \mathbb{E}_\pi[r(s_t, a_t) + \gamma V^\pi(s_{t+1}) | s_t = s] \quad \forall s \in \mathcal{S}.$$

Such a V^π can be solved by linear programming approach, while it may entail a large number of constraints in the linear program. In this section, we focus on introducing a more generally suitable approach, iterative policy evaluation. Iterative policy evaluation is based on fixed point iteration. Let V_i be the approximated value function at i -th iteration. We can define as fixed point iteration procedure as shown below:

$$V_{i+1}(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a_t = a | s_t = s) \sum_{s' \in \mathcal{S}} \mathcal{T}(s_{t+1} = s' | s_t = s, a_t = a) [r(s, a) + \gamma V_i(s')] \quad \forall s \in \mathcal{S}$$

In this iteration update rule, the fixed point solution V_k satisfies $V_k = V_{k+1}$. Thus, the fixed point solution $V_k = V^\pi$ since:

$$V_i(s) = \mathbb{E}_\pi[r(s_t, a_t) + \gamma V_i(s_{t+1}) | s_t = s] \quad \forall s \in \mathcal{S}.$$

We summarize this iterative policy evaluation approach in Algorithm 10, where $\|\cdot\|_\infty$ denotes infinity norm.

Algorithm 10 Iterative policy evaluation

```

1: procedure POLICYEVALUATION( $\pi, \epsilon$ )
2:   Initialize  $V_1(s) = 0 \forall s \in \mathcal{S}$ 
3:   for  $i = 1 \dots$  do
4:      $V_{i+1}(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} \mathcal{T}(s' | s, a) [r(s, a) + \gamma V_i(s')] \quad \forall s \in \mathcal{S}$ 
5:     if  $\|V_{i+1} - V_i\|_\infty \leq \epsilon$  then
6:       break
7:     end if
8:   end for
9: end procedure

```

Policy improvement Improving the policy π is equivalent of finding a policy π' such that

$$V^{\pi'}(s) \geq V^\pi(s) \quad \forall s \in \mathcal{S}.$$

At the first glance, finding such a policy π' may require performing policy evaluation over a large number of π' and pick the one satisfying the above constraint. Fortunately, policy improvement theorem [Sutton and Barto, 2018] suggests an useful property allowing one

to find such a π' by inspecting the the value function of the current policy π . Before we construct π' using policy improvement theorem, let's first define the Q-function as below

$$Q^\pi(s, a) = \mathbb{E}_\pi[r(s_t, a_t) + \gamma V^\pi(s_{t+1}) | s_t = s, a_t = a] \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

Since policy improvement theorem shows that for any π'

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s) \quad \forall s \in \mathcal{S} \implies V^{\pi'}(s) \geq V^\pi(s) \quad \forall s \in \mathcal{S},$$

one can construct π' by greedily selecting actions w.r.t. Q^π as shown below

$$\begin{aligned} \pi'(s) &= \arg \max_a Q^\pi(s, a) \quad \forall s \in \mathcal{S} \\ &= \arg \max_a \mathbb{E}_\pi[r(s_t, a_t) + \gamma V^\pi(s_{t+1}) | s_t = s, a_t = a] \quad \forall s \in \mathcal{S}. \end{aligned}$$

Since $\max_a Q^\pi(s, a) \geq V^\pi(s) \quad \forall s$, such a policy π' is guaranteed that $Q^\pi(s, \pi'(s)) \geq V^\pi(s) \quad \forall s$. In summary, the policy improvement step can be constructed as Algorithm 11.

Algorithm 11 Policy Improvement

```

1: procedure POLICYIMPROVEMENT( $V^\pi$ )
2:    $\pi'(s) = \arg \max_a \mathbb{E}_\pi[r(s_t, a_t) + \gamma V^\pi(s_{t+1}) | s_t = s, a_t = a] \quad \forall s \in \mathcal{S}$ 
3: end procedure

```

9.1.2 Value iteration

One caveat of policy iteration is that we need to perform policy evaluation for each policy π during iteration, which results in huge time complexity. To reduce the time complexity for each iteration, the idea of value iteration is to mix policy evaluation and policy improvement in one step so that we do not require running a full policy evaluation step for each iteration. The value iteration method is layout in Algorithm 12.

Algorithm 12 Value Iteration

```

1: procedure VALUEITERATION( $\pi, \epsilon$ )
2:   Initialize  $V_1(s) = 0 \quad \forall s \in \mathcal{S}$ 
3:   for  $i = 1 \dots$  do
4:      $V_{i+1}(s) \leftarrow \max_a \sum_{s' \in \mathcal{S}} \mathcal{T}(s' | s, a) [r(s, a) + \gamma V_i(s')] \quad \forall s \in \mathcal{S}$ 
5:     if  $\|V_{i+1} - V_i\|_\infty \leq \epsilon$  then
6:       break
7:     end if
8:   end for
9:   Set policy  $\pi(s) = \arg \max_a \mathbb{E}_\pi[r(s_t, a_t) + \gamma V^\pi(s_{t+1}) | s_t = s, a_t = a] \quad \forall s \in \mathcal{S}$ 
10: end procedure

```

9.1.3 Comparison between policy iteration and value iteration

Policy iteration has higher time complexity per iteration than value iteration. For each iteration, the time complexity of policy iteration is $O(|\mathcal{S}|^3 + |\mathcal{S}|^2|\mathcal{A}|)$. The term $|\mathcal{S}|^3$ in the time complexity results from policy evaluation step. This time complexity of policy evaluation can be obtained by inspecting the solution of V^π for $V^\pi(s) = \mathbb{E}_\pi[r(s_t, a_t) +$

$\gamma V^\pi(s_{t+1})|s_t = s]$ $\forall s \in \mathcal{S}$ in a matrix multiplication form¹². For value iteration, the time complexity is $O(|\mathcal{S}|^2|\mathcal{A}|)$.

However, policy iteration often requires less number of iterations to obtain the optimal policy in practice. It is because policy iteration only requires the relative ranking of the values for each action to be correct, while value iteration will not stop until the value function converge.

9.2 Q-Learning

Q-Learning [Watkins and Dayan, 1992] is a more practical algorithm than value iteration because Q-Learning (i) does not require the access to the state transition function and (ii) does not require a large sweep over the entire state space for updating the value function.

Instead of sweeping over the entire state space, Q-Learning updates the Q-function using data (s_t, a_t, r_t, s_{t+1}) . The data can be collected by any policy since the Q-Learning formulation removes the reliance on the samples from the current policy. This makes Q-Learning an *off-policy* algorithm.

Because of the model-free property (i.e., not rely on state transition function $\mathcal{T}$) and off-policy nature, Q-Learning is the core of a number of state-of-the-art reinforcement learning algorithms. In this section, we will start by the simplest formulation of Q-Learning, and cover more advanced variants of Q-Learning algorithms.

9.2.1 Tabular Q-Learning

In tabular Q-Learning, we assume the state space $\mathcal{S}$ and the action space $\mathcal{A}$ are finite. Recall that the Q-function is defined as:

$$Q^\pi(s, a) = \mathbb{E}_\pi[r(s_t, a_t) + \gamma V^\pi(s_{t+1})|s_t = s, a_t = a] \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

Let $\pi(s) = \arg \max_a Q^\pi(s, a) \quad \forall s \in \mathcal{S}$. We can rewrite the Q-function as:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi[r(s_t, a_t) + \gamma \max_{a'} Q^\pi(s_{t+1} = s', a')|s_t = s, a_t = a] \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \\ &= r(s_t = s, a_t = a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s'|s, a) [\max_{a'} Q^\pi(s_{t+1} = s', a')] \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \\ &= r(s_t = s, a_t = a) + \gamma \mathbb{E}_{s'} [\max_{a'} Q^\pi(s_{t+1} = s', a')] \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \end{aligned}$$

Similar to policy evaluation (see Section 9), $Q^\pi(s, a)$ can be estimated by fixed point iteration method, where we define the update rule for each iteration i as below:

$$Q_{i+1}(s, a) \leftarrow r(s_t = s, a_t = a) + \gamma \mathbb{E}_{s'} [\max_{a'} Q_i(s_{t+1} = s', a')] \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

Instead of computing $\mathbb{E}_{s'} [\max_{a'} Q_i(s_{t+1} = s', a')]$ using the state transition function $\mathcal{T}$, Q-Learning samples s' from the interaction with the environment.

As Q-Learning is designed to update the Q-function interleavely with online data collection, Q-Learning updates the Q-function with a small step size α incrementally as shown below:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)), \quad (42)$$

¹²See this link for the details <https://rltheory.github.io/lecture-notes/planning-in-mdps/lec4/>

where each step of online data collection produces a tuple $(s, a, r(s, a), s')$. Typically the data is collected by a *behavior policy* π_b instead of the greedy policy $\pi(s) = \arg \max_a Q(s, a)$ since the greedy policy may not be collect data with sufficient coverage over the entire state space $\mathcal{S}$. A typical choice of the behavior policy π_b is ϵ -greedy policy shown as below:

$$\pi_b(s) = \begin{cases} \arg \max_a Q(s, a), & \text{Uniform}(0, 1) \geq \epsilon \\ \text{Uniform}(\mathcal{A}), & \text{otherwise.} \end{cases}$$

The Q-Learning algorithm is summarized in Algorithm 13.

Algorithm 13 TabularQLearning

```

1: procedure TABULARQLearning
2:   Initialize  $Q(s, a) = 0 \ \forall s \in \mathcal{S}, a \in \mathcal{A}$ 
3:   while not terminate do
4:     Observe the state  $s$  and select action  $a = \pi_b(s)$ 
5:      $Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a))$ 
6:   end while
7: end procedure

```

9.2.2 Fitted Q iteration

In this section, we will cover how function approximation enables Q-Learning to scale up to larger state spaces. The tabular Q-Learning introduced in Section 9.2.1 cannot scale up to a continuous state space $\mathcal{S}$, where the elements of $\mathcal{S}$ is not enumerable. In tabular Q-Learning, the Q-function is represented as a table with $|\mathcal{S}| \times |\mathcal{A}|$ entries. As in a continuous state space (e.g., $\mathbb{R}^3$), $|\mathcal{S}| \rightarrow \infty$, it is impossible to build a table over the state space. Even though discretizing the state space can turn a continuous state space into a grid of states for building the table (see tile encoding in Sutton and Barto [2018]), such a discretized state space often has a considerably large size of $\mathcal{S}$. In addition, such a state space with a large number of states also poses a challenge to tabular Q-Learning since updating and building the tabular Q-function will take much time.

An alternative is to represent the Q-function using a set of basis functions [Sutton and Barto, 2018]. The Q-function is then represented as the following:

$$Q_\phi(s, a) = \sum_{i=1}^d \mathbf{w}_i \phi_i(s, a),$$

where $\mathbf{w}_i$ and ϕ_i denotes the i -th coefficient and the i -th basis. As such, discretization over the state space is not required. The Q-function is treated as a linear mapping $Q_\phi : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$. The challenge of using linear basis function approximation is designing the basis function ϕ . Typical choices include radial Fourier basis [Konidaris et al., 2011] or neural networks [Riedmiller, 2005]

The use of function approximation prevents the Q-function from being updated using direct value assignment, as shown in Equation 42. In fitted Q-iteration (FQI) proposed in Riedmiller [2005] treats the Q-function updates as a regression problem shown as below:

$$\min_{\phi} \frac{1}{|\mathcal{D}|} \sum_{(s, a, r, s') \in \mathcal{D}} \|y - Q_\phi(s, a)\|^2,$$

where $\mathcal{D}$ denotes the dataset $\{(s, a, r, s')\}$ and $y = r(s, a) + \gamma \max_{a'} Q(s', a')$ denotes the regression target for $Q_\phi(s, a)$. Note that as Q-Learning does not require samples from the current policy, the dataset $\mathcal{D}$ can be produced by an arbitrary policy. The FQI algorithm can be summarized as Algorithm 14

Algorithm 14 Fitted Q iteration

```

1: procedure FQI
2:   Initialize the neural network weights  $\phi$  randomly
3:   while not terminate do
4:     Collect dataset  $\mathcal{D}$  using an arbitrary policy to interact with the environment
5:     for  $i = 1 \dots$  do
6:       Compute the target  $y \leftarrow r(s, a) + \gamma \max_{a'} Q_\phi(s', a') \quad \forall (s, a, r, s') \in \mathcal{D}$ 
7:       Update the network  $\phi \leftarrow \arg \min_{\phi'} \frac{1}{|\mathcal{D}|} \sum_{(s, a, r, s') \in \mathcal{D}} \|y - Q_{\phi'}(s, a)\|^2$ 
8:     end for
9:   end while
10: end procedure

```

9.2.3 Deep Q Network (DQN)

This section introduces deep Q network (DQN) showing superior performance in video game playing [Mnih et al., 2015] using only images as inputs. We will start by comparing DQN with FQI, and then discuss the challenges of DQN and the corresponding solutions/workarounds.

Difference to FQI Using function approximation to learn the Q-function requires a large dataset in complex environments with large state spaces such as images [Mnih et al., 2015]. Moreover, it is unlikely to collect a dataset covering the whole state space in a few iterations. As a result, gradient descent used in FQI is prohibitive. To overcome this limitation, deep Q network (DQN) learning [Mnih et al., 2015] uses stochastic gradient descent (SGD) to update the Q-function interleavely with dataset collection. We summarize the online learning procedure of DQN in Algorithm 15.

Algorithm 15 Online DQN

```

1: procedure ONLINEDQN( $\alpha$ )
2:   Initialize the neural network weights  $\phi$  randomly
3:   while not terminate do
4:     Observe  $s$ , take action  $a \leftarrow \pi(s)$ , and receive reward  $r$  and next state  $s'$ 
5:     Compute the target  $y \leftarrow r(s, a) + \gamma \max_{a'} Q_\phi(s', a')$ 
6:     Update the network  $\phi \leftarrow \phi + \alpha \nabla_\phi \|y - Q_\phi(s, a)\|^2$ 
7:   end while
8: end procedure

```

Experience replay However, the minibatches of data collected online violates the identically independent distributed (i.i.d.) assumption of SGD. The data consists of sequences of states, actions, and rewards during interaction with the environment. As these data are collected sequentially, high correlation among data violates the i.i.d. assumption. Mnih et al. [2015] proposed to use experience replay [Lin, 1992] to make the training data satisfy i.i.d.

assumption. Using experience replay, we gather data (s, a, r, s') collected online as a huge buffer and randomly sample minibatches of data from the buffer for training the Q-network. The online DQN procedure with experience replay is outlined in Algorithm 16.

Algorithm 16 Online DQN with experience replay

```

1: procedure ONLINEDQN( $\alpha$ )
2:   Initialize the neural network weights  $\phi$  randomly
3:   Initialize the experience replay buffer  $\mathcal{Z} = \emptyset$ 
4:   while not terminate do
5:     Observe  $s$ , take action  $a \leftarrow \pi(s)$ , and receive reward  $r$  and next state  $s'$ 
6:     Append  $(s, a, r, s')$  to  $\mathcal{Z}$ 
7:     Sample a minibatch  $\mathcal{B} \sim \text{Uniform}(\mathcal{Z})$ 
8:     Compute the target  $y \leftarrow r(s, a) + \gamma \max_{a'} Q_{\phi}(s', a') \forall (s, a, r, s') \in \mathcal{B}$ 
9:     Update the network  $\phi \leftarrow \phi + \alpha \nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \|y - Q_{\phi}(s, a)\|^2$ 
10:  end while
11: end procedure

```

Stationary targets In addition to the issues of data correlation, such an online learning procedure raises an odd issue of non-stationary Q-value targets in Q-function updates. Recall that in FQI (Algorithm 14), the Q-value target y is pre-computed before the gradient descent, thus being stationary while computing the gradient of the Q-function. In contrast, the Q-value targets changes across batches of updates in SGD. Since the Q-value targets $r(s, a) + \gamma \max_{a'} Q(s', a')$ “bootstraps” the prediction from the Q-function, updating the Q-function affects the Q-value targets y . Mnih et al. [2015] showed that naively updating the Q-function online using SGD did not exhibit satisfactory performance. To tackle the problem of non-stationary value targets, Mnih et al. [2015] proposed to use delayed Q-value networks as the value target. Let $Q_{\bar{\phi}}$ as the target network giving the Q-value targets, where $\bar{\phi}$ is the old copy of the weights ϕ of the Q-value network. As another Q-value network updating faster is used for selecting actions during exploration, this Q-network is termed as “behavior Q-network”. Adding the target network to Algorithm 16, we modified the online DQN algorithm to Algorithm 17.

Algorithm 17 Online DQN with target network

```

1: procedure ONLINEDQN( $\alpha$ )
2:   Initialize the neural network weights  $\phi$  randomly
3:   Initialize the experience replay buffer  $\mathcal{Z} = \emptyset$ 
4:   while not terminate do
5:     Observe  $s$ , take action  $a \leftarrow \pi(s)$ , and receive reward  $r$  and next state  $s'$ 
6:     Append  $(s, a, r, s')$  to  $\mathcal{Z}$ 
7:     Sample a minibatch  $\mathcal{B} \sim \text{Uniform}(\mathcal{Z})$ 
8:     Compute the target with  $\bar{\phi} \ y \leftarrow r(s, a) + \gamma \max_{a'} Q_{\bar{\phi}}(s', a') \forall (s, a, r, s') \in \mathcal{B}$ 
9:     Update the network  $\phi \leftarrow \phi + \alpha \nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \|y - Q_{\phi}(s, a)\|^2$ 
10:    if time to update the target network then
11:       $\bar{\phi} \leftarrow \phi$ 
12:    end if
13:  end while
14: end procedure

```

Overestimation bias Hasselt [2010] showed that Q-Learning suffers from overestimation bias in the Q-value target due to max operations. Overestimating Q-value means that $Q_\phi(s, a) > Q(s, a)$, where $Q(s, a)$ is the ground truth Q-value. Van Hasselt et al. [2016] shows that overestimation results from approximation errors of the Q-function (see Theorem 1 in Van Hasselt et al. [2016]). Consider for a state s , where $Q(s, a) = V(s) \forall a \in \mathcal{A}$, and the Q-value approximation error is

$$C = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} (Q_\phi(s, a) - V(s))^2.$$

Van Hasselt et al. [2016] shows that the lower bound of the $\max_a Q_\phi(s, a)$ is the follows:

$$\max_a Q_\phi(s, a) \geq V(s) + \sqrt{\frac{C}{|\mathcal{A}| - 1}}$$

It has been shown that in Van Hasselt et al. [2016], overestimation drastically impacts the performance of DQN. Van Hasselt et al. [2016] proposed double DQN using the behavior Q-network to produce the greedy action a' for the value target $r(s, a) + \gamma \max_{a'} Q_{\bar{\phi}}(s, a')$. The resulting Q-value target turns to be:

$$y = r(s, a) + \gamma Q_{\bar{\phi}}(s, \arg \max_{a'} Q_\phi(s, a')).$$

Similar Q-values over actions Wang et al. [2016] shows that in Atari benchmarks, the Q-values for many actions are similar since a lot of actions have no effect on the environment. Thus, Wang et al. [2016] proposed to separately learn the value of a state s and the advantage of taking action a over the average value at the state s . The Q-network is parameterized as

$$Q_\phi(s, a) = V_\phi(s) + A_\phi(s, a),$$

where V_ϕ and A_ϕ denote the value of a state and the advantage of an action, respectively. Note that V_ϕ and A_ϕ only share the weights in parts of the layers in the network, but have different weights at their respective last layers (see Wang et al. [2016] for details).

Sampling strategy Schaul et al. [2015] shows that uniform sampling over the replay buffer $\mathcal{Z}$ is inefficient to update the Q-function. In Schaul et al. [2015], it is hypothesized that most of batches of gradient steps do not progress the Q-function learning due to temporal difference (TD) errors (i.e., $r(s, a) + \gamma \max_{a'} Q(s, a') - Q(s, a)$). As a result, Schaul et al. [2015] proposed to prioritize value updates on data with high TD errors. Each data (s, a, r, s') is then sampled according to the following probability distribution:

$$P(s, a, r, s') \propto |r(s, a) + \gamma \max_{a'} Q_\phi(s, a') - Q(s, a)|.$$

Such a sampling scheme can be viewed as “hard negative sampling” in machine learning literature.

Summary A plethora of variants of DQN have been proposed in the past few years. We are only able to cover the common techniques used in the recent works. Hessel et al. [2018] combined most of the variants and showed superior performance improvement over the original DQN.

9.2.4 Deep deterministic policy gradient

Function approximation enables Q-Learning to approximate the Q-values in a continuous state space. However, the continuous action space poses a computational challenge on the action selection and Q-value target estimation. Recall that the Q-value target is the follows:

$$r(s, a) + \gamma \max_{a'} Q_\phi(s, a').$$

It is easy to evaluate $\max_{a'} Q_\phi(s, a')$ by enumerating all actions a , while it is impossible to enumerate a' in a continuous action space. One option is evaluating $\max_{a'} Q_\phi(s, a')$ using an optimization algorithm (e.g., cross entropy method (CEM) or gradient ascent), while it is time-consuming. As such this option is not suitable for estimating the target values since we need to estimate the target values frequently during training. In addition to the difficulty of evaluating the Q-value targets, the ϵ -greedy exploration strategy is inapplicable in a continuous action space since it relies on the knowledge of greedy action $\arg \max_a Q(s, a)$, which also requires enumerating over the action space.

We introduce a commonly used algorithm, deep deterministic policy gradient (DDPG) [Lillicrap et al., 2015], below, and discuss how DDPG evaluate the value targets and select actions for exploration.

Q-value targets DDPG trains an actor (i.e., policy) π_θ to solve $\max_{a'} Q_\phi(s, a')$. The policy π_θ is parameterized as a neural network with weights θ . DDPG updates the policy using gradient descent, as shown below:

$$\theta \leftarrow \theta + \alpha \nabla_a Q_\phi(s, a)|_{a=\pi_\theta(s)}.$$

Taking the gradient w.r.t. actions $\nabla_a Q_\phi(s, a)|_{a=\pi_\theta(s)}$ produces the gradient w.r.t. θ due to chain rule. The value targets is turned into:

$$y = r(s, a) + \gamma Q_\phi(s', \pi_\theta(s')).$$

Note that as the policy is updated frequently over the training time, using such a policy to produce $Q_\phi(s', \pi_\theta(s'))$ causes noisy target Q-values. To mitigate this issue, Lillicrap et al. [2015] maintains a delayed copy of the policy as the target policy $\pi_{\bar{\theta}}$ to produce the value targets $Q_\phi(s', \pi_{\bar{\theta}}(s'))$. Similar to target networks, $\pi_{\bar{\theta}}$ will be updated to π_θ periodically. The difference is that DDPG uses a soft update to update the target value networks $Q_{\bar{\phi}}$ and the target policy $\pi_{\bar{\theta}}$. For timestep, both target networks are updated as the follows:

$$\begin{aligned}\bar{\phi} &\leftarrow (1 - \beta)\bar{\phi} + \beta\phi \\ \bar{\theta} &\leftarrow (1 - \beta)\bar{\theta} + \beta\theta.\end{aligned}$$

Exploration To explore non-greedy actions, DDPG adds noises to the predicted actions. The noises can be generated from an arbitrary distribution (e.g., normal distribution). The resulting action to execute during training becomes:

$$a \leftarrow a + \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, \sigma)$ denotes a normal distribution with variance σ . Note that the normal distribution can be replaced with any distribution (e.g., Ornstein-Uhlenbeck process [Lillicrap et al., 2015]).

In summary, the DDPG algorithm can be summarized as Algorithm

Algorithm 18 Deep deterministic policy gradient (DDPG)

```

1: procedure DDPG( $\alpha$ )
2:   Initialize the neural network weights  $\phi, \theta$  randomly
3:   Initialize the experience replay buffer  $\mathcal{Z} = \emptyset$ 
4:   while not terminate do
5:     Observe  $s$ , take action  $a \leftarrow \pi_\theta(s) + \epsilon$ , and receive reward  $r$  and next state  $s'$ 
6:     Append  $(s, a, r, s')$  to  $\mathcal{Z}$ 
7:     Sample a minibatch  $\mathcal{B} \sim \text{Uniform}(\mathcal{Z})$ 
8:     Compute the target with  $\bar{\phi}$ :  $y \leftarrow r(s, a) + \gamma Q_{\bar{\phi}}(s', \pi_{\bar{\theta}}(s')) \quad \forall (s, a, r, s') \in \mathcal{B}$ 
9:     Update the network  $\phi \leftarrow \phi + \alpha \nabla_\phi \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \|y - Q_\phi(s, a)\|^2$ 
10:     $\bar{\phi} \leftarrow (1 - \beta)\bar{\phi} + \beta\phi$ 
11:     $\bar{\theta} \leftarrow (1 - \beta)\bar{\theta} + \beta\theta$ 
12:  end while
13: end procedure

```

Advanced tricks Overestimation in the target Q-values is a crucial issue in DDPG. [Fujimoto et al., 2018] proposed several tricks to fix this issue and empirically showed superior performance improvements.

- **Clipped Double Q-values:** This technique is introduced to address the overestimation bias in target Q-values. Fujimoto et al. [2018] showed that simply applying double DQN technique to DDPG cannot eliminate overestimation bias. As a result, Fujimoto et al. [2018] proposed to train two Q-value networks, Q_{ϕ_1} and Q_{ϕ_2} and construct the target Q-values by the minimum estimates among Q_{ϕ_1} and Q_{ϕ_2} , as shown below:

$$r(s, a) + \gamma \min_{i=1,2} Q(s, \pi_{\theta_1}(s')).$$

One rationale that double DQN technique did not work in DDPG is that the target and the behavior Q-value networks are highly correlated (intuitively, producing similar values) since DDPG uses a soft update (see “Q-value targets” paragraph above) to update the target network. Correlated Q-value networks violates the required assumption in double DQN. Note that Q_{ϕ_1} and Q_{ϕ_2} are trained along with the respective target networks $Q_{\bar{\phi}_1}$ and $Q_{\bar{\phi}_2}$.

- **Delayed updates:** Fujimoto et al. [2018] showed that frequent policy updates incur overestimation on the target Q-value estimations. Thus, Fujimoto et al. [2018] suggested that updating the policy less frequently mitigates this issue.
- **Target policy smoothing:** Fujimoto et al. [2018] suggested that a deterministic policy can overfit to the target Q-value network, thus being susceptible to value approximation errors. To fix this issue Fujimoto et al. [2018] added noises to the action predicted by the policy. The target Q-value estimates turned out to be:

$$r(s, a) + \gamma \min_{i=1,2} Q(s, \pi_{\theta_1}(s') + \epsilon)$$

$$\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c),$$

where σ denotes the variance of the distribution and c is the limits of the noises.

9.2.5 Soft actor critic

Though DDPG makes Q-Learning tractable in continuous action spaces, DDPG is sensitive to hyperparameters. It is believed [Haarnoja et al., 2018] that the hyperparameter sensitivity issue results from the use of deterministic policy. Haarnoja et al. [2018] proposed an off-policy algorithm **Soft Actor Critic (SAC)**, attempting to mitigate this issue using maximum entropy principle. The key idea of SAC is augmenting the policy optimization objective via the entropy of the policy as the follows:

$$\theta^* = \arg \max_{\theta} \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) + \lambda \mathcal{H}(\pi_{\theta}(\cdot|s)) \right],$$

where the policy π_{θ} is a stochastic policy (i.e., a state-conditional probability distribution actions), λ is a scaling factor controlling the importance of entropy, and $\mathcal{H}(\pi_{\theta}(\cdot|s))$ denotes entropy of the action distribution output by $\pi_{\theta}(\cdot|s)$. As a disclaimer before we present the details of SAC, there is no rigorous analysis or argument saying that such a maximum entropy rewards can address the hyperparameter sensitivity of DDPG, while Haarnoja et al. [2018] empirically showed that SAC works better than DDPG. Acute readers might already notice the connection between SAC and TD3. We will compare them at the end of this section.

Q-value function To optimize the policy with maximum entropy, SAC defines the “soft” value functions as below:

$$Q(s, a) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) + \lambda \sum_{t=2}^{\infty} \mathcal{H}(\pi_{\theta}(\cdot|s_t)) | s_1 = s, a_1 = a \right]$$

(Notice that as a_1 is determined, there is no entropy term at $t = 1$.)

$$\begin{aligned} V(s, a) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) + \lambda \sum_{t=1}^{\infty} \mathcal{H}(\pi_{\theta}(\cdot|s_t)) | s_1 = s \right] \\ &= Q(s, a) + \lambda \mathcal{H}(\pi_{\theta}(\cdot|s)). \end{aligned}$$

Based on these definitions, we can derive the Q-value targets using bellmen equation as the follows:

$$\begin{aligned} Q(s, a) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[r(s_1, a_1) + \sum_{t=2}^{\infty} \gamma^{t-1} r(s_t, a_t) + \lambda \sum_{t=2}^{\infty} \mathcal{H}(\pi_{\theta}(\cdot|s_t)) | s_1 = s, a_1 = a \right] \\ &= r(s_1, a_1) + \gamma \mathbb{E}_{s' \sim \mathcal{T}, a' \sim \pi} [V(s')] \\ &= r(s_1, a_1) + \gamma \mathbb{E}_{s' \sim \mathcal{T}, a' \sim \pi} [Q(s', a')] + \lambda \mathbb{E}_{s' \sim \mathcal{T}} [\mathcal{H}(\pi_{\theta}(\cdot|s'))] \\ &= r(s_1, a_1) + \gamma \mathbb{E}_{s' \sim \mathcal{T}, a' \sim \pi} [Q(s', a')] - \lambda \mathbb{E}_{s' \sim \mathcal{T}, a' \sim \pi_{\theta}} [\log(\pi_{\theta}(a'|s'))] \\ &= r(s_1, a_1) + \gamma \mathbb{E}_{s' \sim \mathcal{T}, a' \sim \pi} [Q(s', a') - \lambda \log(\pi_{\theta}(a'|s'))]. \end{aligned}$$

As the expectation term $\mathbb{E}_{s' \sim \mathcal{T}, a' \sim \pi} [Q(s', a') - \lambda \log(\pi_{\theta}(a'|s'))]$ can be approximated by sampling¹³, one can train a Q-value network Q_{ϕ} by gradient descent as show below:

$$\phi \leftarrow \phi + \alpha \nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} \|y - Q_{\phi}(s, a)\|^2,$$

where the Q-value target y is defined as:

$$y = r(s, a) + \gamma Q(s', a') - \lambda \log(\pi_{\theta}(a'|s')), \quad a' \sim \pi_{\theta}(\cdot|s').$$

¹³SAC paper made this claim, while I think this requires careful justification.

Why is SAC off-policy?

How is it different from the entropy regularizer in A2C?

10 Offline reinforcement learning

Reinforcement learning algorithms discussed in the above sections require collecting training data from interaction with the environment. This is undesirable in the real world since interaction with the real environment can be unsafe and time-consuming. Robots, for instance, could be damaged during interaction with physical world. Besides,

10.1 Overcoming distributional shift in offline RL

10.2 Conservative Q-Learning

11 Multi-goal Reinforcement Learning

11.1 Goal-conditioned value function

- HER - BVN

12 Hierarchical Reinforcement Learning

In the previous section, we introduced multi-goal reinforcement learning where the objective of the policy is to reach the goal state or achieve the goal. When achieving the goal requires a large number of steps, the exploration and the credit assignment is becoming difficult. For example, in Figure **TODO**, the objective is to navigate the white dot to the goal state (yellow dot). Since two dots are far away from each other, it is extremely challenging to find the successful path to reach the goal during exploration. Even though a successful path is found, it can be still difficult to train the policy due to credit assignment (see variance reduction in policy gradient section).

Hierarchical reinforcement learning (HRL) is purposed to address the challenges of exploration and credit assignment in long-horizon tasks. The spirit of HRL is central at *divide-and-conquer*. HRL breaks down a long-horizon task into several pieces, where each piece is a subtask solved by the low-level policy.

— Long horizon goal is hard to achieve, we can break down it into several subgoals and use a goal-conditioned low-level policy to achieve them, since subgoal is easier to achieve. Shorter horizon makes credit assignment easier, easier explore, easier transfer.

This is the prominent idea of HRL. The question is how to find subgoals? From high-level policies. How to define rewards for high-level policies? Reward from env? Reward as distance

to goal.

Example: Feudal RL, multiple levels, Feudal deep RL: Manager module predicting goals,

Practical: Hi planner, lo skills

Issues: non-stationarity, hi suffer from bad lo pi, lo pi suffer from bad hi goal

Relay policy learning: Pretrain both, finetune together

Option critic:

13 Curriculum learning

The order of presenting the task matters.

Example: Yoshua bengio curriculum learning. See frequent words and then less frequent words.

Teacher-student curriculum learning

locomotion train in simulation

block stacking

References

- Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. 2020.
- Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *ICML*, 2004.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 2015.
- Oriol Vinyals, Igor Babuschkin, Junyoung Chung, Michael Mathieu, Max Jaderberg, Wojciech M Czarnecki, Andrew Dudzik, Aja Huang, Petko Georgiev, Richard Powell, et al. Alphastar: Mastering the real-time strategy game starcraft ii. *DeepMind blog*, page 2, 2019.
- Pulkit Agrawal. *Computational Sensorimotor Learning*. PhD thesis, 2018. URL <https://www.proquest.com/dissertations-theses/computational-sensorimotor-learning/docview/2139675360/se-2?accountid=12492>.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 2002.
- Lihong Li, Wei Chu, John Langford, and Robert E Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, pages 661–670, 2010.
- Donald W Marquardt and Ronald D Snee. Ridge regression in practice. *The American Statistician*, 29(1):3–20, 1975.
- Thomas J Walsh, István Szita, Carlos Diuk, and Michael L Littman. Exploring compact reinforcement-learning representations with linear regression. *arXiv preprint arXiv:1205.2606*, 2012.
- Daishi Harada. Reinforcement learning with time. In *Conference on Artificial Intelligence*, 1997.
- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.
- Gerald Tesauro. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- Vijay Konda and John Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy P Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *ICML*, 2016.

- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *International Conference of Representation Learning*, 2015a.
- Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in neural information processing systems*, 24, 2011.
- Dimitris Bertsimas and John N Tsitsiklis. *Introduction to linear optimization*, volume 6. Athena Scientific Belmont, MA, 1997.
- Sham M Kakade. A natural policy gradient. *Advances in neural information processing systems*, 14, 2001.
- Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural computation*, 10(2): 251–276, 1998.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML-15)*, pages 1889–1897, 2015b.
- Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *In Proc. 19th International Conference on Machine Learning*. Citeseer, 2002.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012.
- Yan Duan, Xi Chen, Rein Houthoofd, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. In *International conference on machine learning*, pages 1329–1338. PMLR, 2016.
- Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on ppo and trpo. *arXiv preprint arXiv:2005.12729*, 2020.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. What matters in on-policy reinforcement learning? a large-scale empirical study. *arXiv preprint arXiv:2006.05990*, 2020a.
- Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Icml*, volume 99, pages 278–287, 1999.
- A Gasparetto and L Scalera. From the unimate to the delta robot: the early decades of industrial robotics. In *Explorations in the history and heritage of machines and mechanisms*, pages 284–295. 2019.

- Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 661–668. JMLR Workshop and Conference Proceedings, 2010.
- Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.
- Pete Florence, Corey Lynch, Andy Zeng, Oscar A Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning*, pages 158–168. PMLR, 2022.
- Anurag Ajay, Aviral Kumar, Pulkit Agrawal, Sergey Levine, and Ofir Nachum. Opal: Offline primitive discovery for accelerating offline reinforcement learning. *arXiv preprint arXiv:2010.13611*, 2020.
- Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.
- Chuan Wen, Jierui Lin, Jianing Qian, Yang Gao, and Dinesh Jayaraman. Keyframe-focused visual imitation learning. 2021.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530*, 2016.
- Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Madry. Adversarial examples are not bugs, they are features. In *Advances in Neural Information Processing Systems*, pages 125–136, 2019.
- Aaron Van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv e-prints*, pages arXiv–1807, 2018.
- Anurag Ajay and Pulkit Agrawal. Learning action prior for visuomotor transfer. In *ICML Inductive Biases, Invariances and Generalization in RL workshop*, 2020.
- Andrew Y Ng and Stuart J Russell. Algorithms for inverse reinforcement learning. In *ICML*, pages 663–670, 2000.
- Stuart Russell. Learning agents for uncertain environments. In *Proceedings of the eleventh annual conference on Computational learning theory*, pages 101–103, 1998.
- Brian D. Ziebart, Andrew Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. In *AAAI*, 2008.
- Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *NIPS*, 2016.

- OpenAI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020b.
- Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *Intelligent Robots and Systems (IROS), 2017 IEEE/RSJ International Conference on*, pages 23–30. IEEE, 2017.
- Nataniel Ruiz, Samuel Schuler, and Manmohan Chandraker. Learning to simulate. *arXiv preprint arXiv:1810.02513*, 2018.
- Roberta Raileanu, Maxwell Goldstein, Denis Yarats, Ilya Kostrikov, and Rob Fergus. Automatic data augmentation for generalization in reinforcement learning. *Advances in Neural Information Processing Systems*, 34, 2021.
- Stephen James, Paul Wohlhart, Mrinal Kalakrishnan, Dmitry Kalashnikov, Alex Irpan, Julian Ibarz, Sergey Levine, Raia Hadsell, and Konstantinos Bousmalis. Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12627–12637, 2019.
- Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. 2018.
- Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.
- George Konidaris, Sarah Osentoski, and Philip Thomas. Value function approximation in reinforcement learning using the fourier basis. In *Twenty-fifth AAAI conference on artificial intelligence*, 2011.
- Martin Riedmiller. Neural fitted q iteration—first experiences with a data efficient neural reinforcement learning method. In *European conference on machine learning*, pages 317–328. Springer, 2005.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8(3):293–321, 1992.
- Hado Hasselt. Double q-learning. *Advances in neural information processing systems*, 23, 2010.
- Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Thirty-second AAAI conference on artificial intelligence*, 2018.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *arXiv preprint arXiv:1802.09477*, 2018.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.

A Supervised Learning v.s. Reinforcement Learning

Supervised learning (SL) and reinforcement learning (RL) share similar traits in their optimization objectives

$$\begin{aligned} \mathbb{E}_{\tau^{gt}} [\nabla_{\theta} \log p_{\theta}(\tau^{gt})] \quad (\text{SL}) \\ \mathbb{E}_{\tau} [\nabla_{\theta} \log p_{\theta}(\tau) R^{\gamma}(\tau)] \quad (\text{RL}) \end{aligned}$$

where $\tau^{gt} := (s_1, a_1^{gt}, s_2, a_2^{gt}, \dots)$. Nevertheless, the distribution of the expectation taken from results in interesting difference in the KL divergence interpretations

$$\begin{aligned} \min_{\theta} D_{KL}(P||Q_{\theta}) &\implies \nabla_{\theta} \mathbb{E}_{x \sim P(x)} \left[\log \frac{P(x)}{Q_{\theta}(x)} \right] = \mathbb{E}_{x \sim P(x)} [\nabla_{\theta} \log Q_{\theta}(x)] \quad (\text{Forward KL}) \\ \min_{\theta} D_{KL}(Q_{\theta}||P) &\implies \mathbb{E}_{x \sim Q_{\theta}(x)} \left[\log \frac{Q_{\theta}(x)}{P(x)} \right] \quad (\text{Reverse KL}) \\ &\implies -\mathbb{E}_{x \sim Q_{\theta}(x)} [\log P(x)] + \mathbb{E}_{x \sim Q_{\theta}(x)} [\log Q_{\theta}(x)] \\ &\implies \max_{\theta} \mathbb{E}_{x \sim Q_{\theta}(x)} [\log P(x)] + \mathcal{H}(Q_{\theta}(x)) \\ &\implies \max_{\theta} \mathbb{E}_{\tau \sim Q_{\theta}(x)} [R^{\gamma}(\tau)] + \mathcal{H}(Q_{\theta}(x)) \quad (P(x) \sim e^{R^{\gamma}(\tau)}). \end{aligned}$$

As we see, SL is minimizing forward KL divergence while RL is minimizing reverse KL divergence. Notably, the reverse KL interpretation aligns with **Maximum Entropy RL** [Haarnoja et al., 2018] objective.

B Second-order KL-divergence Approximation

Let random variable $x = (a|s)$. By second-order Taylor expansion, we have:

$$D_{KL}(\pi_{\theta}(x)||\pi_{\theta'}(x)) \approx D_{KL}(\pi_{\theta}(x)||\pi_{\theta}(x)) + d^T \nabla_{\theta'} D_{KL}(\pi_{\theta}(x)||\pi_{\theta'}(x)) + \frac{1}{2} d^T \nabla_{\theta'}^2 D_{KL}(\pi_{\theta}(x)||\pi_{\theta'}(x)) d, \quad (43)$$

where the first-order term is

$$\nabla_{\theta'} D_{KL}(\pi_{\theta}(x) || \pi_{\theta'}(x)) = \nabla_{\theta'} \mathbb{E}_{x \sim \pi_{\theta}} [\log \pi_{\theta}(x)] - \nabla_{\theta'} \mathbb{E}_{x \sim \pi_{\theta}} [\log \pi_{\theta'}(x)] \quad (44)$$

$$= 0 - \nabla_{\theta'} \mathbb{E}_{x \sim \pi_{\theta}} [\log \pi_{\theta'}(x)] \quad (\mathbb{E}_{x \sim \pi_{\theta}} [\log \pi_{\theta'}(x)] = 1) \quad (45)$$

$$= -\mathbb{E}_{x \sim \pi_{\theta}} [\nabla_{\theta'} \log \pi_{\theta'}(x)], \quad (46)$$

and the second-order term is

$$\nabla_{\theta'}^2 D_{KL}(\pi_{\theta}(x) || \pi_{\theta'}(x)) = -\mathbb{E}_{x \sim \pi_{\theta}} [\nabla_{\theta'}^2 \log \pi_{\theta'}(x)] \quad (47)$$

$$= \mathbf{F}(\theta) \quad (\text{Fisher information matrix}). \quad (48)$$

Plugging $\theta' = \theta + d$, we have

$$D_{KL}(\pi_{\theta}(x) || \pi_{\theta+d}(x)) \quad (49)$$

$$\approx D_{KL}(\pi_{\theta}(x) || \pi_{\theta}(x)) + d^T \nabla_{\theta'} D_{KL}(\pi_{\theta}(x) || \pi_{\theta'}(x))|_{\theta'=\theta} + \frac{1}{2} d^T \nabla_{\theta'}^2 D_{KL}(\pi_{\theta}(x) || \pi_{\theta'}(x))|_{\theta'=\theta} d \quad (50)$$

$$= \frac{1}{2} d^T \nabla_{\theta'}^2 D_{KL}(\pi_{\theta}(x) || \pi_{\theta'}(x))|_{\theta'=\theta} d \quad (51)$$

$$= \frac{1}{2} d^T \mathbf{F}(\theta) d \quad (52)$$

C Behavior cloning derivation

Let the probability of a trajectory τ be:

$$\begin{aligned} P(\tau) &= P(s_1, a_1, \dots, s_T) \\ &= P(s_1) P(a_1 | s_1) \cdots P(a_{T-1} | s_1, a_1, \dots, s_{T-1}) P(s_T | s_1, a_1, \dots, a_{T-1}) \quad (\text{By Bayes rule}) \\ &= P(s_1) \prod_{t=1}^T P(s_t | s_1, a_1, \dots, a_{t-1}) P(a_t | s_1, a_1, \dots, s_t) \\ &= P(s_1) \prod_{t=1}^T P(s_t | s_{t-1}, a_{t-1}) P(a_t | s_t) \quad (\text{Assume Markov property}) \\ &= \rho(s_1) \prod_{t=1}^T \mathcal{T}(s_t | s_{t-1}, a_{t-1}) \pi(a_t | s_t). \end{aligned}$$

We write the expression of $\nabla J(\theta)$ as:

$$\begin{aligned}
& \nabla J(\theta) \\
&= \nabla D_{KL}(P_D || P_\theta) \\
&= \nabla \left(\sum_{\tau} P_D(\tau) \log \frac{P_D(\tau)}{P_\theta(\tau)} \right) \\
&= \nabla \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) \log \frac{P_D(s_1, a_1, \dots, s_T)}{P_\theta(s_1, a_1, \dots, s_T)} \right) \\
&= \nabla \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) (\log P_D(s_1, a_1, \dots, s_T) - \log P_\theta(s_1, a_1, \dots, s_T)) \right) \\
&= -\nabla \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) \log P_\theta(s_1, a_1, \dots, s_T) \right) \\
&= -\nabla \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) (\log \rho(s_1) \prod_{t=1}^T \mathcal{T}(s_t | s_{t-1}, a_{t-1}) \pi_\theta(a_t | s_t)) \right) \\
&= -\nabla \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) (\log \rho(s_1) \prod_{t=1}^T \mathcal{T}(s_t | s_{t-1}, a_{t-1}) \pi_\theta(a_t | s_t)) \right) \\
&= -\nabla \left(\sum_{\tau} P_D(s_1, a_1, \dots, s_T) (\log \rho(s_1) + \sum_{t=1}^T \log \mathcal{T}(s_t | s_{t-1}, a_{t-1}) + \sum_{t=1}^T \log \pi_\theta(a_t | s_t)) \right) \\
&= -\sum_{\tau} P_D(s_1, a_1, \dots, s_T) \sum_{t=1}^T \nabla \log \pi_\theta(a_t | s_t) \\
&= -\sum_{t=1}^T \sum_{\tau} P_D(s_1, a_1, \dots, s_T) P_D(a_t | s_t) \nabla \log \pi_\theta(a_t | s_t) \\
&= -\sum_{t=1}^T \sum_{\tau} P_D(s_1, a_1, \dots, s_T | s_t, a_t) P_D(s_t = s, a_t = a) \nabla \log \pi_\theta(a_t = a | s_t = s) \\
&= -\sum_{t=1}^T \sum_{s, a \in \mathcal{S} \times \mathcal{A}} P_D(s_t = a, a_t = a) \sum_{\tau \setminus (s_t, a_t)} P_D(s_1, a_1, \dots, s_T | s_t, a_t) \nabla \log \pi_\theta(a_t = a | s_t = a) \\
&= -\sum_{t=1}^T \sum_{s, a \in \mathcal{S} \times \mathcal{A}} P_D(s_t = s, a_t = a) \nabla \log \pi_\theta(a_t = a | s_t = s) \underbrace{\left[\sum_{\tau \setminus (s_t, a_t)} P_D(s_1, a_1, \dots, s_T | s_t, a_t) \right]}_1 \\
&= -\sum_{t=1}^T \sum_{s, a \in \mathcal{S} \times \mathcal{A}} P_D(s_t = s, a_t = a) \nabla \log \pi_\theta(a_t = a | s_t = s) \\
&= -\mathbb{E}_{s, a \sim D} [\nabla \log \pi_\theta(a | s)]
\end{aligned}$$

D Policy Parameterization

Continuous actions A continuous action space is defined as $\mathbb{R}^d$, where d denotes the dimension of the action space. For a continuous action space, in practice, BC makes an assumption that the expert's policy π_D is a Gaussian distribution, and approximates the

mean of π_D by π_θ . With this assumption, the MLE objective can be rewritten as:

$$\begin{aligned}
 J(\theta) &= \mathbb{E}_{s,a \sim \mathcal{D}} [\log \pi_\theta(\hat{a} = a|s)] \\
 &= \mathbb{E}_{s,a \sim \mathcal{D}} \left[\log \left(\frac{1}{\sigma\sqrt{2\pi}} \exp\left(\frac{-(\pi_\theta(s) - a)^2}{2\sigma^2}\right) \right) \right] \\
 &= \mathbb{E}_{s,a \sim \mathcal{D}} \left[-\log(\sigma\sqrt{2\pi}) - \log \left(\exp\left(\frac{-(\pi_\theta(s) - a)^2}{2\sigma^2}\right) \right) \right] \\
 &= \mathbb{E}_{s,a \sim \mathcal{D}} [-\log(\sigma\sqrt{2\pi})] - \mathbb{E}_{s,a \sim \mathcal{D}} \left[\log \left(\exp\left(\frac{-(\pi_\theta(s) - a)^2}{2\sigma^2}\right) \right) \right] \\
 &= \mathbb{E}_{s,a \sim \mathcal{D}} [-\log(\sigma\sqrt{2\pi})] - \mathbb{E}_{s,a \sim \mathcal{D}} \left[\frac{-(\pi_\theta(s) - a)^2}{2\sigma^2} \right] \\
 &= \mathbb{E}_{s,a \sim \mathcal{D}} [-\log(\sigma\sqrt{2\pi})] - \frac{1}{2\sigma^2} \mathbb{E}_{s,a \sim \mathcal{D}} [-(\pi_\theta(s) - a)^2],
 \end{aligned}$$

where note that π denotes the ratio of a circle's circumference to its diameter, and σ denotes a constant standard deviation. Since except for $-(\pi_\theta(s) - a)^2$ the remaining terms are constants, we can rewrite the optimization problem as:

$$\begin{aligned}
 &\max_{\theta} J(\theta) \\
 &= \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} [-(\pi_\theta(s) - a)^2] \\
 &= \min_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} [(\pi_\theta(s) - a)^2].
 \end{aligned}$$

It can be seen that training the policy π_θ with BC is equivalent to minimizing the mean squared errors between the $\pi_\theta(s)$ and the expert's action $a \sim \pi_D(s)$.

E Consideration of Markov assumption

Markov assumption can be stated formally as:

Definition E.1 (Markov assumption). A probability distribution P satisfies Markov property if and only if

$$P(x_t|x_{t-1}) = P(x_t|x_{t-1}, \dots, x_1).$$

Recall in Section 4 and Section 6.1, we mentioned that Markov assumption is made in practice. The reason behind making this assumption is *data sparsity*. Consider a policy π not satisfying Markov property and a policy π_M satisfying Markov property:

$$\begin{aligned}
 &\pi(a|s_t, \dots s_1) \quad (\text{Non-markovian policy}) \\
 &\pi_M(a|s_t) \quad (\text{Markovian policy})
 \end{aligned}$$

It can be seen that for a non-markovian policy π , we need a sequence of states $(s_t, \dots s_1)$ to compute the policy gradient $\log \pi(a = a_t|s_t, \dots s_1)$, while a markovian policy simply needs a state s_t . From machine learning aspect, the training data of a non-markovian policy π is sparser than the markovian policy π_M since the occurrence of a state sequence $(s_t, \dots, s_1)$ is lower than a single state s_t . For example, let's consider an environment with three states *red*, *green*, and *blue* and three sequences $(red, red, green)$, $(blue, red, blue)$, and $(blue, green, green)$. The occurrence of *red*, *green*, and *blue* are all 3 while the occurrence of each sequence of states is all 1.

Despite being more data efficient, Markov assumption could require additional state space engineering. Let's consider an example of training a cheetah robot to run as fast as possible, and the following two state space designs:

1. **Joint positions:** $s = [x]$
2. **Joint positions, velocities, and accelerations:** $s = [x, \dot{x}, \ddot{x}]$,

where x denotes the joint positions, and $\dot{x}$ and $\ddot{x}$ are velocities and accelerations. If the task requires velocities, the markovian policy π_M using the state space of joint positions is unlikely to succeed due to lack of velocities in the state inputs. Though the second state space can enable the markovian policy π_M to account velocities and acceleration, it might require careful state space design. In contrast, a non-markovian policy π is able to recover velocities and accelerations from a sequence of positions $(x_1, \dots, x_T)$.

F Consideration of action space design

In video game playing like Atari [Mnih et al. \[2015\]](#), the action space is usually designed as a discrete set (e.g., *move left*, *move right*, *move up*, *move down*). For a locomotion or manipulation tasks in robotics, the action space is typically designed as a continuous space (e.g., motor commands). It is natural to use these action spaces for the above two examples since the gamepad of a video game is usually discrete (maybe include one dimension of analog control) and the motor commands in robotics are often continuous values.

For a task that does not have obvious characteristics on actions, how do we decide the action space for a task? Consider training an agent to invest stock. One design can be enumerating $(buy, sell, hold) \times (Stock A, Stock B, Stock C, \dots)$, while the action space can grow rapidly with the number of stocks.

To circumvent this dimensionality issue, one alternative is to move stock ID to the state input s . The policy π simply requires predicting three actions (*buy*, *sell*, *hold*) for each stock s . This design reduces the dimension of the action space while one requires iterating over all stocks to make action predictions to know the best action to do. A workaround is featurizing each stock by their attributes. Representing the stock (i.e., state s) as a feature vector by a featurization function could remove the needs of enumerating all stocks.